

MI SYS API

Version 3.9

© 2022 SigmaStar Technology Corp. All rights reserved.

SigmaStar Technology makes no representations or warranties including, for example but not limited to, warranties of merchantability, fitness for a particular purpose, non-infringement of any intellectual property right or the accuracy or completeness of this document, and reserves the right to make changes without further notice to any products herein to improve reliability, function or design. No responsibility is assumed by SigmaStar Technology arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights, nor the rights of others.

SigmaStar is a trademark of SigmaStar Technology Corp. Other trademarks or names herein are only for identification purposes only and owned by their respective owners.

REVISION HISTORY

Revision No.	Description	Date
3.0	Initial release	12/05/2020
3.1	- 添加新的模块 ID E_MI_MODULE_ID_WBC	01/06/2021
3.2	Add E_MI_SYS_PIXEL_FRAME_YUV422_PLANAR and E_MI_SYS_PIXEL_FRAME_YUV420_PLANAR format in MI_SYS_PixelFormat_e	04/21/2021
3.3	- Add new moudles MI_SYS_ChnlInputPortSetUserPicture MI_SYS_EnableUserPicture MI_SYS_DisableUserPicture MI_SYS_UserPictureInfo_t	04/27/2021
3.4	- Delete MI_SYS internal structure hierarchy introduction and Pass introduction - Delete modules MI_SYS_SetGlobalFlag MI_SYS_GlobalFlagType_e MI_SYS_GlobalFlagParam_t - Added insmod parameter description - Modify "Idr handle" to "handle" - Modify modules MI_SYS_BindChnlPort2 MI_SYS_SetChnlOutputPortDepth MI_SYS_SetChnlMMAConf MI_SYS_BindType_e	06/02/2021
3.5	Modify MI_SYS_MMA_Free	06/29/2021
3.6	- Add XVR flow diagram - Add FRC user guide - Add New Frame tile mode	07/22/2021
	Added PROCFS INTRODUCTION	08/25/2021
3.7	- Add bCrcCheck in MI_SYS_BufConf_t and MI_SYS_BufInfo_t - Add u16BufCompressAlignment and u16BufExtraSize in MI_SYS_FrameBufExtraConfig_t	11/05/2021
3.8	Add new pixel format E_MI_SYS_PIXEL_FRAME_RGB888_PLANAR	11/26/2021
3.9	- Add new moudles MI_SYS_SetChnlOutputPortUserFrc	11/30/2021
	Modify 1.6 Insmod Parameter List	12/08/2021

Revision No.	Description	Date
3.0	Initial release	12/05/2020
	- Modify 5.20 mma_heap_name0 - Modify 2.5.14 MI_SYS_PrivateDevChnHeapFree	
3.10	- Add new private pool config MI_SYS_PerDevPrivRingPoolConf_t - Modify memory alignment requirements MI_SYS_Mmap MI_SYS_FlushInvCache - Modify 2.4.2 MI_SYS_BindChnPort2 - Modify 3.2.4 MI_SYS_FrameTileMode_e - Modify 3.2.23 MI_SYS_InsidePrivatePoolType_e - Modify 3.2.29 MI_SYS_GlobalPrivPoolConfig_t	03/30/2022

TABLE OF CONTENTS

REVISION HISTORY.....	i
TABLE OF CONTENTS.....	iii
1. 概述.....	1
1.1. 模块说明.....	1
1.2. 文档格式约束.....	1
1.3. 关键字说明.....	2
1.4. 流程框图.....	3
1.4.1 Dev/Chn/Port 的关系.....	3
1.4.2 一个典型的 NVR 数据流.....	5
1.4.3 一个典型的 IPC 数据流.....	6
1.4.4 一个典型的 DVR 数据流.....	7
1.4.5 一个典型的 XVR 数据流.....	8
1.5. Bind 和帧率控制场景说明.....	8
1.5.1 一对一 Bind.....	9
1.5.2 一对多 Bind.....	10
1.5.3 Bind 的同时设置了 UserDepth.....	11
1.5.4 NVR 场景下 Vdec 帧率控制的补充.....	12
1.6. insmod 参数说明.....	13
2. API 参考.....	14
2.1. API 描述格式说明.....	14
2.2. 功能模块 API 列表.....	14
2.3. 系统功能类 API.....	17
2.3.1 MI_SYS_Init.....	17
2.3.2 MI_SYS_Exit.....	19
2.3.3 MI_SYS_GetVersion.....	20
2.3.4 MI_SYS_GetCurPts.....	21
2.3.5 MI_SYS_InitPtsBase.....	22
2.3.6 MI_SYS_SyncPts.....	22
2.3.7 MI_SYS_SetReg.....	23
2.3.8 MI_SYS_GetReg.....	24
2.3.9 MI_SYS_ReadUuid.....	25
2.3.10 MI_SYS_EnableChnOutputPortLowLatency.....	26
2.4. 数据流类 API.....	28
2.4.1 MI_SYS_BindChnPort.....	28
2.4.2 MI_SYS_BindChnPort2.....	29
2.4.3 MI_SYS_UnBind_ChnPort.....	33
2.4.4 MI_SYS_GetBindbyDest.....	34
2.4.5 MI_SYS_ChnInputPortGetBuf.....	35
2.4.6 MI_SYS_ChnInputPortPutBuf.....	37
2.4.7 MI_SYS_ChnOutputPortGetBuf.....	38

2.4.8	MI_SYS_ChnOutputPortPutBuf.....	44
2.4.9	MI_SYS_ChnInputPortGetBufPa.....	45
2.4.10	MI_SYS_ChnInputPortPutBufPa.....	47
2.4.11	MI_SYS_ChnOutputPortGetBufPa.....	49
2.4.12	MI_SYS_ChnOutputPortPutBufPa.....	52
2.4.13	MI_SYS_SetChnOutputPortDepth.....	53
2.4.14	MI_SYS_ChnPortInjectBuf.....	55
2.4.15	MI_SYS_GetFd.....	56
2.4.16	MI_SYS_CloseFd.....	57
2.4.17	MI_SYS_DupBuf.....	58
2.4.18	MI_SYS_ChnInputPortSetUserPicture.....	61
2.4.19	MI_SYS_EnableUserPicture.....	63
2.4.20	MI_SYS_DisableUserPicture.....	64
2.4.21	MI_SYS_SetChnOutputPortUserFrc.....	65
2.5.	内存管理类 API.....	66
2.5.1	MI_SYS_SetChnMMAConf.....	66
2.5.2	MI_SYS_GetChnMMAConf.....	67
2.5.3	MI_SYS_MMA_Alloc.....	68
2.5.4	MI_SYS_MMA_Free.....	70
2.5.5	MI_SYS_Mmap.....	71
2.5.6	MI_SYS_FlushInvCache.....	72
2.5.7	MI_SYS_Munmap.....	73
2.5.8	MI_SYS_MemsetPa.....	74
2.5.9	MI_SYS_MemcpyPa.....	76
2.5.10	MI_SYS_BufFillPa.....	77
2.5.11	MI_SYS_BufBlitPa.....	79
2.5.12	MI_SYS_ConfigPrivateMMAPool.....	81
2.5.13	MI_SYS_PrivateDevChnHeapAlloc.....	84
2.5.14	MI_SYS_PrivateDevChnHeapFree.....	87
2.5.15	MI_SYS_Va2Pa.....	88
3.	数据类型.....	90
3.1.	数据结构描述格式说明.....	90
3.2.	数据结构列表.....	90
3.2.1	MI_ModuleId_e.....	92
3.2.2	MI_SYS_PixelFormat_e.....	95
3.2.3	MI_SYS_CompressMode_e.....	98
3.2.4	MI_SYS_FrameTileMode_e.....	100
3.2.5	MI_SYS_FieldType_e.....	101
3.2.6	MI_SYS_BufDataType_e.....	102
3.2.7	MI_SYS_FrameIspInfoType_e.....	103
3.2.8	MI_SYS_ChnPort_t.....	104

3.2.9	MI_SYS_MetaData_t.....	104
3.2.10	MI_SYS_RawData_t.....	105
3.2.11	MI_SYS_WindowRect_t.....	106
3.2.12	MI_SYS_FrameData_t.....	107
3.2.13	MI_SYS_BufInfo_t.....	109
3.2.14	MI_SYS_FrameBufExtraConfig_t.....	111
3.2.15	MI_SYS_BufFrameConfig_t.....	112
3.2.16	MI_SYS_BufRawConfig_t.....	113
3.2.17	MI_SYS_MetaDataConfig_t.....	114
3.2.18	MI_SYS_BufConf_t.....	114
3.2.19	MI_SYS_Version_t.....	116
3.2.20	MI_VB_PoolListConf_t.....	116
3.2.21	MI_SYS_BindType_e.....	117
3.2.22	MI_SYS_FrameData_PhySignalType.....	118
3.2.23	MI_SYS_InsidePrivatePoolType_e.....	119
3.2.24	MI_SYS_PerChnPrivHeapConf_t.....	120
3.2.25	MI_SYS_PerDevPrivHeapConf_t.....	121
3.2.26	MI_SYS_PerVpe2VencRingPoolConf_t.....	122
3.2.27	MI_SYS_PerChnPortOutputPool_t.....	123
3.2.28	MI_SYS_PerDevPrivRingPoolConf_t.....	124
3.2.29	MI_SYS_GlobalPrivPoolConfig_t.....	125
3.2.30	MI_SYS_FrameIspInfo_t.....	126
3.2.31	MI_SYS_FbcData_t.....	126
3.2.32	MI_SYS_BufFrameMultiPlaneConfig_t.....	129
3.2.33	MI_SYS_FrameDataSubPlane_t.....	129
3.2.34	MI_SYS_BufFrameMetaConfig_t.....	131
3.2.35	MI_SYS_FrameDataMultiPlane_t.....	132
3.2.36	MI_SYS_UserPictureInfo_t.....	132
4.	错误码.....	134
4.1.	错误码的组成.....	134
4.2.	MI_SYS API 错误码表.....	134
5.	PROCFS 介绍.....	136
5.1.	common.....	136
5.1.1	dump_mmap.....	136
5.1.2	dump_config.....	137
5.2.	mi_log_info.....	143
5.2.1	cat.....	143
5.2.2	echo.....	145
5.3.	mi_global_info.....	146
5.3.1	cat.....	146
5.4.	mi_bufqueue_status.....	148
5.4.1	cat.....	148

5.5.	mi_dump_buffer_delay_worker.....	150
5.5.1	cat.....	150
5.5.2	echo.....	151
5.6.	mi_infer_graph.py.....	151
5.6.1	cat.....	151
5.7.	mi_graph_contrl.....	152
5.7.1	cat.....	152
5.7.2	echo.....	153
5.8.	debug_level.....	153
5.8.1	cat/echo.....	153
5.9.	debug_file.....	154
5.9.1	echo.....	154
5.10.	debug_func.....	155
5.10.1	echo.....	155
5.11.	module_version_file.....	156
5.11.1	cat.....	156
5.12.	show threads.....	157
5.12.1	echo.....	157
5.13.	debug_frc.....	158
5.13.1	echo.....	158
5.14.	set_outputport_depth.....	159
5.14.1	echo.....	159
5.15.	set_thread_priority.....	160
5.15.1	echo.....	160
5.16.	mmu debug.....	160
5.16.1	echo.....	160
5.16.2	insmod.....	161
5.17.	set_inputport_pattern.....	161
5.17.1	echo.....	161
5.18.	mi_sys0.....	162
5.18.1	cat.....	162
5.19.	miu_protect.....	164
5.19.1	cat.....	164
5.20.	mma_heap_name0.....	167
5.20.1	cat.....	167
5.20.2	echo.....	173
5.21.	imi_mma_heap.....	173
5.21.1	cat.....	173
5.22.	meta.....	175
5.22.1	cat.....	175
5.22.2	echo.....	176
5.23.	mi_modulename devid.....	177

5.23.1	cat.....	177
5.23.2	echo.....	184
5.24.	mem_stat.....	185
5.24.1	cat.....	185

1. 概述

1.1. 模块说明

MI_SYS 是整个 MI 系统的基础模块，它给其他 MI 模块的运行提供了基础。

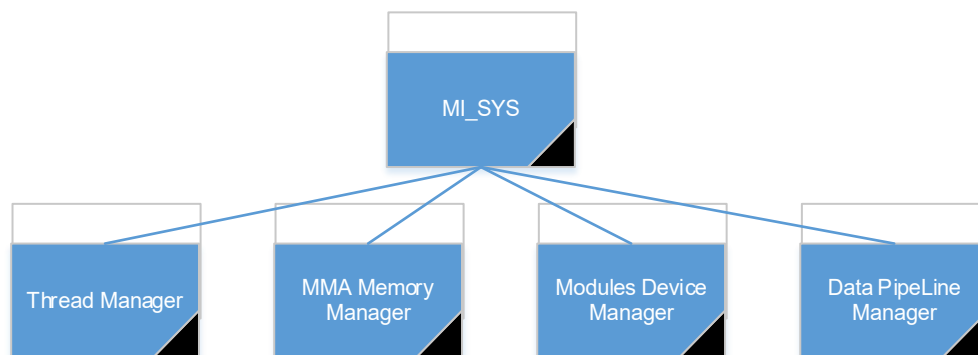


图 1-1: MI_SYS 系统框架图

MI_SYS 主要功能概述如下：

- 实现 MI 系统初始化、MMA 内存缓冲池管理。
- 提供各模块注册设备节点，`proc` 系统的建立的通用接口。
- 提供各模块建立绑定关系的接口，管理各模块之间数据流动。
- 提供各模块申请 MMA 连续物理内存的接口，管理内存分配，映射虚拟地址，内存回收。
- 提供各模块建立工作线程的接口，管理各模块线程的创建，运行，销毁。

MI_SYS 模块对应的 ko 文件为 `mi_sys.ko`，库文件为 `libmi_sys.so`、`libmi_sys.a`，头文件为 `mi_sys.h`、`mi_sys_datatype.h`。具体功能请查阅 API 接口说明。

1.2. 文档格式约束

正体：用于文档正文内容的书写，其中代码段的书写需要用等宽字体。

正体 + 加粗：用于文档正文中重要内容的书写。

斜体：用于文档中 *Tips* 部分的书写。

斜体 + 加粗：用于文档中 ***Tips*** 部分重要内容的书写。

1.3. 关键字说明

ID: Identity document 的缩写，表示唯一编码。

MI: SStar SDK Middle Interface 的缩写，本文中，如果出现“MI_SYS”类似的结构表示的是 MI 的 SYS 模块，如果是单纯的“MI”，泛指整个 SDK。

Hex: 十六进制。

Kernel Mode: 指的是工作在 Kernel 环境下的代码，代码具有直接操作硬件的控制权限，比如 ko 中的函数和线程等。

User Mode: 指的是工作在 User 环境下的，比如客户的应用程序，系统调用等。

APP: Application 的缩写，此处主要指调用 MI API 的应用程序。

API: Application Programming Interface，应用程序接口。

NVR: 全称 Network Video Recorder，即网络视频录像机。

DVR: 全称 Digital Video Recorder，即数字视频录像机或数字硬盘录像机。

XVR: 全称 X(代表任何一种或多种功能) Video Recorder，目前 sstar 实现是 NVR+DVR。

IPC: 全称 IP Camera，即网络摄像机。

HW: 全称 hardware，即硬件。

Dev: 全称 Device，本文中表示 MI 模块设备，1.4.1 有详细解释。

Chn: 全称 Channel，本文中表示 MI 模块设备的某个通道，1.4.1 有详细解释。

Port: 本文中表示 MI 模块设备通道中的某个端口，1.4.1 有详细解释。

Soc: 全称 System on Chip，即系统级芯片，也称片上系统，主芯片。

SocId: 本文中指的是描述特定 Soc 的序列号。某些平台会通过 PCIE 进行多芯片级联，此时需

要用 SocId 进行区分。MI_SYS 仅有部分接口支持跨 Soc 设置，这些接口都带有 SocId 参数。

如图 1-2 所示，通过 PCIE 桥接 Soc1 和 Soc2。Soc1 作为 Master 芯片，运行 Linux 环境、MI SDK 和客户业务程序，Soc2 作为 Slave 芯片，仅运行 Linux 环境和 MI SDK，而没有业务程序。但是可以通过调用携带 SocId 参数的 API 选择对 Soc1 或 Soc2 进行操作。

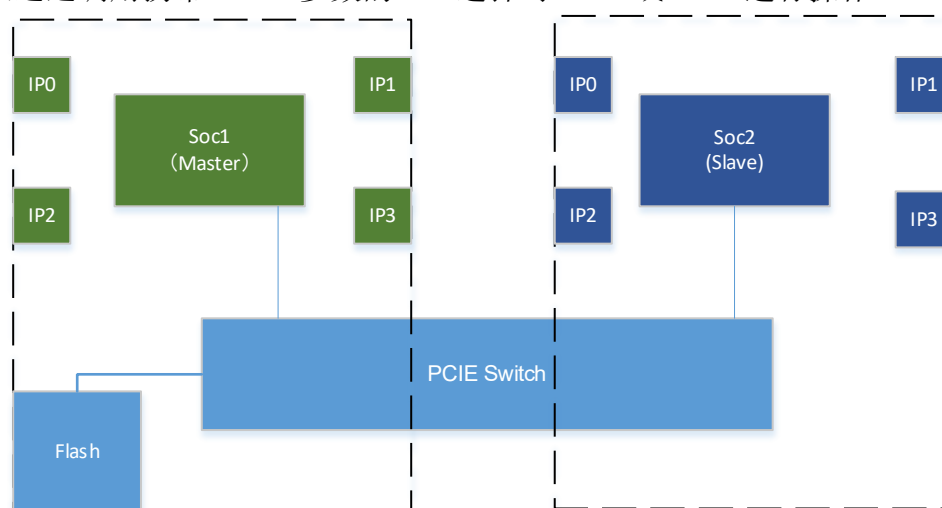


图 1-2: PCIE 级联 Soc 模型

补充说明:

- 如无单独说明，本文中 MI_SYS 和 SYS，MI_DISP 和 DISP 是相同意思，其余模块名称类似。

- 本文中出现的模块名称可以在 [MI ModuleId_e](#) 中查阅其功能描述。

1.4. 流程框图

1.4.1 Dev/Chn/Port 的关系

一个典型的 MI 模块都会有 Dev/Chn/Port 三级结构如图 1-3。

Dev

一个 MI 模块会有一个或多个 Dev，一般来说不同的 Dev 表示该模块设备需要调用不同的 HW 资源，或者工作在不同的工作模式。比如说，VENC 在编码 H264/H265 时和编码 Jpeg 时，需要调用不同的 HW 资源，这个时候 Dev 就要分开了。

Chn

一个 Dev 会有一个或多个 Chn（通道），一般来说不同的 Chn 表示该通道虽然和同 Dev 下的其他 Chn 共享 HW 资源，但是数据来源或者工作模式是不一样的。比如码流来源不同，Chn 一般也不一样。

Port

一个 Chn 会有一个或多个 Port（端口），分为 InputPort（数据流入的端口）和 OutputPort（数据流出的端口）。一般来说不同的 Port 表示该通道虽然和同 Dev 同 Chn 下的其他 Port 共享 HW 资源和数据来源，但是需要设置的参数是不一样的，比如分辨率不同。

一般来说，Port 才是客户操作 MI 模块的最小独立单位，因为它明确了所有的信息：**HW 资源、数据来源、参数属性**。

Tips:

一个 Chn 并不是总是必须有 InputPort 和 OutputPort 的，其 InputPort 和 OutputPort 的数量取决于该模块的行为。

图 1-3 列出了 MI_SYS 中单个 Chn 下 Port 排列的不同场景：

- 1) 只有 InputPort，用于需要数据输入，但是无需输出的模块。比如，Disp 这样的模块，只需要 InputPort，无需 OutputPort，它直接把结果显示到了 Panel 上。
- 2) 只有 OutputPort，用于无数据输入，或者数据输入不经过 MI_SYS 的模块。比如，Vdec 这样的模块，数据来源可以是用户调用 Vdec 接口直接送入 Vdec RingPool，不经过 InputPort。

- 3) 一个 InputPort, 一个 OutputPort, 用于单个数据输入, 单个数据输出的模块。比如, Ldc 这样的模块, 先接收一个数据输入, 然后硬件处理后, 输入对应的结果。
- 4) 一个 InputPort, 多个 OutputPort, 用于单个数据输入, 但是会有不同的处理, 输出不同数据的模块。比如, Scl 这样的模块, 共享一个数据源, 然后会按照不同的 OutputPort 设置的参数来进行处理, 输出不同的结果。

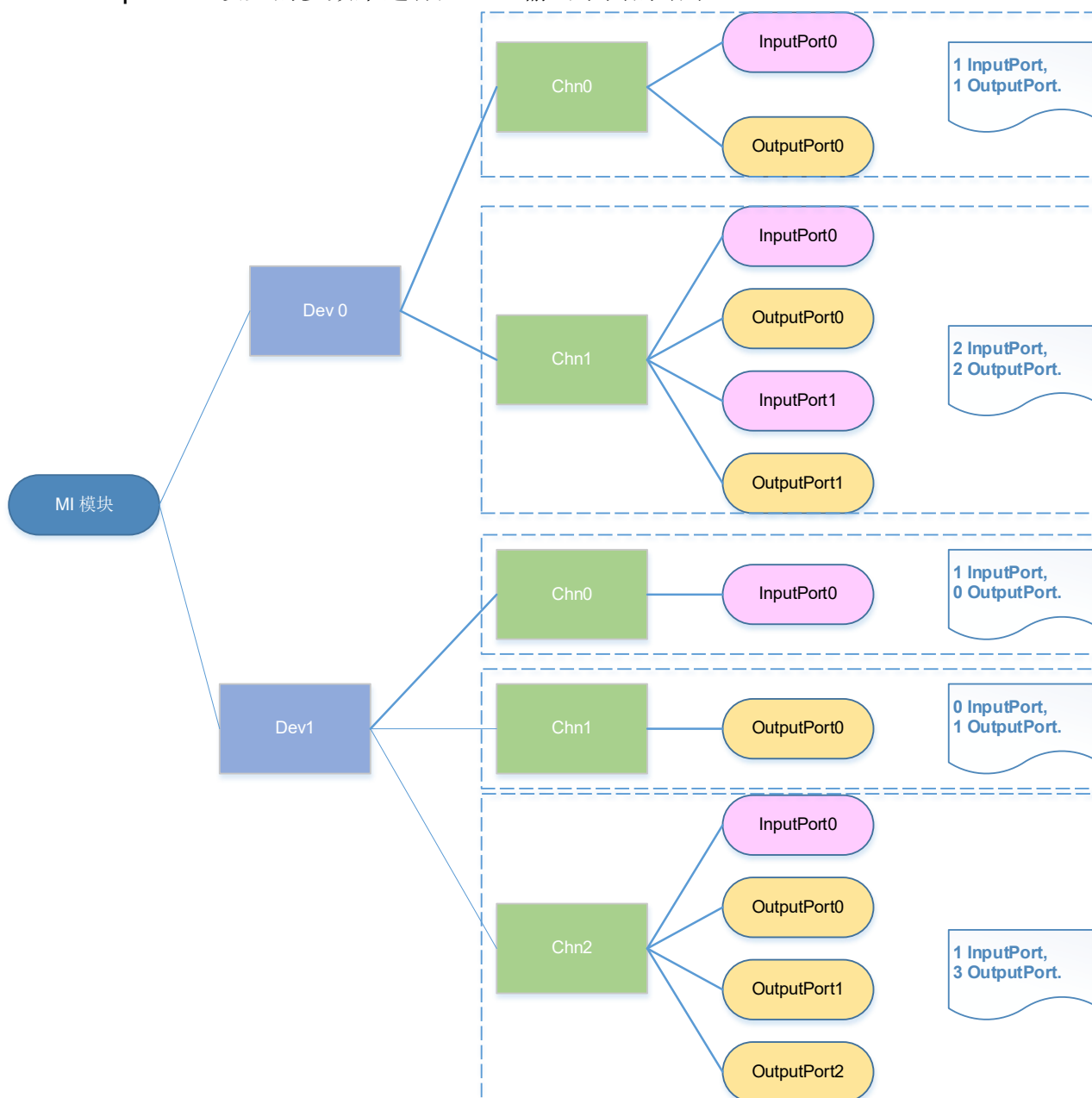


图 1-3: MI 模块的三级结构

Tips:

Chn 和 Port 的界限并不是总是能区分清楚, 如果某模块 API 文档的解释和上文的解释有差异, 调用该模块请以它的 API 文档为准。

1.4.2 一个典型的 NVR 数据流

下图是一个典型的 NVR 数据流模型。流动过程如下：

- 1) 建立 Vdec->Disp 的绑定关系；
- 2) 用户接收码流数据存盘并写入到 Vdec 的 RingPool；
- 3) Vdec 解码，写入解码后的数据到 Vdec OutputPort 申请的内存，送入下一级；
- 4) Disp 将接收到的数据显示出来。

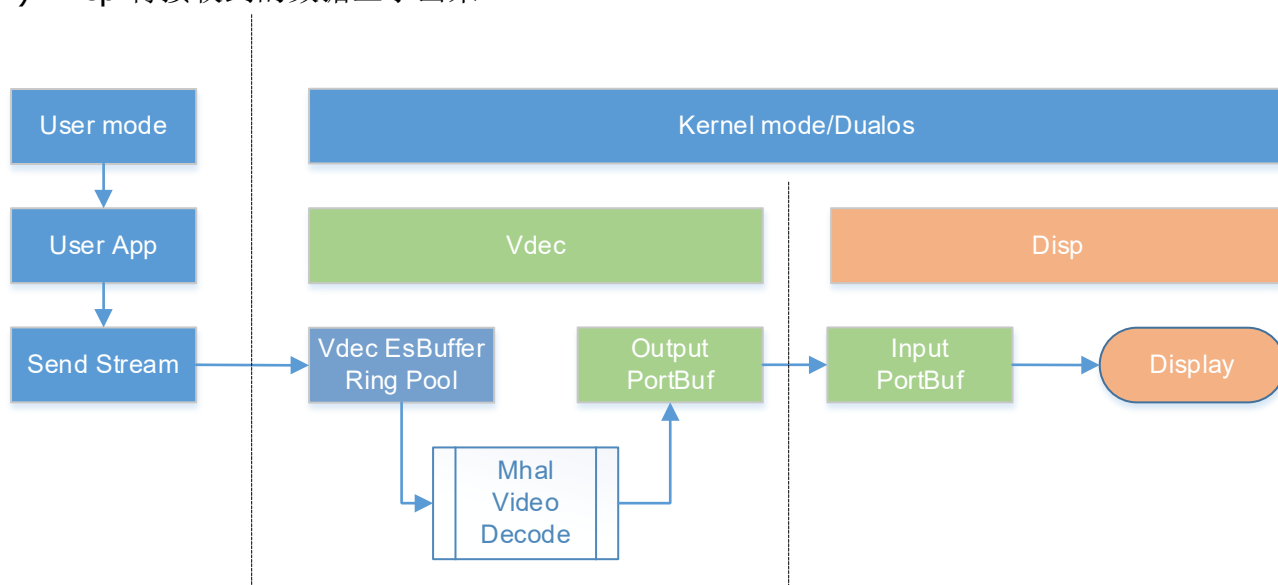


图 1-4: 一个典型的 NVR 数据流模型

1.4.3 一个典型的 IPC 数据流

下图是一个典型的 IPC 数据流模型，流动过程如下：

- 1) 建立 Vif->Isp->Scl->Venc 的绑定关系；
- 2) Sensor 将数据送入 vif 处理；
- 3) Vif 将处理后的数据写入 Output Port 申请的内存，送入下一级；
- 4) Isp 接收数据，进行处理，将结果写入 Output Port 申请的内存，送入下一级；
- 5) Scl 接收数据，进行处理，将结果写入 Output Port 申请的内存，送入下一级；
- 6) Venc 接收数据，送入编码器进行编码处理，将编码后的数据写入 RingPool 内存区；
- 7) 用户调用 Venc 的接口取流，送入用户业务层 App，通过网络转送。

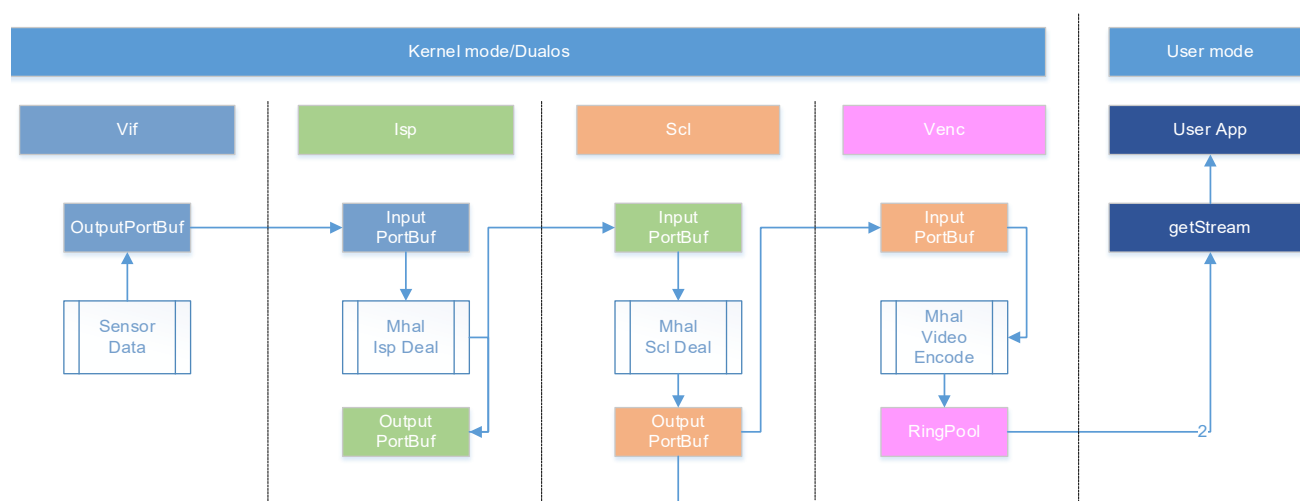


图 1-5: 一个典型的 IPC 数据流模型

1.4.4 一个典型的 DVR 数据流

下图是一个典型的 DVR 数据流模型，根据输入数据流类型及应用场景不同分为两种情况：

- 第一种情况，输入源为从磁盘读取的 H264/H265 编码数据类型的回放场景，流动过程如下：
 - 1) 建立 Vdec->Disp 的绑定关系；
 - 2) 用户写入码流到 Vdec 的 RingPool；
 - 3) Vdec 解码，写入解码后的数据到 Vdec OutputPort 申请的内存，送入下一级；
 - 4) Disp 将接收到的数据显示出来。
- 第二种情况，输入源为 sensor，流动过程如下：
 - 1) 建立 Vif->Isp->Scl->Disp 的绑定关系；
 - 2) Sensor 将数据送入 vif 处理；
 - 3) Vif 将处理后的数据写入 Output Port 申请的内存，送入下一级；
 - 4) Isp 接收数据，进行处理，将结果写入 Output Port 申请的内存，送入下一级；
 - 5) Scl 接收数据，进行处理，将结果写入 Output Port 申请的内存，送入下一级；
 - 6) Venc 接收数据，送入编码器进行编码处理，将编码后的数据写入 RingPool 内存区；
 - 7) Disp 将接收到的数据进行预览显示；
 - 8) 用户调用 Venc 的接口取流，送入用户业务层 App，存入磁盘。

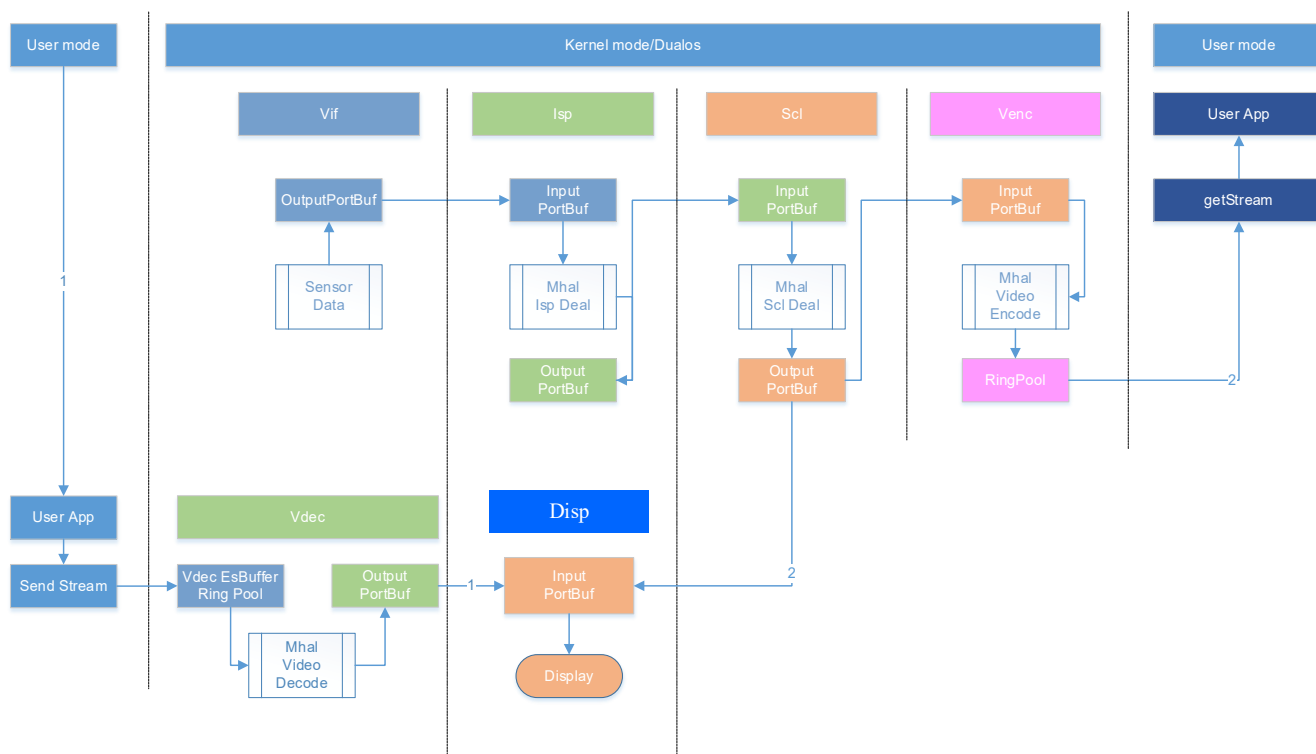


图 1-6: 一个典型的 DVR 数据流模型

1.4.5 一个典型的 XVR 数据流

XVR 同时具备 NVR 和 DVR 的功能，可自由在两种设备功能间自由切换，其数据流也是如此。

1.5. Bind 和帧率控制场景说明

下面列出了一些常见的数据流场景来说明帧率的工作机制，图中成员的定义如下：

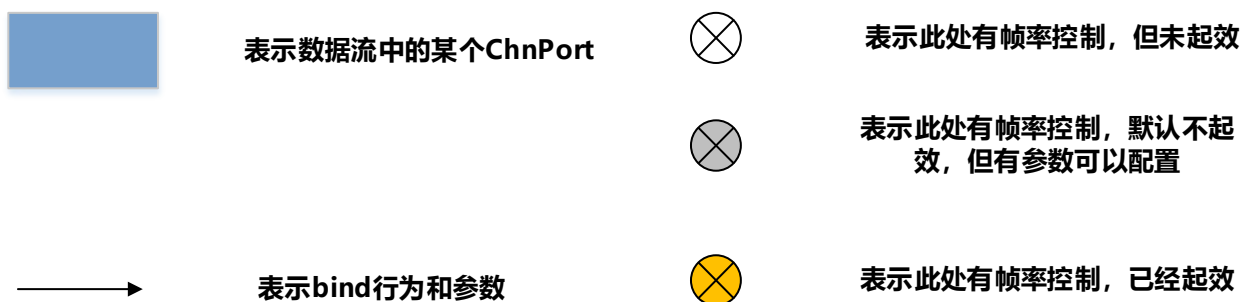


图 1-7: Bind 场景成员定义

Tips:

为了保证接口易用性，MI_SYS 内部会自动校准前级 Output 的帧率。

用户设置的 *u32SrcFrmrate* 和 *MI_SYS* 内部计算值有偏差时，以 *MI_SYS* 内部计算值替代 *u32SrcFrmrate* 来做帧率控制。但是 *MI_SYS* 内部计算需要时间，当帧率发生较大变化时，会出现 1-2 秒的抖动。

因此，使用 *MI_SYS_BindXXX* 接口时，请尽量保证这两个帧率设置准确。并且帧率有变化时，主动调用 *MI_SYS_BindXXX* 更新帧率。

下文中假定用户设置的 *u32SrcFrmrate* 是准确的前级帧率，*u32DstFrmrate* 是客户想要的后级帧率。

1.5.1 一对一 Bind

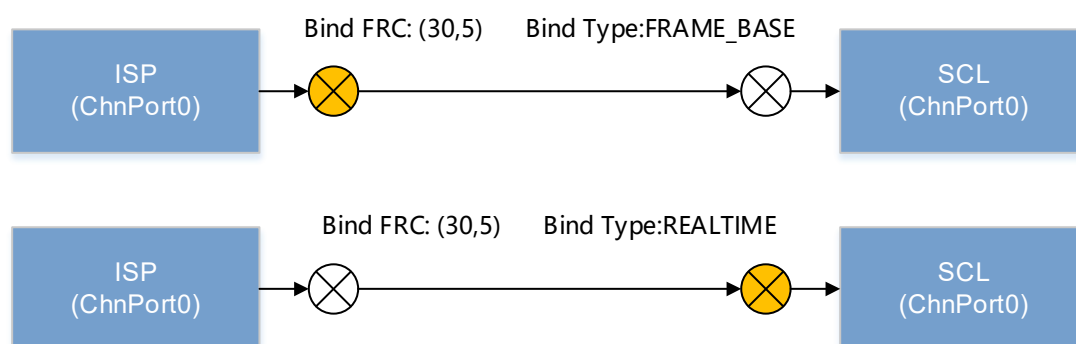


图 1-8: 一对一 Bind 场景

对应前级 Chnport 的 Output 一对一 bind 后级 Chnport Input 的场景，由 Bind Type 决定帧率控制策略。

如果是 FRAME_BASE 模式，前级 Output 帧率控制起效，把输出帧率控制到目标值，后级 Input 帧率控制不起效。如果是 REALTIME/HW_RING 模式，前级 Output 帧率控制不起效，后级 Input 帧率控制直接控制到目标值。

以图示为例：

Case1 设置 Bind FRC: (30,5) ,Bind Type:FRAME_BASE, 此时前级 ISP 的输出帧率就被控制到 5FPS。

Case2 设置 Bind FRC: (30,5) Bind Type:REALTIME, 此时前级 ISP 的输出帧率维持 30FPS 不变，后端 SCL 的输入帧率被控制到 5FPS。

Tips:

对于 *REALTIME/HW_RING* 模式，无论是哪种场景下，前级 *Output* 帧率控制都不起效的，此时只能控制后级 *Input* 帧率，这是硬件工作机制决定的，下文不再复述。

1.5.2 一对多 Bind

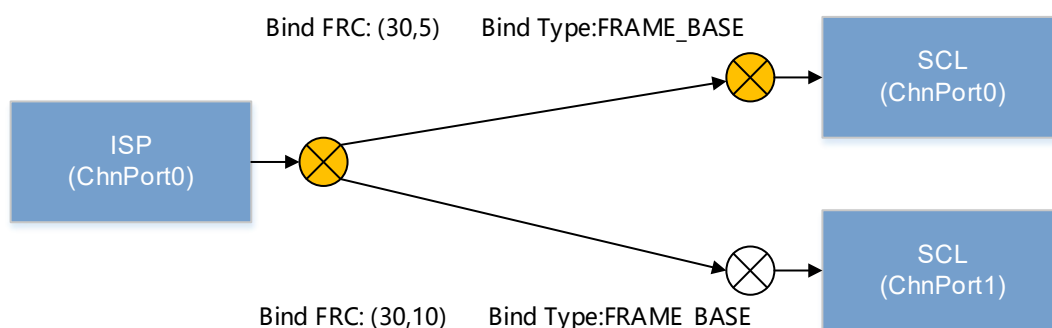


图 1-9: 一对多 Bind 场景

对应前级 Chnport 的 Output 一对多 bind 后级 Chnport Input 的场景，Bind Type 和 Bind FRC 共同决定帧率控制策略。

如果是 *FRAME_BASE* 模式，由于前级 ISP 同时 Bind 多个后级，只能把帧率控制到能满足所有后级 bind 模块的帧率需求，然后在后级 Input 分别再做帧率控制。如果是 *REALTIME/HW_RING* 模式，前级 Output 帧率控制不起效，后级 Input 分别做帧率控制。

以图示为例：

在该场景中，对 ISP ChnPort0 的两次 bind 帧率分别是 (30,5) (30,10)，因此，ISP ChnPort0 的 Output 帧率为 10FPS。

SCL ChnPort0 的 input 需要进一步把帧率从 10FPS 控制到 5FPS。SCL ChnPort1 则无需再进行帧率控制。

1.5.3 Bind 的同时设置了 UserDepth

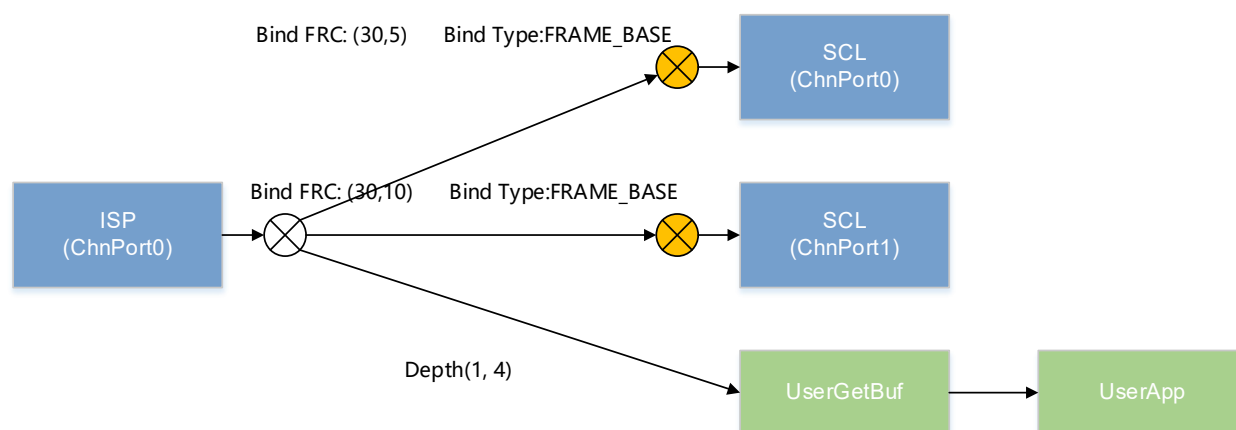


图 1-10: Bind 的同时设置了 UserDepth

对应 Bind 后级的同时，前级 Output 端同时设置了 UserDepth 的场景，帧率控制策略是固定且唯一的。

调用 MI 接口设置的 UserDepth，表明用户需要拿输出数据去使用，这个时候前级 Output 总是倾向把最新的数据给到用户，因此对前端输出不做帧率控制。

假如有 bind 后级，则在后级 Input 做帧率控制。假如没有 bind 后级，帧率控制就完全不起效了，此时数据输出就由数据产生和消费的速度来决定。我们会在说明 Depth 概念时，再详细解释。

以图示为例：

在该场景中，对 ISP ChnPort0 的两次 bind 帧率分别是 (30,5) (30,10)，但是设置了 ISP ChnPort0 的 Output Depth 为(1, 4)。因此，ISP ChnPort0 的 Output 帧率为 30FPS。SCL ChnPort0 的 input 把帧率从 30FPS 控制到 5FPS。SCL ChnPort1 把帧率从 30FPS 控制到 10FPS。

1.5.4 NVR 场景下 Vdec 帧率控制的补充

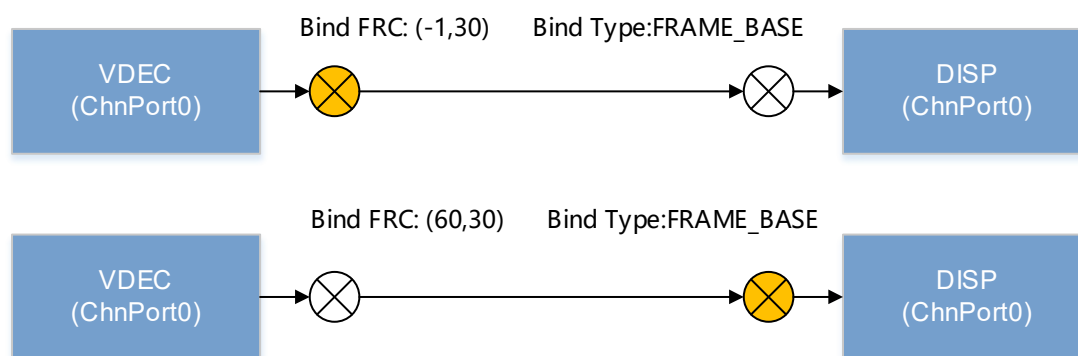


图 1-11: 对于 NVR 场景下 Vdec 帧率控制的补充

对应 Vdec Chnport 的 Output 一对一 bind 后级 Chnport Input 的场景，由 BindFRC 决定帧率控制策略。

由于 NVR 需要用户输入码流到 Vdec，很难严格保证送码流的速度是均衡的，这一点和 IPC 这种数据源由中断严格控制时序的场景是不一样的。如果用户无法保证送入的帧率稳定，可以使用 MI 提供的自动计算送帧速度的功能，该功能在 u32SrcFrmrate 设置为-1 时自动开启，此时 MI 会严格按照 u32DstFrmrate 来决定是否分配 Buffer 给 Vdec。

此外，由于 NVR 会提供倍速播放的功能，当后级 Disp 的 Timing 跟不上，就需要在输入 Disp 之前主动丢帧。如果 Vdec 解码速度快于 Disp 显示 Timing，可使用 MI 提供的按比例丢帧功能，通过设置 u32SrcFrmrate 和 u32DstFrmrate 的比例来做达到想要的帧率。

以图示为例：

Case1 设置 Bind FRC: (-1,30) Bind Type:FRAME_BASE，假设前级 Vdec Output 输出帧率是 30，此时 Vdec Output 每 $1000\text{ms}/30 = 33.3\text{ms}$ 才能得到 MI 分配的 Buffer。

Case2 设置 Bind FRC: (60, 30) Bind Type:FRAME_BASE，比例为 $60/30 = 2/1$ ，假设前级 Vdec Output 输出帧率是 60，此时 Vdec Output 每解码两张，只有一张会推送到 Disp 的 Input。

Tips:

- 对于 NVR 产品，可以通过 MI_VDEC 的 API 接口查询当前 ChnPort 的遗留帧来动态选择两种策略，以达到更好的效果，此处不再赘述。
- 对于 Vdec Bind 后级时，自动校准前级 Output 的真实帧率功能不再起效。

1.6. insmod 参数说明

使用 insmod mi_sys.ko 后面可以带的参数使用情况如下所示：

表 1-1 insmod mi_sys.ko 可加参数

参数	含义	取值
bEnableMmu	是否使能 MMU	1:enable mmu 0:disable mmu 由于需要底层逻辑的配合，设置成 1 不一定会 enable MMU。设置成 0 会 disable MMU。默认值是 1。 Ex: bEnableMmu=1，表示使能 mmu bEnableMmu=0，表示禁止 mmu
logBufSize	缓存 log 的内存长度	默认为 4KByte，单位：Byte Ex: logBufSize=4096，表示设置 4096Byte，即 4KByte
entrySize	enable mmu 后配置这个才有效	Tiramisu 支援 128/256/512，单位：Kbyte entrySize 的默认值是 0，默认会使用系统配置的 size，不同 IC 用的 size 有可能不一样。 Ex: entrySize=128，表示设置 128KByte
cmdQBufSize	cmdq 内存大小	由产品规格决定，单位：Kbyte cmdQBufSize 的默认值在 linux 是 1024，在 RTK 针对不同的 IC 可能会不一样。 Ex: cmdQBufSize=256，表示设置 256KByte

2. API 参考

2.1. API 描述格式说明

本手册使用 8 个参考域描述 API 的相关信息，它们作用如下表示。

表 2-2: API 描述格式说明

标签	功能
功能	简要描述 API 的主要功能。
语法	列出调用 API 应包括的头文件以及 API 的原型声明。

参数	列出 API 的参数、参数说明及参数属性。
返回值	列出 API 所有可能的返回值及其含义。
依赖	列出 API 包含的头文件和 API 编译时要链接的库文件。
注意	列出使用 API 时应注意的事项。
举例	列出使用 API 的实例。
相关主题	调用上下文中有关联的接口。

2.2. 功能模块 API 列表

如前文所述，我们可以粗略的把 MI_SYS 的 API 分为三大类：系统功能类、数据流类、内存管理类。

Tips:

MI_SYS 仅有部分接口支持跨 Soc 设置，这些接口都带有 *u16SocId* 参数，没有携带此参数的接口默认不支持跨 Soc 设置。

表 2-3: API 列表

API 名	功能
系统功能类	
MI_SYS_Init	初始化 MI_SYS 系统
MI_SYS_Exit	析构 MI_SYS 系统
MI_SYS_GetVersion	获取 MI 的系统版本号
MI_SYS_GetCurPts	获取 MI 系统当前时间戳
MI_SYS_InitPtsBase	初始化 MI 系统基准时间戳
MI_SYS_SyncPts	同步 MI 系统时间戳
MI_SYS_SetReg	设置寄存器的值，调试用
MI_SYS_GetReg	获取寄存器的值，调试用
MI_SYS_ReadUuid	获取 Chip 的 Unique ID
MI_SYS_EnableChnOutputPortLowLatency	打开或者关闭通道 output 端口的低延迟输出功能，默认关闭

API 名	功能
数据流类	
MI_SYS_BindChnPort	数据源输出端口到接收者输入端口的绑定
MI_SYS_BindChnPort2	数据源输出端口到接收者输入端口的绑定，需要指定工作模式
MI_SYS_UnBindChnPort	数据源输出端口到接收者输入端口的解绑定
MI_SYS_GetBindbyDest	查询数据接收者输入端口的对应的源输出端口
MI_SYS_ChInputPortGetBuf	获取通道 inputPort 的 buf
MI_SYS_ChInputPortPutBuf	将通道 inputPort 的 buf 加到待处理队列
MI_SYS_ChOutputPortGetBuf	获取通道 outputPort 的 buf
MI_SYS_ChOutputPortPutBuf	释放通道 outputPort 的 buf
MI_SYS_ChInputPortGetBufPa	分配通道 input 端口对应的 buf object，仅提供 MIU 物理地址
MI_SYS_ChInputPortPutBufPa	把通道 input 端口对应的 buf object 加到待处理队列，与 MI_SYS_ChInputPortGetBufPa 成对使用
MI_SYS_ChOutputPortGetBufPa	分配通道 output 端口对应的 buf object，仅提供 MIU 物理地址
MI_SYS_ChOutputPortPutBufPa	释放通道 output 端口对应的 buf object，与 MI_SYS_ChOutputPortGetBufPa 成对使用
MI_SYS_SetChnOutputPortDepth	设置通道 OutputPort 的深度
MI_SYS_ChPortInjectBuf	向模块通道 inputPort 注入 outputPort Buf 数据
MI_SYS_GetFd	获取当前 output Port 等待事件的文件描述符
MI_SYS_CloseFd	关闭 Output Port 对应的文件描述符
MI_SYS_DupBuf	复制一个 buf object
MI_SYS_ChInputPortSetUserPicture	按照设定帧率，驱动会重复输入该 buf 到当前 input port，并加到待处理队列
MI_SYS_EnableUserPicture	打开 user picture 功能
MI_SYS_DisableUserPicture	关闭 user picture 功能
MI_SYS_SetChnOutputPortUserFrc	设置 output 用户得到的帧率，在某些情况下可以降低带宽

API 名	功能
内存管理类	
MI_SYS_SetChnMMAConf	设置模块设备通道输出端口默认分配内存的 MMA 池名称
MI_SYS_GetChnMMAConf	获取模块设备通道输出端口默认分配内存的 MMA 池名称
MI_SYS_MMA_Alloc	应用程序从 MMA 内存管理池申请物理连续内存
MI_SYS_MMA_Free	在用户态释放 MMA 内存管理池中分配到的内存
MI_SYS_Mmap	映射物理内存到 CPU 虚拟地址
MI_SYS_Munmap	取消物理内存到虚拟地址的映射
MI_SYS_FlushInvCache	Flush cache CPU 虚拟地址
MI_SYS_ConfigPrivateMMAPool	为模块配置私有 MMA Heap
MI_SYS_PrivateDevChnHeapAlloc	从模块通道私有 MMA Pool 申请内存
MI_SYS_PrivateDevChnHeapFree	从模块通道私有 MMA pool 释放内存
MI_SYS_Va2Pa	将 MI 内存块的 CPU 虚拟地址转成内存物理地址

2.3. 系统功能类 API

2.3.1 MI_SYS_Init

➤ 功能

MI_SYS 初始化，MI_SYS 模块为系统内其它 MI 模组提供基础支援，需要早于系统内其它 MI 模组初始化，否则其它 stream 类型的模组初始化时会失败。

➤ 语法

```
MI_S32 MI_SYS_Init(MI_U16 u16SocId);
```

➤ 形参

参数名称	描述	输入/输出
u16SocId	指定执行本操作的目标 Soc	输入

➤ 返回值

- 0 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

※ 注意

- [MI_SYS_Init](#) 需要早于其他 MI 模组的 Init 函数调用。
- 可以在同一进程或多进程中重复调用 [MI_SYS_Init](#)，但必须和 [MI_SYS_Exit](#) 成对使用，否则会报错。
- 系统在内核启动参数内，需要配置好 MMA 内存堆的配置参数。

➤ 举例

系统 Init 调用 Sample

```
#define ST_DEFAULT_SOC_ID 0
MI_S32 ST_Sys_Init(void)
{
    MI_SYS_Version_t stVersion;
```

```

MI_U64 u64Pts = 0;
MI_U16 u16SocId = ST_DEFAULT_SOC_ID;
MI_U64 u64Uuid;
STCHECKRESULT(MI_SYS_Init(u16SocId));

memset(&stVersion, 0x0, sizeof(MI_SYS_Version_t));
STCHECKRESULT(MI_SYS_GetVersion(u16SocId, &stVersion));
ST_INFO("u8Version:%s\n", stVersion.u8Version);

STCHECKRESULT(MI_SYS_ReadUuid(u16SocId, &u64Uuid));
INFO("u64Uuid:%llx\n", u64Uuid);

STCHECKRESULT(MI_SYS_GetCurPts(u16SocId, &u64Pts));
ST_INFO("u64Pts:0x%llx\n", u64Pts);

u64Pts = 0xF1237890F1237890;
STCHECKRESULT(MI_SYS_InitPtsBase(u16SocId, u64Pts));

u64Pts = 0xE1237890E1237890;
STCHECKRESULT(MI_SYS_SyncPts(u16SocId, u64Pts));

return MI_SUCCESS;
}

MI_S32 ST_Sys_Exit(void)
{
    MI_U16 u16SocId = ST_DEFAULT_SOC_ID;
    STCHECKRESULT(MI_SYS_Exit(u16SocId));

    return MI_SUCCESS;
}

```

Tips:

本示例适用于: [MI_SYS_Init](#) / [MI_SYS_Exit](#) / [MI_SYS_GetVersion](#) / [MI_SYS_GetCurPts](#) / [MI_SYS_InitPtsBase](#) / [MI_SYS_SyncPts](#) / [MI_SYS_ReadUuid](#) .

➤ 相关主题

[MI_SYS_Exit](#)

2.3.2 MI_SYS_Exit

➤ 功能

MI_SYS 去初始化，调用 [MI_SYS_Exit](#) 之前，需要确保系统内所有其他模组都已经完成去初始化，所有 VBPOOL 都已经销毁，否则 [MI_SYS_Exit](#) 会返回失败。

➤ 语法

```
MI_S32 MI_SYS_Exit (MI_U16 u16SocId);
```

➤ 形参

参数名称	描述	输入/输出
u16SocId	指定执行本操作的目标 Soc	输入

➤ 返回值

- 0 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

※ 注意

- [MI_SYS_Exit](#) 调用前需要确保系统内所有其他模组都已经完成去初始化。
- [MI_SYS_Exit](#) 调用前需要确保系统内所有创建的 VBPOOL 已经成功销毁。

➤ 举例

参见 [系统 Init 调用 Sample](#) 举例。

➤ 相关主题

[MI_SYS_Init](#)

2.3.3 MI_SYS_GetVersion

➤ 功能

获取 MI 的系统版本号。

➤ 语法

MI_S32 MI_SYS_GetVersion (MI_U16 u16SocId, [MI_SYS_Version_t](#) *pstVersion);

➤ 形参

参数名称	描述	输入/输出
u16SocId	指定执行本操作的目标 Soc	输入
pstVersion	系统版本号返回数据结构指针	输出

➤ 返回值

- 0 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

※ 注意

N/A

➤ 举例

参见 [系统 Init 调用 Sample](#) 举例。

➤ 相关主题

N/A

2.3.4 MI_SYS_GetCurPts

➤ 功能

获取 MI 系统当前时间戳。

➤ 语法

MI_S32 MI_SYS_GetCurPts (MI_U16 u16SocId, MI_U64 *pu64Pts);

➤ 形参

参数名称	描述	输入/输出
u16SocId	指定执行本操作的目标 Soc	输入
pu64Pts	系统当前时间戳返回地址	输出

➤ 返回值

- 0 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

※ 注意

N/A

➤ 举例

参见 [系统 Init 调用 Sample](#) 举例。

➤ 相关主题

N/A

2.3.5 MI_SYS_InitPtsBase

➤ 功能

初始化 MI 系统时间戳基准。

➤ 语法

MI_S32 MI_SYS_InitPtsBase (MI_U16 u16SocId, MI_U64 u64PtsBase);

➤ 形参

参数名称	描述	输入/输出
u16SocId	指定执行本操作的目标 Soc	输入
u64PtsBase	设置的系统时间戳基准	输入

➤ 返回值

- 0 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

※ 注意

N/A

➤ 举例

参见 [系统 Init 调用 Sample](#) 举例。

➤ 相关主题

N/A

2.3.6 MI_SYS_SyncPts

➤ 功能

微调同步 MI 系统时间戳。

➤ 语法

MI_S32 MI_SYS_SyncPts (MI_U16 u16SocId, MI_U64 u64Pts);

➤ 形参

参数名称	描述	输入/输出
u16SocId	指定执行本操作的目标 Soc	输入
u64Pts	微调后的系统时间戳基准	输入

➤ 返回值

- 0 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

※ 注意

N/A

➤ 举例

参见 [系统 Init 调用 Sample](#) 举例。

➤ 相关主题

N/A

2.3.7 MI_SYS_SetReg

➤ 功能

设置寄存器的值，调试用。

➤ 语法

MI_S32 MI_SYS_SetReg (MI_U16 u16SocId, MI_U32 u32RegAddr, MI_U16 u16Value, MI_U16 u16Mask);

➤ 形参

参数名称	描述	输入/输出
u16SocId	指定执行本操作的目标 Soc	输入
u32RegAddr	Register 总线之地址	输入
u16Value	待写入之 16bit 寄存器值	输入
u16Mask	本次写入寄存器值之 Mask 遮挡栏位	输入

返回值

- 0 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

※ 注意

N/A

➤ 举例

N/A

➤ 相关主题

N/A

2.3.8 MI_SYS_GetReg

➤ 功能

读取寄存器的值，调试用。

➤ 语法

```
MI_S32 MI_SYS_GetReg (MI_U16 u16SocId, MI_U32 u32RegAddr, MI_U16
*pu16Value);
```

➤ 形参

参数名称	描述	输入/输出
u16SocId	指定执行本操作的目标 Soc	输入
u32RegAddr	Register 总线之地址	输入
pu16Value	待读回 16bit 寄存器值返回地址	输出

➤ 返回值

- 0 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

※ 注意

N/A

➤ 举例

N/A

➤ 相关主题

N/A

2.3.9 MI_SYS_ReadUuid

➤ 功能

获取 Chip 的 Unique ID

➤ 语法

MI_S32 MI_SYS_ReadUuid (MI_U16 u16SocId, MI_U64 *u64Uuid);

➤ 形参

参数名称	描述	输入/输出
u16SocId	指定执行本操作的目标 Soc	输入
u64Uuid	获取 chip unique ID 值的指针	输出

➤ 返回值

- 0 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

※ 注意

N/A

➤ 举例

参见[系统 Init 调用 Sample](#) [系统 Init 调用 Sample](#) 举例。

2.3.10 MI_SYS_EnableChnOutputPortLowLatency

➤ 功能

打开或者关闭通道 output 端口的低延迟输出功能

➤ 语法

MI_S32 MI_SYS_EnableChnOutputPortLowLatency(MI_U16 u16SocId, MI_SYS_ChnPort_t *pstChnPort, MI_BOOL bEnable , MI_U32 u32Param);

➤ 形参

参数名称	描述	输入/输出
------	----	-------

u16SocId	指定执行本操作的目标 Soc	输入
pstChnPort	指向模块通道之 output 端口的指针	输入
bEnable	TRUE: 打开 FALSE:关闭, 默认为 FALSE	输入
u32Param	Low Latency 参数, 其含义由具体模块决定 用于设置 Line Count, 表示写完多少条 Line 后就 Output 给 User 或后级	输入

➤ 返回值

- 0 成功。
- 非 0 失败, 参照[错误码](#)。

➤ 依赖

- 头文件: mi_sys_datatype.h、mi_sys.h
- 库文件: libmi_sys.a / libmi_sys.so

※ 注意

当 bEnable 为 true 时, u32Param 必须大于 0, 否则设置无效

➤ 举例

MI_SYS_EnableChnOutputPortLowLatency 调用 Sample

```
MI_SYS_ChnPort_t stChnPort;
MI_U16 u16SocId = ST_DEFAULT_SOC_ID;
stChnPort.eModId = E_MI_MODULE_ID_ISP;
stChnPort.u32DevId = 0;
stChnPort.u32ChnId = 0;
stChnPort.u32PortId = 0;

//打开 ISP 通道 0 output 端口 0 的低延迟输出功能,设定完成 100 行即可输出
MI_SYS_EnableChnOutputPortLowLatency(u16SocId, &stChnPort, TRUE, 100);

//关闭 ISP 通道 0 output 端口 0 的低延迟输出功能
MI_SYS_EnableChnOutputPortLowLatency(u16SocId, &stChnPort, FALSE, 0);
```

2.4. 数据流类 API

2.4.1 MI_SYS_BindChnPort

➤ 功能

数据源输出端口到数据接收者输入端口的绑定。

➤ 语法

```
MI_S32 MI_SYS_BindChnPort(MI_U16 u16SocId, MI_SYS_ChnPort_t *pstSrcChnPort,
MI_SYS_ChnPort_t *pstDstChnPort, MI_U32 u32SrcFrmrate, MI_U32 u32DstFrmrate);
```

➤ 形参

参数名称	参数含义	输入/输出
u16SocId	指定执行本操作的目标 Soc	输入
pstSrcChnPort	源端口配置信息数据结构指针	输入
pstDstChnPort	目标端口配置信息数据结构指针	输入
u32SrcFrmrate	源端口配置的帧率	输入
u32DstFrmrate	目标端口配置的帧率	输入

➤ 返回值

- 0 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

※ 注意

- 源端口必须是通道输出端口。
- 目标端口必须是通道输入端口。
- 源和目标端口必须之前没有被绑定过。
- [MI_SYS_BindChnPort](#) 在 pstSrcChnPort、pstDstChnPort 和 eBindType 相同的情况下可以被多次调用，来重新设置需要修改的 u32SrcFrmrate 和 u32DstFrmrate。
- 本接口只支持按照 [E_MI_SYS_BIND_TYPE_FRAME_BASE](#) 方式绑定模块，在 Version 2.0 以上版本不推荐使用，请使用 [MI_SYS_BindChnPort2](#) 替代。

➤ 举例

MI_SYS_BindChnPort 调用 Sample

```
MI_SYS_ChnPort_t stSrcChnPort;
MI_SYS_ChnPort_t stDstChnPort;
MI_U32 u32SrcFrmrate;
MI_U32 u32DstFrmrate;
MI_U16 u16SocId = ST_DEFAULT_SOC_ID;

stSrcChnPort.eModId = E_MI_MODULE_ID_ISP;
stSrcChnPort.u32DevId = 0;
stSrcChnPort.u32ChnId = 0;
stSrcChnPort.u32PortId = 0;
stDstChnPort.eModId = E_MI_MODULE_ID_VENC;
stDstChnPort.u32DevId = 0;
stDstChnPort.u32ChnId = 0;
stDstChnPort.u32PortId = 0;
u32SrcFrmrate = 30;
u32DstFrmrate = 30;
MI_SYS_BindChnPort(u16SocId, &stSrcChnPort, &stDstChnPort, u32SrcFrmrate,
u32DstFrmrate);
```

➤ 相关主题

[MI_SYS_BindChnPort2](#)、[MI_SYS_UnBind_ChnPort](#)。
[Bind 和帧率控制场景说明](#)。

2.4.2 MI_SYS_BindChnPort2

➤ 功能

数据源输出端口到数据接收者输入端口的绑定，需要额外指定工作模式。

➤ 语法

```
MI_S32 MI_SYS_BindChnPort2(MI_U16 u16SocId, MI\_SYS\_ChnPort\_t *pstSrcChnPort,
MI\_SYS\_ChnPort\_t *pstDstChnPort, MI_U32 u32SrcFrmrate, MI_U32 u32DstFrmrate,
MI\_SYS\_BindType\_e eBindType, MI_U32 u32BindParam);
```

➤ 形参

参数名称	参数含义	输入/输出
u16SocId	指定执行本操作的目标 Soc	输入
pstSrcChnPort	源端口配置信息数据结构指针。	输入
pstDstChnPort	目标端口配置信息数据结构指针。	输入
u32SrcFrmrate	源端口配置的帧率	输入
u32DstFrmrate	目标端口配置的帧率	输入
eBindType	源端口与目标端口连接的工作模式，参考 MI_SYS_BindType_e	输入
u32BindParam	不同工作模式需带入的额外参数。	输入

➤ 返回值

- 0 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

※ 注意

- 源端口必须是通道输出端口。
- 目标端口必须是通道输入端口。
- 同一个源端口可以绑定多个目标端口。
- 同一个目标端口只能绑定一个源端口，使用之前必须没有被绑定过。
- [MI_SYS_BindChnPort2](#) 在 pstSrcChnPort、pstDstChnPort 和 eBindType 相同的情况下可以被多次调用，来重新设置需要修改的 u32SrcFrmrate 和 u32DstFrmrate。
- 旧版本 MI SYS 不提供此接口，如没有找到，不用设置。
- 各种 eBindType 和 u32BindParam 使用场景如下：

eBindType	适用场景
E_MI_SYS_BIND_TYPE_SW_LOW_LATENCY	u32BindParam 表示低时延值，单位 ms
E_MI_SYS_BIND_TYPE_HW_RING	在 Maruko 之前的 chip，u32BindParam 表示 ring

	buffer depth，目前是只有 venc（h264/h265）支持这种模式，只支持一路。 在 Maruko 及其之后的 chip，u32BindParam 暂未使用。
E_MI_SYS_BIND_TYPE_REALTIME	u32BindParam 未使用，该模式使用场景：jpe imi，只支持一路；vif->isp，支持一路；isp->scl，支持多路
E_MI_SYS_BIND_TYPE_FRAME_BASE	u32BindParam 未使用，默认是走这种 frame mode

➤ 举例

MI_SYS_BindChnPort2 调用 Sample

```
MI_SYS_ChnPort_t stSrcChnPort;
MI_SYS_ChnPort_t stDstChnPort;
MI_U32 u32SrcFrmrate;
MI_U32 u32DstFrmrate;
MI_SYS_BindType_e eBindType;
MI_U32 u32BindParam;
MI_U16 u16SocId = ST_DEFAULT_SOC_ID;

// a. ISP 与 venc 连接方式为 E_MI_SYS_BIND_TYPE_FRAME_BASE，代码如下：
stSrcChnPort.eModId = E_MI_MODULE_ID_ISP;
stSrcChnPort.u32DevId = 0;
stSrcChnPort.u32ChnId = 0;
stSrcChnPort.u32PortId = 0;
stDstChnPort.eModId = E_MI_MODULE_ID_VENC;
stDstChnPort.u32DevId = 0;
stDstChnPort.u32ChnId = 0;
stDstChnPort.u32PortId = 0;
u32SrcFrmrate = 30;
u32DstFrmrate = 30;
eBindType = E_MI_SYS_BIND_TYPE_FRAME_BASE;
u32BindParam = 0;
```

```
STCHECKRESULT(MI_SYS_BindChnPort2(u16SocId, &stSrcChnPort, &stDstChnPort,
u32SrcFrmrate, u32DstFrmrate, eBindType, u32BindParam));
```

// b. ISP 与 jpe 连接方式为 E_MI_SYS_BIND_TYPE_REALTIME, 代码如下:

```
stSrcChnPort.eModId = E_MI_MODULE_ID_ISP;
stSrcChnPort.u32DevId = 0;
stSrcChnPort.u32ChnId = 0;
stSrcChnPort.u32PortId = 0;
stDstChnPort.eModId = E_MI_MODULE_ID_VENC;
stDstChnPort.u32DevId = 1;
stDstChnPort.u32ChnId = 0;
stDstChnPort.u32PortId = 0;
u32SrcFrmrate = 30;
u32DstFrmrate = 30;
eBindType = E_MI_SYS_BIND_TYPE_REALTIME;
u32BindParam = 0;
STCHECKRESULT(MI_SYS_BindChnPort2(u16SocId, &stSrcChnPort, &stDstChnPort,
u32SrcFrmrate, u32DstFrmrate, eBindType, u32BindParam));
```

// c. ISP 与 venc 连接方式为 E_MI_SYS_BIND_TYPE_HW_RING, 代码如下:

```
tSrcChnPort.eModId = E_MI_MODULE_ID_ISP;
stSrcChnPort.u32DevId = 0;
stSrcChnPort.u32ChnId = 0;
stSrcChnPort.u32PortId = 0;
stDstChnPort.eModId = E_MI_MODULE_ID_VENC;
stDstChnPort.u32DevId = 0;
stDstChnPort.u32ChnId = 0;
stDstChnPort.u32PortId = 0;
u32SrcFrmrate = 30;
u32DstFrmrate = 30;
eBindType = E_MI_SYS_BIND_TYPE_HW_RING;
u32BindParam = 1080; //假设 ISP output resolution 为 1920*1080, 设置 ring buffer depth 为
1080
```



```
STCHECKRESULT(MI_SYS_BindChnPort2(u16SocId, &stSrcChnPort, &stDstChnPort,
u32SrcFrmrate, u32DstFrmrate, eBindType, u32BindParam));
```

➤ 相关主题

[MI_SYS_BindChnPort2](#)、[MI_SYS_UnBind_ChnPort](#)。
[Bind 和帧率控制场景说明](#)。

2.4.3 MI_SYS_UnBind_ChnPort

➤ 功能

数据源输出端口到数据接收者输入端口之间的解绑定。

➤ 语法

```
MI_S32 MI_SYS_UnBindChnPort(MI_U16 u16SocId, MI_SYS_ChnPort_t *pstSrcChnPort,
MI_SYS_ChnPort_t *pstDstChnPort);
```

➤ 形参

参数名称	描述	输入/输出
u16SocId	指定执行本操作的目标 Soc	输入
pstSrcChnPort	源端口配置信息数据结构指针。	输入
pstDstChnPort	目标端口配置信息数据结构指针。	输入

➤ 返回值

- 0 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

※ 注意

- 源端口必须是通道输出端口。

- 目标端口必须是通道输入端口。
- 源和目标端口之间之前必须已经被绑定过

➤ 举例

N/A

➤ 相关主题

[MI_SYS_Bind_ChnPort](#)、[MI_SYS_BindChnPort2](#)。

2.4.4 MI_SYS_GetBindbyDest

➤ 功能

查询数据接收者输入端口的对应的源输出端口。

➤ 语法

MI_S32 MI_SYS_GetBindbyDest (MI_U16 u16SocId, [MI_SYS_ChnPort_t](#) *pstDstChnPort, [MI_SYS_ChnPort_t](#) *pstSrcChnPort, [MI_SYS_BindType_e](#) *peBindType);

➤ 形参

参数名称	描述	输入/输出
u16SocId	指定执行本操作的目标 Soc	输入
pstDstChnPort	目标端口配置信息数据结构指针。	输入
pstSrcChnPort	源端口配置信息数据结构指针。	输出
peBindType	绑定模式	输出

➤ 返回值

- 0 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

※ 注意

- 目标端口必须是通道输入端口。
- 目标端口之前必须已经被绑定过

➤ 举例

N/A

➤ 相关主题

N/A

2.4.5 MI_SYS_ChnInputPortGetBuf

➤ 功能

分配通道 input 端口对应的 buf object。

➤ 语法

MI_S32 MI_SYS_ChnInputPortGetBuf ([MI_SYS_ChnPort_t](#) *pstChnPort, [MI_SYS_BufConf_t](#) *pstBufConf, [MI_SYS_BufInfo_t](#) *pstBufInfo, [MI_SYS_BUF_HANDLE](#) *phHandle , MI_S32 s32TimeOutMs);

➤ 形参

参数名称	描述	输入/输出
pstChnPort	指向模块通道之 input 端口的指针	输入
pstBufConf	待分配内存配置信息	输入
pstPortBuf	返回之 buf 指针	输出
phHandle	获取的 inputPort Buf 的 handle 句柄	输出
s32TimeOutMs	等待超时的毫秒数，>=0 时为具体时间；<0 时会采用默认值 20ms	输入

➤ 返回值

- 0 成功。

- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

※ 注意

N/A

➤ 举例

MI_SYS_ChnInputPortGetBuf 调用 Sample

```
MI_SYS_ChnPort_t stIspChnInput;
MI_SYS_BUF_HANDLE hHandle = 0;
MI_SYS_BufConf_t stBufConf;
MI_SYS_BufInfo_t stBufInfo;
struct timeval stTv;
MI_U16 u16Width = 1920, u16Height = 1080;
FILE *fp = NULL;

memset(&stIspChnInput, 0x0, sizeof(MI_SYS_ChnPort_t));
memset(&stBufConf, 0x0, sizeof(MI_SYS_BufConf_t));
memset(&stBufInfo, 0x0, sizeof(MI_SYS_BufInfo_t));

stIspChnInput.eModId = E_MI_MODULE_ID_ISP;
stIspChnInput.u32DevId = 0;
stIspChnInput.u32ChnId = 0;
stIspChnInput.u32PortId = 0;

fp = fopen("/mnt/ispport0_1920x1080_pixel0_737.raw", "rb");
if(fp == NULL)
{
    printf("file %s open fail\n", "/mnt/ispport0_1920x1080_pixel0_737.raw");
    return 0;
}
```

```

    }

    while(1)
    {
        stBufConf.eBufType = E_MI_SYS_BUFDATA_FRAME;
        gettimeofday(&stTv, NULL);
        stBufConf.u64TargetPts = stTv.tv_sec*1000000 + stTv.tv_usec;
        stBufConf.stFrameCfg.eFormat = E_MI_SYS_PIXEL_FRAME_YUV422_YUYV;
        stBufConf.stFrameCfg.eFrameScanMode = E_MI_SYS_FRAME_SCAN_MODE_PROGRESSIVE;
        stBufConf.stFrameCfg.u16Width = u16Width;
        stBufConf.stFrameCfg.u16Height = u16Height;

        if(MI_SUCCESS ==
MI_SYS_ChnInputPortGetBuf(&stIspChnInput,&stBufConf,&stBufInfo,&hHandle,0))
        {
            if(fread(stBufInfo.stFrameData.pVirAddr[0], u16Width*u16Height*2, 1, fp) <= 0)
            {
                fseek(fp, 0, SEEK_SET);
            }

            MI_SYS_ChnInputPortPutBuf(hHandle,&stBufInfo, FALSE);
        }
    }
}

```

➤ 相关主题

[MI_SYS_ChnInputPortPutBuf](#) / [MI_SYS_ChnPortInjectBuf](#)

2.4.6 MI_SYS_ChnInputPortPutBuf

➤ 功能

把通道 input 端口对应的 buf object 加到待处理队列。

➤ 语法

```
MI_S32 MI_SYS_ChnInputPortPutBuf (MI_SYS_BUF_HANDLE hHandle,  
MI\_SYS\_BufInfo\_t *pstBufInfo, MI_BOOL bDropBuf);
```

➤ 形参

参数名称	描述	输入/输出
hHandle	当前 buf 的 Handle 句柄	输入
pstBufInfo	待提交之 buf 指针	输入
bDropBuf	是否将 buf 直接丢弃而不用提交到待处理队列上	输入

➤ 返回值

- 0 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

※ 注意

N/A

➤ 举例

参见 [MI_SYS_ChnInputPortGetBuf 调用 Sample](#) 举例。

➤ 相关主题

[MI_SYS_ChnInputPortGetBuf](#) / [MI_SYS_ChnPortInjectBuf](#)

2.4.7 MI_SYS_ChnOutputPortGetBuf

➤ 功能

分配通道 output 端口对应的 buf object。

➤ 语法

```
MI_S32 MI_SYS_ChnOutputPortGetBuf (MI\_SYS\_ChnPort\_t *pstChnPort,  
MI\_SYS\_BufInfo\_t *pstBufInfo, MI_SYS_BUF_HANDLE *phHandle);
```

形参

参数名称	描述	输入/输出
pstChnPort	指向模块通道之 output 端口的指针	输入
pstBufInfo	返回之 buf 指针	输出
phHandle	获取的 outputPort Buf 的 handle 句柄	输出

➤ 返回值

- 0 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

※ 注意

N/A

➤ 举例

MI_SYS_ChnOutputPortGetBuf 调用 Sample

```
MI_SYS_ChnPort_t stChnPort;
MI_SYS_BufInfo_t stBufInfo;
MI_SYS_BUF_HANDLE stBufHandle;
MI_S32 s32Ret = MI_SUCCESS;
MI_S32 s32Fd = 0;
fd_set read_fds;
struct timeval TimeoutVal;
char szFileName[128];
int fd = 0;
MI_U32 u32GetFramesCount = 0;
MI_BOOL _bWriteFile = TRUE;
MI_U16 u16SocId = ST_DEFAULT_SOC_ID;

stChnPort.eModId = E_MI_MODULE_ID_SCL;
```



```
stChnPort.u32DevId = 0;
stChnPort.u32ChnId = SCL_CHN_FOR_VDF;
stChnPort.u32PortId = 0;

s32Ret = MI_SYS_GetFd(&stChnPort, &s32Fd);
if(MI_SUCCESS != s32Ret)
{
    ST_ERR("MI_SYS_GetFd 0, error, %X\n", s32Ret);
    return NULL;
}
s32Ret = MI_SYS_SetChnOutputPortDepth(u16SocId, &stChnPort, 2, 3);
if (MI_SUCCESS != s32Ret)
{
    ST_ERR("MI_SYS_SetChnOutputPortDepth err:%x, chn:%d,port:%d\n", s32Ret,
        stChnPort.u32ChnId, stChnPort.u32PortId);
    return NULL;
}
s32Ret = MI_SYS_SetChnOutputPortUserFrc(&stChnPort, 15);
if (MI_SUCCESS != s32Ret)
{
    ST_ERR("MI_SYS_SetChnOutputPortUserFrc err:%x, chn:%d,port:%d\n", s32Ret,
        stChnPort.u32ChnId, stChnPort.u32PortId);
    return NULL;
}
sprintf(szFileName, "scl%d.es", stChnPort.u32ChnId);
printf("start to record %s\n", szFileName);
fd = open(szFileName, O_RDWR | O_CREAT | O_TRUNC, S_IRUSR | S_IWUSR | S_IRGRP |
S_IROTH);
if (fd < 0)
{
    ST_ERR("create %s fail\n", szFileName);
}
```

```
while (1)
{
    FD_ZERO(&read_fds);
    FD_SET(s32Fd, &read_fds);

    TimeoutVal.tv_sec = 1;
    TimeoutVal.tv_usec = 0;

    s32Ret = select(s32Fd + 1, &read_fds, NULL, NULL, &TimeoutVal);

    if(s32Ret < 0)
    {
        ST_ERR("select failed!\n");
        // usleep(10 * 1000);
        continue;
    }
    else if(s32Ret == 0)
    {
        ST_ERR("get scl frame time out\n");
        //usleep(10 * 1000);
        continue;
    }
    else
    {
        if(FD_ISSET(s32Fd, &read_fds))
        {
            s32Ret = MI_SYS_ChnOutputPortGetBuf(&stChnPort, &stBufInfo, &stBufHandle);

            if(MI_SUCCESS != s32Ret)
            {
                //ST_ERR("MI_SYS_ChnOutputPortGetBuf err, %x\n", s32Ret);
                continue;
            }
        }
    }
}
```

```
// save one Frame YUV data
if (fd > 0)
{
    if(!_bWriteFile)
    {
        write(fd, stBufInfo.stFrameData.pVirAddr[0], stBufInfo.stFrameData.u16Height
* stBufInfo.stFrameData.u32Stride[0] +
        stBufInfo.stFrameData.u16Height * stBufInfo.stFrameData.u32Stride[1] / 2);
    }
}

++u32GetFramesCount;
printf("channelId[%u] u32GetFramesCount[%u]\n", stChnPort.u32ChnId,
u32GetFramesCount);

MI_SYS_ChnOutputPortPutBuf(stBufHandle);
}
}
}

if (fd > 0)
{
    close(fd);
    fd = -1;
}

MI_SYS_SetChnOutputPortDepth(u16SocId, &stChnPort, 0, 3);
printf("exit record\n");
return NULL;
```

➤ 相关主题

[MI_SYS_ChnOutputPortPutBuf](#) / [MI_SYS_ChnPortInjectBuf](#)

2.4.8 MI_SYS_ChnOutputPortPutBuf

➤ 功能

释放通道 output 端口对应的 buf object。

➤ 语法

MI_S32 MI_SYS_ChnOutputPortPutBuf (MI_SYS_BUF_HANDLE hBufHandle);

➤ 形参

参数名称	描述	输入/输出
hBufHandle	待提交之 buf 的 handle 句柄	输入

➤ 返回值

- 0 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

※ 注意

N/A

➤ 举例

参见 [MI_SYS_ChnOutputPortGetBuf](#) 调用 Sample 举例。

➤ 相关主题

[MI_SYS_ChnOutputPortGetBuf](#) / [MI_SYS_ChnPortInjectBuf](#)

2.4.9 MI_SYS_ChnlInputPortGetBufPa

➤ 功能

分配通道 input 端口对应的 buf object，仅提供 MIU 物理地址。

➤ 语法

```
MI_S32 MI_SYS_ChnlInputPortGetBufPa (MI_SYS_ChnlPort_t
*pstChnlPort, MI_SYS_BufConf_t *pstBufConf, MI_SYS_BufInfo_t *pstBufInfo,
MI_SYS_BUF_HANDLE *phHandle , MI_S32 s32TimeOutMs);
```

➤ 形参

参数名称	描述	输入/输出
pstChnlPort	指向模块通道之 input 端口的指针	输入
pstBufConf	待分配内存配置信息	输入
pstPortBuf	返回之 buf 指针	输出
phHandle	获取的 inputPort Buf 的 handle 句柄	输出
s32TimeOutMs	等待超时的毫秒数，>=0 时为具体时间；<0 时会采用默认值 20ms	输入

➤ 返回值

- 0 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

※ 注意

- 与 [MI_SYS_ChnlInputPortGetBuf](#) 区别：[MI_SYS_ChnlInputPortGetBufPa](#) 获取的 pstBufInfo 中只能使用物理地址 phyAddr，而 [MI_SYS_ChnlInputPortGetBuf](#) 获取的 pstBufInfo 中物理地址 phyAddr 和虚拟地址 pVirAddr 都可以使用。

➤ 举例

MI_SYS_ChnInputPortGetBufPa 调用 Sample

```

MI_SYS_ChnPort_t stIspChnInput;
MI_SYS_BUF_HANDLE hHandle = 0;
MI_SYS_BufConf_t stBufConf;
MI_SYS_BufInfo_t stBufInfo;
struct timeval stTv;
MI_U16 u16Width = 1920, u16Height = 1080;
FILE *fp = NULL;
void *pVirAddr = NULL;
MI_U16 u16SocId = ST_DEFAULT_SOC_ID;

memset(&stIspChnInput, 0x0, sizeof(MI_SYS_ChnPort_t));
memset(&stBufConf, 0x0, sizeof(MI_SYS_BufConf_t));
memset(&stBufInfo, 0x0, sizeof(MI_SYS_BufInfo_t));

stIspChnInput.eModId = E_MI_MODULE_ID_ISP;
stIspChnInput.u32DevId = 0;
stIspChnInput.u32ChnId = 0;
stIspChnInput.u32PortId = 0;

fp = fopen("/mnt/ispport0_1920x1080_pixel0_737.raw", "rb");
if (fp == NULL)
{
    printf("file %s open fail\n", "/mnt/ispport0_1920x1080_pixel0_737.raw");
    return 0;
}

while (1)
{
    stBufConf.eBufType = E_MI_SYS_BUFDATA_FRAME;
    gettimeofday(&stTv, NULL);
    stBufConf.u64TargetPts = stTv.tv_sec * 1000000 + stTv.tv_usec;
    stBufConf.stFrameCfg.eFormat = E_MI_SYS_PIXEL_FRAME_YUV422_YUYV;

```

```

stBufConf.stFrameCfg.eFrameScanMode = E_MI_SYS_FRAME_SCAN_MODE_PROGRESSIVE;
stBufConf.stFrameCfg.u16Width = u16Width;
stBufConf.stFrameCfg.u16Height = u16Height;

if (MI_SUCCESS == MI_SYS_ChnInputPortGetBufPa(&stIspChnInput, &stBufConf,
&stBufInfo, &hHandle, 0))
{
    pVirAddr = MI_SYS_Mmap(stBufInfo.stFrameData.phyAddr[0],
stBufInfo.stFrameData.u32BufSize, &pVirAddr, TRUE);
    assert(pVirAddr);
    if (fread(pVirAddr, u16Width * u16Height * 2, 1, fp) <= 0)
    {
        fseek(fp, 0, SEEK_SET);
    }
    MI_SYS_Munmap(pVirAddr, stBufInfo.stFrameData.u32BufSize);
    MI_SYS_ChnInputPortPutBufPa(hHandle, &stBufInfo, FALSE);
}
}

```

➤ 相关主题

[MI_SYS_ChnInputPortPutBufPa](#) / [MI_SYS_ChnInputPortPutBuf](#) /
[MI_SYS_ChnPortInjectBuf](#)

2.4.10 MI_SYS_ChnInputPortPutBufPa

➤ 功能

把通道 input 端口对应的 buf object 加到待处理队列，与 MI_SYS_ChnInputPortGetBufPa 成对使用。

➤ 语法

MI_S32 MI_SYS_ChnInputPortPutBufPa (MI_SYS_BUF_HANDLE hHandle,
[MI_SYS_BufInfo_t](#) *pstPortBuf, MI_BOOL bDropBuf);

➤ 形参

参数名称	描述	输入/输出
hHandle	当前 buf 的 Handle 句柄	输入
pstPortBuf	待提交之 buf 指针	输入
bDropBuf	直接放弃对 buf 的修改不提交	输入

➤ 返回值

- 0 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

※ 注意

- 与 [MI_SYS_ChnlInputPortPutBuf](#) 区别：[MI_SYS_ChnlInputPortPutBufPa](#) 提交的 pstBufInfo 中只能使用物理地址 phyAddr，而 [MI_SYS_ChnlInputPortPutBuf](#) 提交的 pstBufInfo 中物理地址 phyAddr 和虚拟地址 pVirAddr 都可以使用。

➤ 举例

参见 [MI_SYS_ChnlInputPortGetBufPa 调用 Sample](#) 举例。

➤ 相关主题

[MI_SYS_ChnlInputPortGetBufPa](#) / [MI_SYS_ChnlInputPortGetBuf](#) / [MI_SYS_ChnlPortInjectBuf](#)

2.4.11 MI_SYS_ChnlOutputPortGetBufPa

➤ 功能

分配通道 output 端口对应的 buf object，仅提供 MIU 物理地址。

➤ 语法

MI_S32 MI_SYS_ChnOutputPortGetBufPa ([MI_SYS_ChnPort_t](#) *pstChnPort,
[MI_SYS_BufInfo_t](#) *pstBufInfo, MI_SYS_BUF_HANDLE *phHandle);

➤ 形参

参数名称	描述	输入/输出
pstChnPort	指向模块通道之 output 端口的指针	输入
pstBufInfo	返回之 buf 指针	输出
phHandle	获取的 outputPort Buf 的 handle 句柄	输出

➤ 返回值

- 0 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

※ 注意

- 与 [MI_SYS_ChnOutputPortGetBuf](#) 区别：[MI_SYS_ChnOutputPortGetBufPa](#) 获取的 pstBufInfo 中只能使用物理地址 phyAddr，而 [MI_SYS_ChnOutputPortGetBuf](#) 获取的 pstBufInfo 中物理地址 phyAddr 和虚拟地址 pVirAddr 都可以使用。

➤ 举例

MI_SYS_ChnOutputPortGetBufPa 调用 Sample

```
MI_SYS_ChnPort_t stChnPort;
MI_SYS_BufInfo_t stBufInfo;
MI_SYS_BUF_HANDLE stBufHandle;
MI_S32 s32Ret = MI_SUCCESS;
MI_S32 s32Fd = 0;
fd_set read_fds;
struct timeval TimeoutVal;
char szFileName[128];
```

```

int fd = 0;
MI_U32 u32GetFramesCount = 0;
MI_BOOL _bWriteFile = TRUE;
void *pVirAddr = NULL;
MI_U16 u16SocId = ST_DEFAULT_SOC_ID;

stChnPort.eModId = E_MI_MODULE_ID_SCL;
stChnPort.u32DevId = 0;
stChnPort.u32ChnId = SCL_CHN_FOR_VDF;
stChnPort.u32PortId = 0;

s32Ret = MI_SYS_GetFd(&stChnPort, &s32Fd);
if (MI_SUCCESS != s32Ret)
{
    ST_ERR("MI_SYS_GetFd 0, error, %X\n", s32Ret);
    return NULL;
}
s32Ret = MI_SYS_SetChnOutputPortDepth(u16SocId, &stChnPort, 2, 3);
if (MI_SUCCESS != s32Ret)
{
    ST_ERR("MI_SYS_SetChnOutputPortDepth err:%x, chn:%d,port:%d\n", s32Ret,
        stChnPort.u32ChnId, stChnPort.u32PortId);
    return NULL;
}

sprintf(szFileName, "scl%d.es", stChnPort.u32ChnId);
printf("start to record %s\n", szFileName);
fd = open(szFileName, O_RDWR | O_CREAT | O_TRUNC, S_IRUSR | S_IWUSR | S_IRGRP |
S_IROTH);
if (fd < 0)
{
    ST_ERR("create %s fail\n", szFileName);
}

while (1)
{
    FD_ZERO(&read_fds);
    FD_SET(s32Fd, &read_fds);

    TimeoutVal.tv_sec = 1;
    TimeoutVal.tv_usec = 0;

```

```

s32Ret = select(s32Fd + 1, &read_fds, NULL, NULL, &TimeoutVal);

if (s32Ret < 0)
{
    ST_ERR("select failed!\n");
    // usleep(10 * 1000);
    continue;
}
else if (s32Ret == 0)
{
    ST_ERR("get scl frame time out\n");
    //usleep(10 * 1000);
    continue;
}
else
{
    if (FD_ISSET(s32Fd, &read_fds))
    {
        s32Ret = MI_SYS_ChnOutputPortGetBufPa(&stChnPort, &stBufInfo, &stBufHandle);

        if (MI_SUCCESS != s32Ret)
        {
            //ST_ERR("MI_SYS_ChnOutputPortGetBuf err, %x\n", s32Ret);
            continue;
        }

        pVirAddr = MI_SYS_Mmap(stBufInfo.stFrameData.phyAddr[0],
stBufInfo.stFrameData.u32BufSize, &pVirAddr, TRUE);
        assert(pVirAddr);
        // save one Frame YUV data
        if (fd > 0)
        {
            if (_bWriteFile)
            {
                write(fd, pVirAddr, stBufInfo.stFrameData.u16Height *
stBufInfo.stFrameData.u32Stride[0] + stBufInfo.stFrameData.u16Height *
stBufInfo.stFrameData.u32Stride[1] / 2);
            }
        }
    }
}

```

```

        ++u32GetFramesCount;
        printf("channelId[%u] u32GetFramesCount[%u]\n", stChnPort.u32ChnId,
u32GetFramesCount);

        MI_SYS_Munmap(pVirAddr, stBufInfo.stFrameData.u32BufSize);
        MI_SYS_ChnOutputPortPutBufPa(stBufHandle);
    }
}
}

if (fd > 0)
{
    close(fd);
    fd = -1;
}

MI_SYS_SetChnOutputPortDepth(u16SocId, &stChnPort, 0, 3);
printf("exit record\n");
return NULL;

```

➤ 相关主题

[MI_SYS_ChnOutputPortPutBufPa](#) / [MI_SYS_ChnOutputPortPutBuf](#) / [MI_SYS_ChnPortInjectBuf](#)

2.4.12 MI_SYS_ChnOutputPortPutBufPa

➤ 功能

释放通道 output 端口对应的 buf object，与 [MI_SYS_ChnOutputPortGetBufPa](#) 成对使用。

➤ 语法

MI_S32 MI_SYS_ChnOutputPortPutBufPa (MI_SYS_BUF_HANDLE hBufHandle);

➤ 形参

参数名称	描述	输入/输出
hBufHandle	待提交之 buf 的 handle 句柄	输入

➤ 返回值

- 0 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

※ 注意

- 与 [MI_SYS_ChOutputPortPutBuf](#) 区别：[MI_SYS_ChOutputPortPutBufPa](#) 提交的 handle 对应的 pstBufInfo 中只能使用物理地址 phyAddr，而 [MI_SYS_ChOutputPortPutBuf](#) 提交的 handle 对应的 pstBufInfo 中物理地址 phyAddr 和虚拟地址 pVirAddr 都可以使用。

➤ 举例

参见 [MI_SYS_ChOutputPortGetBufPa](#) 调用 [Sample](#) 举例。

➤ 相关主题

[MI_SYS_ChOutputPortGetBufPa](#) / [MI_SYS_ChOutputPortGetBuf](#) /
[MI_SYS_ChPortInjectBuf](#)

2.4.13 MI_SYS_SetChnOutputPortDepth

➤ 功能

设置通道 output 端口对应的系统 buf 数量和用户可以拿到的 buf 数量。

➤ 语法

MI_S32 MI_SYS_SetChnOutputPortDepth(MI_U16 u16SocId, [MI_SYS_ChnPort_t](#)
*pstChnPort, MI_U32 u32UserFrameDepth, MI_U32 u32BufQueueDepth);

➤ 形参

参数名称	描述	输入/输出
------	----	-------

u16SocId	指定执行本操作的目标 Soc	输入
pstChnPort	指向模块通道之 output 端口的指针	输入
u32UserFrameDepth	设置该 output 用户可以拿到的 buf 最大数量	输入
u32BufQueueDepth	设置该 output 系统 buf 最大数量	输入

➤ 返回值

- 0 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

※ 注意

- u32UserFrameDepth 默认为 0, u32BufQueueDepth 默认为 4。
- 在使用完 OutputPortBuf（[MI_SYS_ChOutputPortGetBuf](#) / [MI_SYS_ChOutputPortPutBuf](#) 不再需要被调用）之后，建议把 u32UserFrameDepth 设置为 0，可以减少 output buf 的缓存个数，节省内存。
- u32UserFrameDepth 必须小于或者等于 u32BufQueueDepth。
- 当 u32UserFrameDepth 等于 u32BufQueueDepth，并且 user fifo queue 缓存的个数等于 u32BufQueueDepth 时，当前 output port 将会停止工作。
- 如果 output port 没有绑定后级，想让当前 output port 工作，必须设置 u32UserFrameDepth 大于 0。
- 如果 output port 通过 realtime mode 或者 ring mode 方式绑定，当前 output port 的 u32UserFrameDepth 必须设置为 0。其他使用限制可参考 vif/isp/scl/venc/jpd API 文档介绍。

➤ 举例

参见 [MI_SYS_ChOutputPortGetBuf 调用 Sample](#) 举例。

➤ 相关主题

N/A

2.4.14 MI_SYS_ChnPortInjectBuf

➤ 功能

把获取的 outputPort buf 插到指定的 inputPort Buf Queue 中

➤ 语法

MI_S32 MI_SYS_ChnPortInjectBuf(MI_SYS_BUF_HANDLE hHandle, [MI_SYS_ChnPort_t](#) * pstChnPort);

➤ 形参

参数名称	描述	输入/输出
hHandle	获取的 outputPort Buf 的 handle 句柄	输入
pstChnPort	指向模块通道之 input 端口的指针	输入

➤ 返回值

- 0 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

※ 注意

- 调用 [MI_SYS_ChnPortInjectBuf](#) 后，user 不能再显式的 release handle 了。
- 一般不会调用 [MI_SYS_ChnInputPortGetBuf](#) 拿到 buf，然后调用 [MI_SYS_ChnPortInjectBuf](#) 给到指定的 inputPort Buf Queue。

➤ 举例

参见 [MI_SYS_ChnOutputPortGetBuf](#) 调用 [Sample](#) 举例。

➤ 相关主题

[MI_SYS_ChnInputPortGetBuf](#) / [MI_SYS_ChnInputPortPutBuf](#) /

[MI_SYS_ChnOutputPortGetBuf](#) / [MI_SYS_ChnOutputPortPutBuf](#)

2.4.15 MI_SYS_GetFd

➤ 功能

获取当前 output Port 等待事件的文件描述符

➤ 语法

```
MI_S32 MI_SYS_GetFd(MI\_SYS\_ChnPort\_t *pstChnPort, MI_S32 *ps32Fd);
```

➤ 形参

参数名称	描述	输入/输出
pstChnPort	端口信息结构体指针	输入
ps32Fd	等待事件的文件描述符	输出

➤ 返回值

- 0 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

※ 注意

- 需要与 [MI_SYS_CloseFd](#) 成对使用。
- 推荐使用使用 fd + select 的方式取数据，这样 MI_SYS 只有在 fd 对应的 port 口有数据的时候才会唤醒线程，效率比采用 while+sleep 循环取数据的效率要高。

➤ 举例

参见 [MI_SYS_ChnOutputPortGetBuf 调用 Sample](#) 举例。

➤ 相关主题

[MI_SYS_CloseFd](#)

2.4.16 MI_SYS_CloseFd

➤ 功能

关闭 Output Port 对应的文件描述符

➤ 语法

```
MI_S32 MI_SYS_CloseFd(MI_S32 s32ChnPortFd);
```

➤ 形参

参数名称	描述	输入/输出
s32ChnPortFd	等待事件的文件描述符	输入

➤ 返回值

- 0 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

※ 注意

需要与 [MI_SYS_GetFd](#) 成对使用。

➤ 举例

参见 [MI_SYS_ChOutputPortGetBuf](#) 调用 [Sample](#) 举例。

➤ 相关主题

[MI_SYS_GetFd](#)

2.4.17 MI_SYS_DupBuf

➤ 功能

复制一个 buf object。

➤ 语法

```
MI_S32 MI_SYS_DupBuf(MI_SYS_BUF_HANDLE srcBufHandle , MI_SYS_BUF_HANDLE
*pDupTargetBufHandle);
```

➤ 形参

参数名称	描述	输入/输出
srcBufHandle	源 buf object	输入
pDupTargetBufHandle	返回复制的 buf object 指针	输出

➤ 返回值

- 0 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

※ 注意

N/A

➤ 举例

MI_SYS_DupBuf 调用 Sample

```
int test0()
{
    MI_SYS_ChnPort_t stChnPort;
    MI_SYS_BufInfo_t stBufInfo;
    MI_SYS_BUF_HANDLE bufHandle ,DupTargetBufHandle;
    MI_S32 s32Ret;
```

```

stChnPort.eModId = E_MI_MODULE_ID_ISP;
stChnPort.u32DevId = 0;
stChnPort.u32ChnId = 0;
stChnPort.u32PortId = 0;
s32Ret = MI_SYS_ChnOutputPortGetBuf (&stChnPort,&stBufInfo,&bufHandle);
if(s32Ret != MI_SUCCESS)
    return 0;
s32Ret = MI_SYS_DupBuf(bufHandle, &DupTargetBufHandle);
if(s32Ret != MI_SUCCESS)
{
    MI_SYS_ChnOutputPortPutBuf(bufHandle);
    return 0;
}
stChnPort.eModId = E_MI_MODULE_ID_SCL;
stChnPort.u32DevId = 0;
stChnPort.u32ChnId = 0;
stChnPort.u32PortId = 0;
MI_SYS_ChnPortInjectBuf(DupTargetBufHandle , &stChnPort);

/*****
    可以继续操作 stBufInfo
*****/

MI_SYS_ChnOutputPortPutBuf(bufHandle);
return 1;
}

int test1()
{
    MI_SYS_ChnPort_t stChnPort;
    MI_SYS_BufInfo_t stBufInfo;
    MI_SYS_BufConf_t stBufConf;
    MI_SYS_BUF_HANDLE bufHandle ,DupTargetBufHandle;

```

```

MI_S32 s32Ret;

stChnPort.eModId = E_MI_MODULE_ID_SCL;
stChnPort.u32DevId = 0;
stChnPort.u32ChnId = 0;
stChnPort.u32PortId = 0;
memset(&stBufConf, 0, sizeof(stBufConf));
stBufConf.eBufType = E_MI_SYS_BUFDATA_FRAME;
stBufConf.stFrameCfg.eFormat = E_MI_SYS_PIXEL_FRAME_YUV_SEMIPLANAR_422;
stBufConf.stFrameCfg.u16Height = 1080;
stBufConf.stFrameCfg.u16Width = 1920;
stBufConf.stFrameCfg.eFrameScanMode = E_MI_SYS_FRAME_SCAN_MODE_PROGRESSIVE;
s32Ret = MI_SYS_ChnInputPortGetBuf (&stChnPort,&stBufConf,&stBufInfo,&bufHandle,0);
if(s32Ret != MI_SUCCESS)
    return 0;

/*****
    填充 stBufInfo
*****/

s32Ret = MI_SYS_DupBuf(bufHandle, &DupTargetBufHandle);
if(s32Ret != MI_SUCCESS)
{
    MI_SYS_ChnInputPortPutBuf(bufHandle);
    return 0;
}

MI_SYS_ChnInputPortPutBuf(DupTargetBufHandle, &stChnPort, FALSE);

/*****
    可以继续操作 stBufInfo
*****/

return 1;
}

```

➤ 相关主题

[MI_SYS_ChnOutputPortGetBuf/MI_SYS_ChnOutputPortPutBuf /](#)

[MI_SYS_ChnInputPortGetBuf/MI_SYS_ChnInputPortPutBuf/MI_SYS_ChnPortInjectBuf](#)

2.4.18 [MI_SYS_ChnInputPortSetUserPicture](#)

➤ 功能

按照设定帧率驱动会重复输入该 buf 到当前 input port，并加到待处理队列。

➤ 语法

```
MI_S32 MI_SYS_ChnInputPortSetUserPicture(MI_SYS_BUF_HANDLE BufHandle ,
MI_SYS_BufInfo_t *pstBufInfo, MI_SYS_UserPictureInfo_t *pstUserPictureInfo);
```

➤ 形参

参数名称	描述	输入/输出
BufHandle	当前 buf 的 Handle 句柄	输入
pstBufInfo	待提交的 buf 指针	输入
pstUserPictureInfo	向驱动设定帧率信息的指针	输入

➤ 返回值

- 0 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

※ 注意

开启 user picture 功能的 Input port 不能被绑定且需要被开启。

➤ 举例

MI_SYS_ChnInputPortSetUserPicture 调用 Sample

```
int test0()
{
```

```

MI_SYS_ChnPort_t stChnPort;
MI_SYS_BufConf_t stBufConf;
MI_SYS_BufInfo_t stBufInfo;
MI_SYS_BUF_HANDLE bufHandle;
MI\_SYS\_UserPictureInfo\_t stUserPictureInfo;
memset(&stChnPort, 0, sizeof(stChnPort));
memset(&stBufConf, 0, sizeof(stBufConf));
memset(&stBufInfo, 0, sizeof(stBufInfo));

stChnPort.eModId = E_MI_MODULE_ID_ISP;
stChnPort.u32DevId = 0;
stChnPort.u32ChnId = 0;
stChnPort.u32PortId = 0;
stBufConf.eBufType = E_MI_SYS_BUFDATA_FRAME;
stBufConf.stFrameCfg.u16Width = 1920;
stBufConf.stFrameCfg.u16Height = 1080;
stBufConf.stFrameCfg.eFormat = E_MI_SYS_PIXEL_FRAME_YUV422_YUYV;

MI\_SYS\_ChnInputPortGetBuf(&stChnPort, &stBufConf, &stBufInfo, &bufHandle, 100);
/*****
    向 stBufInfo 里填充相应数据
*****/

stUserPictureInfo.u32SrcFrc = 30;//每秒 30 帧
MI\_SYS\_ChnInputPortSetUserPicture(bufHandle, &stBufInfo, &stUserPictureInfo);
MI\_SYS\_EnableUserPicture(bufHandle);

.....

.....

.....

MI\_SYS\_DisableUserPicture(bufHandle);
return 0;
}

```

➤ 相关主题

[MI_SYS_ChnInputPortGetBuf/MI_SYS_ChnInputPortSetUserPicture](#) /
[MI_SYS_EnableUserPicture/ MI_SYS_DisableUserPicture](#)

2.4.19 MI_SYS_EnableUserPicture

➤ 功能

打开 user picture 功能。

➤ 语法

```
MI_S32 MI_SYS_EnableUserPicture(MI_SYS_BUF_HANDLE BufHandle);
```

➤ 形参

参数名称	描述	输入/输出
BufHandle	通过 MI_SYS_ChnInputPortSetUserPicture 向驱动设定 bufinfo 的 Handle 句柄	输入

➤ 返回值

- 0 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

※ 注意

N/A

➤ 举例

参见 [MI_SYS_ChnInputPortSetUserPicture 调用 Sampl](#) 举例。

➤ 相关主题

[MI_SYS_ChnInputPortGetBuf/ MI_SYS_ChnInputPortSetUserPicture /](#)
[MI_SYS_EnableUserPicture/ MI_SYS_DisableUserPicture](#)

2.4.20 MI_SYS_DisableUserPicture

➤ 功能

关闭 user picture 功能,并释放 buf

➤ 语法

```
MI_S32 MI_SYS_DisableUserPicture (MI_SYS_BUF_HANDLE hHandle);
```

➤ 形参

参数名称	描述	输入/输出
BufHandle	通过 MI_SYS_ChnInputPortSetUserPicture 向驱动设定的 bufinfo 的 Handle 句柄	输入

➤ 返回值

- 0 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

※ 注意

N/A

➤ 举例

参见 [MI_SYS_ChnInputPortSetUserPicture 调用 Sampl](#) 举例。

➤ 相关主题

[MI_SYS_ChnInputPortGetBuf/MI_SYS_ChnInputPortSetUserPicture /](#)

[MI_SYS_EnableUserPicture/MI_SYS_DisableUserPicture](#)

2.4.21 MI_SYS_SetChnOutputPortUserFrc

➤ 功能

设置 **output** 输出帧率。

➤ 语法

```
MI_S32 MI_SYS_SetChnOutputPortUserFrc((MI\_SYS\_ChnPort\_t *pstChnPort, MI_U32 u32Frmrate);
```

➤ 形参

参数名称	描述	输入/输出
pstChnPort	端口信息结构体指针	输入
u32Frmrate	设定 out port buf 输入的帧率	输入

➤ 返回值

- 0 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

※ 注意

N/A

➤ 举例

参见 [MI_SYS_ChnOutputPortGetBuf](#) 调用 [Sample](#) 举例。

➤ 相关主题

[MI_SYS_ChnInputPortGetBuf/MI_SYS_ChnInputPortSetUserPicture](#) /
[MI_SYS_EnableUserPicture/MI_SYS_DisableUserPicture](#)

2.5. 内存管理类 API

2.5.1 MI_SYS_SetChnMMAConf

➤ 功能

设置模块设备通道输出默认分配内存的 MMA 池名称。

➤ 语法

```
MI_S32 MI_SYS_SetChnMMAConf (MI_U16 u16SocId, MI_ModuleId_e eModId, MI_U32 u32DevId, MI_U32 u32ChnId, MI_U8 *pu8MMAHeapName);
```

➤ 形参

参数名称	描述	输入/输出
u16SocId	指定执行本操作的目标 Soc	输入
eModId	待配置的模块 ID	输入
u32DevId	待配置的设备 ID	输入
u32ChnId	待配置的通道号	输入
pu8MMAHeapName	设置 MMA heap name	输入

➤ 返回值

- 0 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

※ 注意

- 若不调用 [MI_SYS_SetChnMMAConf](#) 接口设置或者设置 pu8MMAHeapName 值为 NULL 时，则使用默认的 MMA 池名称 mma_heap_name0。

➤ 举例

```
MI_ModuleId_e eVifModeId = E_MI_MODULE_ID_VIF;
MI_VIF_DEV vifDev = 0;
MI_VIF_CHN vifChn = 0;
MI_U16 u16SocId = ST_DEFAULT_SOC_ID;

MI_SYS_SetChnMMAConf(u16SocId, eVifModeId, vifDev, vifChn, "mma_heap_name0");
```

➤ 相关主题

N/A

2.5.2 MI_SYS_GetChnMMAConf

➤ 功能

获取模块设备通道输出端口默认分配内存的 MMA 池名称。

➤ 语法

MI_S32 MI_SYS_GetChnMMAConf (MI_U16 u16SocId, MI_ModuleId_e eModId, MI_U32 u32DevId, MI_U32 u32ChnId, void *pu8MMAHeapName, MI_U32 u32Length);

➤ 形参

参数名称	描述	输入/输出
u16SocId	指定执行本操作的目标 Soc	输入
eModId	待配置的模块 ID	输入
u32DevId	待配置的设备 ID	输入
u32ChnId	待配置的通道号	输入
pu8MMAHeapName	要获取的 MMA heap name	输出
u32Length	pu8MMAHeapName 指向内存的长度，最大为 64Byte	输入

➤ 返回值

- 0 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

※ 注意

N/A

➤ 举例

N/A

➤ 相关主题

[MI_SYS_SetChnMMAConf](#)

2.5.3 MI_SYS_MMA_Alloc

➤ 功能

直接向 MMA 内存管理器申请分配内存。

➤ 语法

```
MI_S32 MI_SYS_MMA_Alloc(MI_U16 u16SocId, MI_U8 *pstMMAHeapName, MI_U32 u32BlkSize ,MI_PHY *phyAddr);
```

➤ 形参

参数名称	描述	输入/输出
u16SocId	指定执行本操作的目标 Soc	输入
pstMMAHeapName	目标 MMA heapname	输入
u32BlkSize	待分配的块字节大小	输入
phyAddr	返回的内存块物理地址	输出

➤ 返回值

- 0 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件: mi_sys_datatype.h、mi_sys.h
- 库文件: libmi_sys.a / libmi_sys.so

※ 注意

pstMMAHeapName 为 NULL 时, 从默认 Heap 申请。

➤ 举例

MI_SYS_MMA_Alloc 调用 Sample

```
MI_PHY phySrcBufAddr = 0;
void *pVirSrcBufAddr = NULL;
MI_U32 srcBuffSize = 1920 * 1980 * 3 / 2;
MI_U16 u16SocId = ST_DEFAULT_SOC_ID;

srcBuffSize = ALIGN_UP(srcBuffSize, 4096);

ret = MI_SYS_MMA_Alloc(u16SocId, NULL, srcBuffSize, &phySrcBufAddr);
if(ret != MI_SUCCESS)
{
    printf("alloc src buff failed\n");
    return -1;
}

ret = MI_SYS_Mmap(phySrcBufAddr, srcBuffSize, &pVirSrcBufAddr, TRUE);
if(ret != MI_SUCCESS)
{
    MI_SYS_MMA_Free(u16SocId, phySrcBufAddr);
    printf("mmap src buff failed\n");
    return -1;
}

memset(pVirSrcBufAddr, 0, srcBuffSize);
MI_SYS_FlushInvCache(pVirSrcBufAddr, srcBuffSize);
MI_SYS_Munmap(pVirSrcBufAddr, srcBuffSize);
MI_SYS_MMA_Free(u16SocId, phySrcBufAddr);
```

➤ 相关主题

[MI_SYS_MMA_Free](#) / [MI_SYS_Mmap](#) / [MI_SYS_FlushCache](#) / [MI_SYS_Munmap](#)

2.5.4 MI_SYS_MMA_Free

➤ 功能

直接向 MMA 内存管理器释放之前分配的内存。

➤ 语法

```
MI_S32 MI_SYS_MMA_Free(MI_U16 u16SocId, MI_U64 phyAddr);
```

➤ 形参

参数名称	描述	输入/输出
u16SocId	指定执行本操作的目标 Soc	输入
phyAddr	待释放之内存的物理地址	输入

➤ 返回值

- 0 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

※ 注意

N/A

➤ 举例

参见 [MI_SYS_MMA_Alloc](#) 调用 [Sample](#) 举例。

➤ 相关主题

[MI_SYS_MMA_Alloc](#) / [MI_SYS_Mmap](#) / [MI_SYS_FlushCache](#) / [MI_SYS_Munmap](#)

2.5.5 MI_SYS_Mmap

➤ 功能

映射任意物理内存到当前用户态进程的 CPU 虚拟地址空间。

➤ 语法

```
MI_S32 MI_SYS_Mmap(MI_U64 u64PhyAddr, MI_U32 u32Size, void  
**ppVirtualAddress, MI_BOOL bCache);
```

➤ 形参

参数名称	描述	输入/输出
u64PhyAddr	待映射的物理地址	输入
u32Size	待映射的物理地址长度	输入
ppVirtualAddress	存储 CPU 虚拟地址指针的指针	输出
bCache	是否 map 成 cache 还是 un-cache	输入

➤ 返回值

- 0 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

※ 注意

- 注意物理地址是 SStar 内存控制器地址。
- 物理地址建议 4KByte 对齐。
- 物理地址长度建议 4KByte 对齐。
- 物理内存必须完整的落在 MMA 管理的内存范围内或者 linux kernel 管理的内存之外。

➤ 举例

参见 [MI_SYS_MMA_Alloc 调用 Sample](#) 举例。

➤ 相关主题

[MI_SYS_MMA_Alloc](#) / [MI_SYS_MMA_Free](#) / [MI_SYS_FlushCache](#) / [MI_SYS_Munmap](#)

2.5.6 MI_SYS_FlushInvCache

➤ 功能

Flush cache。

➤ 语法

```
MI_S32 MI_SYS_FlushInvCache(MI_VOID *pVirtualAddress, MI_U32 u32Size);
```

➤ 形参

参数名称	描述	输入/输出
pVirtualAddress	之前 MI_SYS_Mmap 返回的 CPU 虚拟地址	输入
u32Size	待 flush 的 cache 长度	输入

➤ 返回值

- 0 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

※ 注意

- 待 flush cache 的物理地址建议 4KByte 对齐。
- 待 flush cache 的映射长度建议 4KByte 对齐。
- 待 flush cache 的映射内存范围必须是之前通过 MI_SYS_Mmap API 获取到的。
- 待 flush cache 的映射内存应该是 MI_SYS_Mmap 时采用 cache 方式进行的，nocache 的方式无需 flush cache。

➤ 举例

参见 [MI_SYS_MMA_Alloc 调用 Sample](#) 举例。

➤ 相关主题

[MI_SYS_MMA_Alloc](#) / [MI_SYS_MMA_Free](#) / [MI_SYS_Mmap](#) / [MI_SYS_Munmap](#)

2.5.7 MI_SYS_Munmap

➤ 功能

取消物理内存到虚拟地址的映射。

➤ 语法

```
MI_S32 MI_SYS_Munmap(MI_VOID *pVirtualAddress, MI_U32 u32Size);
```

➤ 形参

参数名称	描述	输入/输出
pVirtualAddress	之前 MI_SYS_Mmap 返回的 CPU 虚拟地址	输入
u32Size	待取消映射的映射长度	输入

➤ 返回值

- 0 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

※ 注意

- 待取消映射的虚拟地址建议 4KByte 对齐。
- 待取消映射的映射长度建议 4KByte 对齐
- 待取消的映射内存范围必须是之前通过 MI_SYS_Mmap API 获取到的。

➤ 举例

参见 [MI_SYS_MMA_Alloc 调用 Sample](#) 举例。

➤ 相关主题

[MI_SYS_MMA_Alloc](#) / [MI_SYS_MMA_Free](#) / [MI_SYS_Mmap](#) / [MI_SYS_FlushCache](#)

2.5.8 MI_SYS_MemsetPa

➤ 功能

通过 DMA 硬件模块，填充整块物理内存。

➤ 语法

```
MI_S32 MI_SYS_MemsetPa(MI_U16 u16SocId, MI_PHY phyPa, MI_U32 u32Val,
MI_U32 u32Lenth);
```

➤ 形参

参数名称	参数含义	输入/输出
u16SocId	指定执行本操作的目标 Soc	输入
phyPa	填充的物理地址	输入
u32Val	填充值（注意是 4 字节）	输入
u32Lenth	填充大小，单位为 byte	输入

➤ 返回值

- 0 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

※ 注意

N/A

➤ 举例

MI_SYS_MemsetPa 调用 Sample

```
MI_PHY phySrcBufAddr = 0;
MI_PHY phyDstBufAddr = 0;
void *pVirSrcBufAddr = NULL;
void *pVirDstBufAddr = NULL;
MI_U32 buffSize = 1920 * 1980 * 3 / 2;
MI_U16 u16SocId = ST_DEFAULT_SOC_ID;

buffSize = ALIGN_UP(buffSize, 4096);

ret = MI_SYS_MMA_Alloc(u16SocId, NULL, buffSize, &phySrcBufAddr);
if(ret != MI_SUCCESS)
{
    printf("alloc src buff failed\n");
    return -1;
}

ret = MI_SYS_MMA_Alloc(u16SocId, NULL, buffSize, &phyDstBufAddr);
if(ret != MI_SUCCESS)
{
    MI_SYS_MMA_Free(u16SocId, phySrcBufAddr);
    printf("alloc dts buff failed\n");
    return -1;
}

MI_SYS_MemsetPa(u16SocId, phySrcBufAddr, 0xffffffff, buffSize);
MI_SYS_MemsetPa(u16SocId, phyDstBufAddr, 0x00, buffSize);
MI_SYS_MemcpyPa(u16SocId, phyDstBufAddr, phySrcBufAddr, buffSize);
MI_SYS_MMA_Free(u16SocId, phySrcBufAddr);
MI_SYS_MMA_Free(u16SocId, phyDstBufAddr);
```

➤ 相关主题

[MI_SYS_MemcpyPa](#)

2.5.9 MI_SYS_MemcpyPa

➤ 功能

通过 DMA 硬件模块，把源内存数据拷贝到目标内存上。

➤ 语法

```
MI_S32 MI_SYS_MemcpyPa(MI_U16 u16SocId, MI_PHY phyDst, MI_PHY phySrc,
MI_U32 u32Lenth);
```

➤ 形参

参数名称	参数含义	输入/输出
u16SocId	指定执行本操作的目标 Soc	输入
phyDst	目的物理地址	输入
phySrc	源物理地址	输入
u32Lenth	拷贝大小，单位为 byte	输入

➤ 返回值

- 0 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

※ 注意

N/A

➤ 举例

参见 [MI_SYS_MemsetPa](#) 举例。

➤ 相关主题

[MI_SYS_MemsetPa](#)

2.5.10 MI_SYS_BufFillPa

➤ 功能

通过 DMA 硬件模块，填充部分物理内存。

➤ 语法

MI_S32 MI_SYS_BufFillPa(MI_U16 u16SocId, [MI_SYS_FrameData_t](#) *pstBuf, MI_U32 u32Val, [MI_SYS_WindowRect_t](#) *pstRect);

➤ 形参

参数名称	参数含义	输入/输出
u16SocId	指定执行本操作的目标 Soc	输入
pstBuf	填充的帧数据描述之结构体	输入
u32Val	填充值	输入
pstRect	填充的数据范围	输入

➤ 返回值

- 0 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

※ 注意

- pstRect 数据范围是以 pstBuf 所描述的首地址为（0，0）点开始，宽高都是以 pstBuf 中的 ePixelFormat 来计算每一个 pixel 所移动的内存数据大小来进行部分填充。
- pstBuf 中 u16Width、u16Height、phyAddr、u32Stride、ePixelFormat 为必填，其余数值无意义。

➤ 举例

MI_SYS_BufFillPa 调用 Sample

```
MI_S32 ret = 0;
MI_SYS_WindowRect_t rect;
```

```

MI_SYS_FrameData_t stSysFrame;
MI_U16 u16SocId = ST_DEFAULT_SOC_ID;

memcpy(&stSysFrame, &buf->stFrameData, sizeof(MI_SYS_FrameData_t));

if(pRect) {
    rect.u16X = pRect->left;
    rect.u16Y = pRect->top;
    rect.u16Height = pRect->bottom-pRect->top;
    rect.u16Width = pRect->right-pRect->left;
} else {
    rect.u16X = 0;
    rect.u16Y = 0;
    rect.u16Height = stSysFrame.u16Height;
    rect.u16Width = stSysFrame.u16Width;
}

DBG_INFO("rect %d %d %d %d \n", rect.u16X, rect.u16Y
        , rect.u16Width, rect.u16Height);
ret = MI_SYS_BufFillPa(u16SocId, &stSysFrame, u32ColorVal, &rect);

return ret;

```

➤ 相关主题

[MI_SYS_BufBlitPa](#)

2.5.11 MI_SYS_BufBlitPa

➤ 功能

通过 DMA 硬件模块，把源内存数据上的部分区域拷贝到目标内存上的部分区域。

➤ 语法

```

MI_S32 MI_SYS_BufBlitPa(MI_U16 u16SocId, MI\_SYS\_FrameData\_t *pstDstBuf,
MI\_SYS\_WindowRect\_t *pstDstRect, MI\_SYS\_FrameData\_t *pstSrcBuf,
MI\_SYS\_WindowRect\_t *pstSrcRect);

```

➤ 形参

参数名称	参数含义	输入/输出
u16SocId	指定执行本操作的目标 Soc	输入
pstDstBuf	目标内存物理首地址	输入
pstDstRect	目标内存拷贝的区域	输入
pstSrcBuf	源内存物理首地址	输入
pstSrcRect	源内存拷贝的区域	输入

➤ 返回值

- 0 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

※ 注意

- pstDstRect/pstSrcRect 数据范围是以 pstDstBuf/pstSrcBuf 所描述的首地址为（0，0）点开始，宽高都是以其中的 ePixelFormat 来计算每一个 pixel 所移动的内存数据大小来进行部分填充。
- pstDstBuf/pstSrcBuf 中 u16Width、u16Height、phyAddr、u32Stride、ePixelFormat 为必填，其余数值无意义。
- 源内存或者目标内存的区域部分超过了其本来的范围，则只会拷贝其未超过部分上的数据。

➤ 举例

MI_SYS_BufBlitPa 调用 Sample

```
MI_S32 ret = MI_SUCCESS;
vdisp_copyinfo_plane_t *plane;
MI_SYS_FrameData_t stSrcFrame, stDstFrame;
MI_SYS_WindowRect_t stSrcRect, stDstRect;
MI_U16 u16SocId = ST_DEFAULT_SOC_ID;

plane = &copyinfo->plane[0];
```

```
stSrcFrame.ePixelFormat = E_MI_SYS_PIXEL_FRAME_I8;
stSrcFrame.phyAddr[0] = plane->src_paddr;
stSrcFrame.u16Width = plane->width;
stSrcFrame.u16Height = plane->height;
stSrcFrame.u32Stride[0] = plane->src_stride;

stDstFrame.ePixelFormat = E_MI_SYS_PIXEL_FRAME_I8;
stDstFrame.phyAddr[0] = plane->dst_paddr;
stDstFrame.u16Width = plane->width;
stDstFrame.u16Height = plane->height;
stDstFrame.u32Stride[0] = plane->dst_stride;

stSrcRect.u16X = 0;
stSrcRect.u16Y = 0;
stSrcRect.u16Height = stSrcFrame.u16Height;
stSrcRect.u16Width = stSrcFrame.u16Width;

stDstRect.u16X = 0;
stDstRect.u16Y = 0;
stDstRect.u16Height = stDstFrame.u16Height;
stDstRect.u16Width = stDstFrame.u16Width;

ret = MI_SYS_BufBlitPa(u16SocId, &stDstFrame, &stDstRect, &stSrcFrame, &stSrcRect);

return ret;
```

➤ 相关主题

[MI_SYS_BufFillPa](#)

2.5.12 MI_SYS_ConfigPrivateMMAPool

➤ 功能

为模块配置私有 MMA Heap.

➤ 语法

```
MI_S32 MI_SYS_ConfigPrivateMMAPool(MI_U16 u16SocId,
MI\_SYS\_GlobalPrivPoolConfig\_t *pstGlobalPrivPoolConf);
```


➤ 形参

参数名称	参数含义	输入/输出
u16SocId	指定执行本操作的目标 Soc	输入
pstGlobalPrivPoolConf	配置私有 Mma Pool 结构体指针	输入

➤ 返回值

- 0 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

※ 注意

- 设备私有 **MMA Heap** 与通道私有 **MMA Heap** 不能共存。
- 建议在 [MI_SYS_Init](#) 后就为各模块创建好私有 MMA heap。
- 当 pstGlobalPrivPoolConf->bCreate 为 TRUE 时，创建私有 POOL，为 FALSE 时，销毁私有 POOL
- 各种 eConfigType 使用场景如下：

eConfigType	适用场景
E_MI_SYS_VPE_TO_VENC_PRIVATE_RING_POOL	为绑定为 E_MI_SYS_BIND_TYPE_HW_RING 模式的 scl 与 venc 端口设置私有 ring heap pool。 如果是 mi_sys v2.0，则表示 vpe 与 venc 端口的设置。
E_MI_SYS_PRE_CHN_PRIVATE_POOL	为模块通道设置私有 heap pool
E_MI_SYS_PRE_DEV_PRIVATE_POOL	为模块设备设置私有 heap pool
E_MI_SYS_PER_CHN_PORT_OUTPUT_POOL	为模块 output port 设置 heap pool，设置后该 port 口的 output buf 优先从其中分配

➤ 举例

假设场景为：IPC 下两路流，一路 Main Stream H265 最大分辨率 1920*1080,一路 Sub

Stream H264 最大分辨率 720*576。

VENC 创建 MMA Heap:

```
//Main Sream:
//为通道 1 创建大小为 38745760 的私有 MMA heap
MI_SYS_GlobalPrivPoolConfig_t stConfig;
MI_U16 u16SocId = ST_DEFAULT_SOC_ID;

memset(&stConfig , 0 ,sizeof(stConfig));
stConfig.eConfigType = E_MI_SYS_PER_CHN_PRIVATE_POOL;
stConfig.bCreate = TRUE;
stConfig.uConfig.stPreChnPrivPoolConfig.eModule = E_MI_MODULE_ID_VENC;
stConfig.uConfig.stPreChnPrivPoolConfig.u32Devid = 0;
stConfig.uConfig.stPreChnPrivPoolConfig.u32Channel= 1;
stConfig.uConfig.stPreChnPrivPoolConfig.u32PrivateHeapSize = 38745760;
MI_SYS_ConfigPrivateMMAPool(u16SocId, &stConfig);

//Sub Stream:
//为通道 2 创建大小为 805152 的私有 MMA Heap
stConfig.eConfigType = E_MI_SYS_PER_CHN_PRIVATE_POOL;
stConfig.bCreate = TRUE;
stConfig.uConfig.stPreChnPrivPoolConfig.eModule = E_MI_MODULE_ID_VENC;
stConfig.uConfig.stPreChnPrivPoolConfig.u32Devid = 0;
stConfig.uConfig.stPreChnPrivPoolConfig.u32Channel= 2;
stConfig.uConfig.stPreChnPrivPoolConfig.u32PrivateHeapSize = 805152;
MI_SYS_ConfigPrivateMMAPool(u16SocId, &stConfig);
```

VENC 销毁 MMA Heap:

```
// Main Sream:
//销毁通道 1 的私有 MMA heap
stConfig.eConfigType = E_MI_SYS_PER_CHN_PRIVATE_POOL;
stConfig.bCreate = FALSE;
stConfig.uConfig.stPreChnPrivPoolConfig.eModule = E_MI_MODULE_ID_VENC;
```

```
stConfig.uConfig.stPreChnPrivPoolConfig.u32Devid = 0;
stConfig.uConfig.stPreChnPrivPoolConfig.u32Channel= 1;
MI_SYS_ConfigPrivateMMAPool(u16SocId, &stConfig);

//Sub Stream:
//销毁通道 2 的私有 MMA Heap
stConfig.eConfigType = E_MI_SYS_PER_CHN_PRIVATE_POOL;
stConfig.bCreate = FALSE;
stConfig.uConfig.stPreChnPrivPoolConfig.eModule = E_MI_MODULE_ID_VENC;
stConfig.uConfig.stPreChnPrivPoolConfig.u32Devid = 0;
stConfig.uConfig.stPreChnPrivPoolConfig.u32Channel= 2;
MI_SYS_ConfigPrivateMMAPool(u16SocId, &stConfig);
```

ISP 创建 MMA Heap:

```
//为设备 0 创建大小为 0x4f9200 的私有 MMA Heap
stConfig.eConfigType = E_MI_SYS_PER_DEV_PRIVATE_POOL;
stConfig.bCreate = TRUE;
stConfig.uConfig.stPreDevPrivPoolConfig.eModule = E_MI_MODULE_ID_ISP;
stConfig.uConfig.stPreDevPrivPoolConfig.u32Devid = 0;
MI_SYS_ConfigPrivateMMAPool(u16SocId, &stConfig);
```

ISP 销毁 MMA Heap:

```
//销毁设备 0 的私有 MMA Heap
stConfig.eConfigType = E_MI_SYS_PER_DEV_PRIVATE_POOL;
stConfig.bCreate = FALSE;
stConfig.uConfig.stPreDevPrivPoolConfig.eModule = E_MI_MODULE_ID_ISP;
stConfig.uConfig.stPreChnPrivPoolConfig.u32Devid = 0;
MI_SYS_ConfigPrivateMMAPool(u16SocId, &stConfig);
```

创建 SCL 与 VENC 之间的 ring MMA Heap:

```
stConfig.eConfigType = E_MI_SYS_VPE_TO_VENC_PRIVATE_RING_POOL;
stConfig.bCreate = TRUE;
```

```
stConfig.uConfig.stPreVpe2VencRingPrivPoolConfig.u32VencInputRingPoolStaticSize =  
8*1024*1024;  
MI_SYS_ConfigPrivateMMAPool(u16SocId, &stConfig);
```

销毁 SCL 与 VENC 之间的 ring MMA Heap:

```
stConfig.eConfigType = E_MI_SYS_VPE_TO_VENC_PRIVATE_RING_POOL;  
stConfig.bCreate = FALSE;  
MI_SYS_ConfigPrivateMMAPool(u16SocId, &stConfig);
```

不同分辨率或开启不同功能 size 大小都不一样，详细参数请根据使用的场景和规格计算。

➤ 相关主题

[MI_SYS_PrivateDevChnHeapAlloc](#) / [MI_SYS_PrivateDevChnHeapFree](#)

2.5.13 MI_SYS_PrivateDevChnHeapAlloc

➤ 功能

从模块通道私有 MMA Pool 申请内存。

➤ 语法

```
MI_S32 MI_SYS_PrivateDevChnHeapAlloc(MI_U16 u16SocId, MI_ModuleId_e eModule,  
MI_U32 u32Devid, MI_S32 s32ChnId, MI_U8 *pu8BufName, MI_U32 u32blkSize, MI_PHY  
*pphyAddr, MI_BOOL bTailAlloc);
```

形参

参数名称	参数含义	输入/输出
u16SocId	指定执行本操作的目标 Soc	输入
eModule	模块 ID	输入
u32Devid	模块的设备 ID	输入
s32ChnId	模块的通道 ID	输入
pu8BufName	申请私有内存的名字，传 Null 时，使用 "app-privAlloc" 作为默认的名字	输入

参数名称	参数含义	输入/输出
u32blkSize	申请私有内存的大小	输入
pphyAddr	分配出的私有内存的物理地址	输出
bTailAlloc	是否从通道私有 pool 的尾部申请 0-否, 1-是	输入

➤ 返回值

- 0 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

※ 注意

使用该接口时，请确定已经先使用 [MI_SYS_ConfigPrivateMMAPool](#) 申请了 E_MI_SYS_PRE_CHN_PRIVATE_POOL 类型的私有内存池。

➤ 举例

MI_SYS_PrivateDevChnHeapAlloc 调用 Sample

```
MI_PHY *pphyAddr = NULL;
MI_SYS_GlobalPrivPoolConfig_t stConfig;
MI_U16 u16SocId = ST_DEFAULT_SOC_ID;

Memset(&stConfig, 0, sizeof(stConfig));
stConfig.eConfigType = E_MI_SYS_PER_CHN_PRIVATE_POOL;
stConfig.bCreate = TRUE;
stConfig.uConfig.stPreChnPrivPoolConfig.eModule = E_MI_MODULE_ID_VENC;
stConfig.uConfig.stPreChnPrivPoolConfig.u32Devid = 0;
stConfig.uConfig.stPreChnPrivPoolConfig.u32Channel = 1;
stConfig.uConfig.stPreChnPrivPoolConfig.u32PrivateHeapSize = 38745760;
MI_SYS_ConfigPrivateMMAPool(u16SocId, &stConfig);
```

```
ret = MI_SYS_PrivateDevChnHeapAlloc(u16SocId, E_MI_MODULE_ID_VENC, 0, 1, NULL,
4096, pphyAddr, FALSE);
if(ret != MI_SUCCESS)
{
    MI_SYS_ConfigPrivateMMAPool(u16SocId, &stConfig);
    printf("alloc buff from chn private heap failed\n");
    return -1;
}

//do something...

MI_SYS_PrivateDevChnHeapFree(u16SocId, E_MI_MODULE_ID_VENC, 0, 1, *pphyAddr);
MI_SYS_ConfigPrivateMMAPool(u16SocId, &stConfig);
```

➤ 相关主题

[MI_SYS_ConfigPrivateMMAPool](#) / [MI_SYS_PrivateDevChnHeapFree](#)

2.5.14 MI_SYS_PrivateDevChnHeapFree

➤ 功能

从模块通道私有 MMA Pool 释放内存。

➤ 语法

MI_S32 MI_SYS_PrivateDevChnHeapFree(MI_U16 u16SocId, [MI_ModuleId_e](#) eModule, MI_U32 u32Devid, MI_S32 s32ChnId, MI_PHY phyAddr);

➤ 形参

参数名称	参数含义	输入/输出
u16SocId	指定执行本操作的目标 Soc	输入
eModule	模块 ID	输入
u32Devid	模块的设备 ID	输入
s32ChnId	模块的通道 ID	输入

参数名称	参数含义	输入/输出
phyAddr	需要释放的内存对应的物理地址	输入

➤ 返回值

- 0 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

※ 注意

使用该接口时，请确定已经先使用 [MI_SYS_ConfigPrivateMMAPool](#) 申请了 E_MI_SYS_PRE_CHN_PRIVATE_POOL 类型的私有内存池。如果没有配置对应的私有池，默认会在 mma_heap_name0 申请内存。

➤ 举例

参见 [MI_SYS_PrivateDevChnHeapAlloc](#) 调用 [Sample](#) 举例。

➤ 相关主题

[MI_SYS_ConfigPrivateMMAPool](#) / [MI_SYS_PrivateDevChnHeapAlloc](#)

2.5.15 MI_SYS_Va2Pa

➤ 功能

将 MI 内存块的 CPU 虚拟地址转成内存物理地址。

➤ 语法

```
MI_S32 MI_SYS_Va2Pa (void *pVirtualAddress, MI_PHY *pPhyAddr);
```

➤ 形参

参数名称	参数含义	输入/输出
------	------	-------

pVirtualAddress	CPU 虚拟地址指针	输入
pPhyAddr	存储内存块物理地址的指针	输出

➤ 返回值

- 0 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_sys_datatype.h、mi_sys.h
- 库文件：libmi_sys.a / libmi_sys.so

※ 注意

使用该接口时，请确定输入的 CPU 虚拟地址有效且指向 MI 中分配的地址空间。

➤ 举例

MI_SYS_Va2Pa 调用 Sample

```
MI_PHY phySrcBufAddr = 0;
MI_PHY phyAddrVa2Pa = 0;
void *pVirSrcBufAddr = NULL;
MI_U32 srcBuffSize = 1920 * 1980 * 3 / 2;
MI_U16 u16SocId = ST_DEFAULT_SOC_ID;

srcBuffSize = ALIGN_UP(srcBuffSize, 4096);

ret = MI_SYS_MMA_Alloc(u16SocId, NULL, srcBuffSize, &phySrcBufAddr);
if(ret != MI_SUCCESS)
{
    printf("alloc src buff failed\n");
    return -1;
}

ret = MI_SYS_Mmap(phySrcBufAddr, srcBuffSize, &pVirSrcBufAddr, TRUE);
if(ret != MI_SUCCESS)
```



```
{  
    MI_SYS_MMA_Free(u16SocId, phySrcBufAddr);  
    printf("mmap src buff failed\n");  
    return -1;  
}  
  
MI_SYS_Va2Pa(pVirSrcBufAddr, &phyAddrVa2Pa);  
assert(phySrcBufAddr == phyAddrVa2Pa);  
memset(pVirSrcBufAddr, 0, srcBuffSize);  
MI_SYS_FlushInvCache(pVirSrcBufAddr, srcBuffSize);  
MI_SYS_Munmap(pVirSrcBufAddr, srcBuffSize);  
MI_SYS_MMA_Free(u16SocId, phySrcBufAddr);
```

➤ 相关主题

[MI_SYS_Mmap](#)

3. 数据类型

3.1. 数据结构描述格式说明

本手册使用 5 个参考域描述数据类型的相关信息，它们作用如 个参考域描述数据类型的相关信息，它们作用如 表 3-1 所示。

表 3-4: 数据结构描述格式说明

标签	功能
说明	简要描述数据类型的主要功能。
定义	列出数据类型的定义语句
成员	列出数据结构的成员及含义。
注意事项	列出使用数据类型时应注意的事项。
相关数据类型及接口	列出与本数据类型相关联的其他数据类型和接口。

3.2. 数据结构列表

相关数据类型、数据结构定义如下：

表 3-5: 数据结构汇总

数据类型	定义

数据类型	定义
MI_SYS_PixelFormat_e	定义像素枚举类型
MI_SYS_CompressMode_e	定义压缩方式枚举类型
MI_SYS_FrameTileMode_e	定义 Tile 格式枚举类型
MI_SYS_FieldType_e	定义 Field 枚举类型
MI_SYS_BufDataType_e	定义模块 ID 枚举类型
MI_SYS_FrameIspInfoType_e	定义 frame data 携带的 ISP info 枚举类型
MI_SYS_ChnPort_t	定义模块设备通道结构体
MI_SYS_MetaData_t	定义码流 MetaData 的结构体
MI_SYS_RawData_t	定义码流 RawData 的结构体
MI_SYS_WindowRect_t	定义 Window 坐标的结构体
MI_SYS_FrameData_t	定义码流 FrameData 的结构体
MI_SYS_BufInfo_t	Buf 信息结构体
MI_SYS_FrameBufExtraConfig_t	定义码流 Frame buffer 额外配置的结构体
MI_SYS_BufFrameConfig_t	Frame buf 配置信息结构体
MI_SYS_BufRawConfig_t	Raw buf 配置信息结构体
MI_SYS_MetaDataConfig_t	Meta buf 配置信息结构体
MI_SYS_BufConf_t	配置 Buf 信息结构体
MI_SYS_Version_t	Sys 版本信息结构体
MI_VB_PoolListConf_t	描述 VB Pool 链表信息的结构体
MI_SYS_BindType_e	定义前后级工作模式的枚举类型
MI_SYS_FrameData_PhySignalType	描述 frame data 隶属的 buffer 类型的枚举类型
MI_SYS_InsidePrivatePoolType_e	描述创建的私有 MMA POOL 类型的枚举类型
MI_SYS_PerChnPrivHeapConf_t	定义描述通道私有 MMA Pool 的结构体
MI_SYS_PerDevPrivHeapConf_t	定义描述设备私有 MMA Pool 的结构体
MI_SYS_PerVpe2VencRingPoolConf_t	定义描述 SCL 与 VENC 之间的私有 Ring MMA Pool 的结构体 Mi_sys v2.0 表示 VPE 与 VENC
MI_SYS_PerChnPortOutputPool_t	定义描述 output port 的私有 MMA Pool 的结构体

数据类型	定义
MI_SYS_GlobalPrivPoolConfig_t	定义描述配置私有 MMA Pool 的结构体
MI_SYS_FrameIspInfo_t	定义 frame data 中 ISP info 的结构体
MI_SYS_FbcData_t	定义 frame buffer compress 数据结构体
MI_SYS_BufFrameMultiPlaneConfig_t	MultiPlane Frame buf 配置信息结构体
MI_SYS_FrameDataSubPlane_t	SubPlane Frame buf 配置信息结构体
MI_SYS_BufFrameMetaConfig_t	定义 frame meta data 的结构体
MI_SYS_FrameDataMultiPlane_t	MultiPlane Frame buf 配置信息结构体
MI_SYS_UserPictureInfo_t	定义描述 User picture info 的结构体

3.2.1 MI_ModuleId_e

➤ 说明

定义模块 ID 枚举类型。

➤ 定义

typedef enum

```
{
    E_MI_MODULE_ID_IVE    = 0,
    E_MI_MODULE_ID_VDF    = 1,
    E_MI_MODULE_ID_VENC    = 2,
    E_MI_MODULE_ID_RGN    = 3,
    E_MI_MODULE_ID_AI     = 4,
    E_MI_MODULE_ID_AO     = 5,
    E_MI_MODULE_ID_VIF    = 6,
    E_MI_MODULE_ID_VPE    = 7,
    E_MI_MODULE_ID_VDEC    = 8,
    E_MI_MODULE_ID_SYS     = 9,
    E_MI_MODULE_ID_FB     = 10,
    E_MI_MODULE_ID_HDMI    = 11,
    E_MI_MODULE_ID_DIVP    = 12,
```

```
E_MI_MODULE_ID_GFX = 13,  
E_MI_MODULE_ID_VDISP = 14,  
E_MI_MODULE_ID_DISP = 15,  
E_MI_MODULE_ID_OS = 16,  
E_MI_MODULE_ID_IAE = 17,  
E_MI_MODULE_ID_MD = 18,  
E_MI_MODULE_ID_OD = 19,  
E_MI_MODULE_ID_SHADOW = 20,  
E_MI_MODULE_ID_WARP = 21,  
E_MI_MODULE_ID_UAC = 22,  
E_MI_MODULE_ID_LDC = 23,  
E_MI_MODULE_ID_SD = 24,  
E_MI_MODULE_ID_PANEL = 25,  
E_MI_MODULE_ID_CIPHER = 26,  
E_MI_MODULE_ID_SNR = 27,  
E_MI_MODULE_ID_WLAN = 28,  
E_MI_MODULE_ID_IPU = 29,  
    E_MI_MODULE_ID_MIPITX = 30,  
E_MI_MODULE_ID_GYRO = 31,  
E_MI_MODULE_ID_JPD = 32,  
E_MI_MODULE_ID_ISP = 33,  
    E_MI_MODULE_ID_SCL = 34,  
    E_MI_MODULE_ID_WBC = 35,  
    E_MI_MODULE_ID_DSP = 36,  
    E_MI_MODULE_ID_PCIE = 37,  
    E_MI_MODULE_ID_DUMMY = 38,  
  
//E_MI_MODULE_ID_SED = 29,  
E_MI_MODULE_ID_MAX,  
} MI_ModuleId_e;
```

➤ 成员

模块 ID	模块 ID HEX 值	成员名称	模块
0	0x00	E_MI_MODULE_ID_IVE	图像智能算子 IVE 的模块 ID
1	0x01	E_MI_MODULE_ID_VDF	视频智能算法框架模块 VDF 的模块 ID
2	0x02	E_MI_MODULE_ID_VENC	视频编码模块 VENC 的模块 ID
3	0x03	E_MI_MODULE_ID_RGN	OSD 叠加和遮挡模块 REG 的模块 ID
4	0x04	E_MI_MODULE_ID_AI	音频输入模块 AI 的模块 ID
5	0x05	E_MI_MODULE_ID_AO	音频输出模块 AO 的模块 ID
6	0x06	E_MI_MODULE_ID_VIF	视频输入 VIF 的模块 ID
7	0x07	E_MI_MODULE_ID_VPE	视频图像处理模块 VPE 的模块 ID
8	0x08	E_MI_MODULE_ID_VDEC	视频解码 VEDC 的模块 ID
9	0x09	E_MI_MODULE_ID_SYS	系统模块 SYS 的模块 ID
10	0x0A	E_MI_MODULE_ID_FB	SSStar UI 显示 FrameBuffer Device 模块的模块 ID
11	0x0B	E_MI_MODULE_ID_HDMI	HMDI 模块 ID
12	0x0C	E_MI_MODULE_ID_DIVP	视频 DI 及后处理模块 DIVP 的模块 ID
13	0x0D	E_MI_MODULE_ID_GFX	2D 图形处理加速模块 GFX 的模块 ID
14	0x0E	E_MI_MODULE_ID_VDISP	图像拼图模块 VDISP 的模块 ID
15	0x0F	E_MI_MODULE_ID_DISP	视频显示模块 DISP 的模块 ID
16	0x10	E_MI_MODULE_ID_OS	RTOS 系统模块 ID
17	0x11	E_MI_MODULE_ID_IAE	音频智能算子 IAE 的模块 ID
18	0x12	E_MI_MODULE_ID_MD	移动侦测模块 MD 的模块 ID
19	0x13	E_MI_MODULE_ID_OD	遮挡侦测模块 OD 的模块 ID
20	0x14	E_MI_MODULE_ID_SHADOW	虚拟 MI 框架 SHADOW 的模块 ID
21	0x15	E_MI_MODULE_ID_WARP	图像畸形校正算法 WARP 的模块 ID
22	0x16	E_MI_MODULE_ID_UAC	USB Aduio Class 专用 ALSA 设备

模块 ID	模块 ID HEX 值	成员名称	模块
			的模块 ID
23	0x17	E_MI_MODULE_ID_LDC	图像畸形校正算法 LDC 的模块 ID
24	0x18	E_MI_MODULE_ID_SD	图像缩放功能的模块 ID
25	0x19	E_MI_MODULE_ID_PANEL	屏显示 PANEL 的模块 ID
26	0x1A	E_MI_MODULE_ID_CIPHER	芯片加密 CIPHER 的模块 ID
27	0x1B	E_MI_MODULE_ID_SNR	Sensor 传感器模块 ID
28	0x1C	E_MI_MODULE_ID_WLAN	无线通信网络 WLAN 的模块 ID
29	0x1D	E_MI_MODULE_ID_IPU	图像识别处理 IPU 的模块 ID
30	0x1E	E_MI_MODULE_ID_MIPITX	使用 MIPI 协议发送数据模块 ID
31	0x1F	E_MI_MODULE_ID_GYRO	陀螺仪的模块 ID
32	0x20	E_MI_MODULE_ID_JPD	JPD 处理的模块 ID
33	0x21	E_MI_MODULE_ID_ISP	图像信号处理的模块 ID
34	0x22	E_MI_MODULE_ID_SCL	图像缩放处理的模块 ID
35	0x23	E_MI_MODULE_ID_WBC	视频回写的模块 ID
36	0X24	E_MI_MODULE_ID_DSP	DSP 模块 ID
37	0X25	E_MI_MODULE_ID_PCIE	PCIE 模块 ID
38	0X26	E_MI_MODULE_ID_DUMMY	内部测试模块 ID

※ 注意事项

N/A

➤ 相关数据接口及类型

N/A

3.2.2 MI_SYS_PixelFormat_e

➤ 说明

定义像素枚举类型。

➤ 定义

```
typedef enum
{
    E_MI_SYS_PIXEL_FRAME_YUV422_YUYV = 0,
    E_MI_SYS_PIXEL_FRAME_ARGB8888,
    E_MI_SYS_PIXEL_FRAME_ABGR8888,
    E_MI_SYS_PIXEL_FRAME_BGRA8888,

    E_MI_SYS_PIXEL_FRAME_RGB565,
    E_MI_SYS_PIXEL_FRAME_ARGB1555,
    E_MI_SYS_PIXEL_FRAME_ARGB4444,
    E_MI_SYS_PIXEL_FRAME_I2,
    E_MI_SYS_PIXEL_FRAME_I4,
    E_MI_SYS_PIXEL_FRAME_I8,

    E_MI_SYS_PIXEL_FRAME_YUV_SEMIPLANAR_422,
    E_MI_SYS_PIXEL_FRAME_YUV_SEMIPLANAR_420,
    E_MI_SYS_PIXEL_FRAME_YUV_SEMIPLANAR_420_NV21,
    E_MI_SYS_PIXEL_FRAME_YUV_TILE_420,
    E_MI_SYS_PIXEL_FRAME_YUV422_UYVY,
    E_MI_SYS_PIXEL_FRAME_YUV422_YVYU,
    E_MI_SYS_PIXEL_FRAME_YUV422_VYUY,

    E_MI_SYS_PIXEL_FRAME_YUV422_PLANAR,
    E_MI_SYS_PIXEL_FRAME_YUV420_PLANAR,

    E_MI_SYS_PIXEL_FRAME_FBC_420,
    E_MI_SYS_PIXEL_FRAME_RGB_BAYER_BASE,
    E_MI_SYS_PIXEL_FRAME_RGB_BAYER_NUM = E_MI_SYS_PIXEL_FRAME_RGB_BAYER_BASE
+ E_MI_SYS_DATA_PRECISION_MAX * E_MI_SYS_PIXEL_BAYERID_MAX - 1,

    E_MI_SYS_PIXEL_FRAME_RGB888,
    E_MI_SYS_PIXEL_FRAME_BGR888,
```

```

E_MI_SYS_PIXEL_FRAME_GRAY8,
E_MI_SYS_PIXEL_FRAME_RGB101010,
E_MI_SYS_PIXEL_FRAME_RGB888_PLANAR,

E_MI_SYS_PIXEL_FRAME_FORMAT_MAX,
} MI_SYS_PixelFormat_e;

```

➤ 成员

成员名称	描述
E_MI_SYS_PIXEL_FRAME_YUV422_YUYV	YUV422_YUYV 格式
E_MI_SYS_PIXEL_FRAME_ARGB8888	ARGB8888 格式
E_MI_SYS_PIXEL_FRAME_ABGR8888	ABGR8888 格式
E_MI_SYS_PIXEL_FRAME_BGRA8888	BGRA8888 格式
E_MI_SYS_PIXEL_FRAME_RGB565	RGB565 格式
E_MI_SYS_PIXEL_FRAME_ARGB1555	ARGB1555 格式
E_MI_SYS_PIXEL_FRAME_ARGB4444	ARGB4444 格式
E_MI_SYS_PIXEL_FRAME_I2	2 bpp 格式
E_MI_SYS_PIXEL_FRAME_I4	4 bpp 格式
E_MI_SYS_PIXEL_FRAME_I8	8 bpp 格式
E_MI_SYS_PIXEL_FRAME_YUV_SEMIPLANAR_422	YUV422 semi-planar 格式
E_MI_SYS_PIXEL_FRAME_YUV_SEMIPLANAR_420	YUV420sp nv12 格式
E_MI_SYS_PIXEL_FRAME_YUV_SEMIPLANAR_420_NV21	YUV420sp nv21 格式
E_MI_SYS_PIXEL_FRAME_YUV_TILE_420	内部自定义 TILE 格式
E_MI_SYS_PIXEL_FRAME_YUV422_UYVY	YUV422_UYVY 格式
E_MI_SYS_PIXEL_FRAME_YUV422_YVYU	YUV422_YVYU 格式
E_MI_SYS_PIXEL_FRAME_YUV422_VYUY	YUV422_VYUY 格式
E_MI_SYS_PIXEL_FRAME_YUV422_PLANAR	YUV422_PLANAR 格式
E_MI_SYS_PIXEL_FRAME_YUV420_PLANAR	YUV420_PLANAR 格式
E_MI_SYS_PIXEL_FRAME_FBC_420	内部自定义格式
E_MI_SYS_PIXEL_FRAME_RGB_BAYER_BASE	RGB raw data 组合格式的起始

成员名称	描述
	值
E_MI_SYS_PIXEL_FRAME_RGB_BAYER_NUM	RGB raw data 组合格式的最大值
E_MI_SYS_PIXEL_FRAME_RGB888	RGB888 格式
E_MI_SYS_PIXEL_FRAME_BGR888	BGR888 格式
E_MI_SYS_PIXEL_FRAME_GRAY8	8bit 灰度格式
E_MI_SYS_PIXEL_FRAME_RGB101010	RGB101010 格式
E_MI_SYS_PIXEL_FRAME_RGB888_PLANAR	RGB888_PLANER 格式
E_MI_SYS_PIXEL_FRAME_FORMAT_MAX	枚举的最大值

※ 注意事项

- E_MI_SYS_PIXEL_FRAME_RGB_BAYER_BASE, E_MI_SYS_PIXEL_FRAME_RGB_BAYER_NUM 是提供给 RGB raw data 组合格式使用的。此格式下，sensor 提供的 bit 位 ePixPrecision 和排列顺序 eBayerId,通过 RGB_BAYER_PIXEL 宏对其组合生成 RGB raw data 组合格式的 enum 值；
- RGB_BAYER_PIXEL 宏定义：#define RGB_BAYER_PIXEL(BitMode, PixelID) (E_MI_SYS_PIXEL_FRAME_RGB_BAYER_BASE+ BitMode*E_MI_SYS_PIXEL_BAYERID_MAX+ PixelID)，其取值范围为：(E_MI_SYS_PIXEL_FRAME_RGB_BAYER_BASE, E_MI_SYS_PIXEL_FRAME_RGB_BAYER_NUM)

下面提供了一个使用的小例子，更详细的信息请参考对应模块的 API 文档。

```
static MI_SYS_PixelFormat_e getRawType(STUB_SensorType_e sensorType)
{
    MI_SYS_PixelFormat_e eRawType = E_MI_SYS_PIXEL_FRAME_RGB_BAYER_NUM;
    MI_SNR_PlaneInfo_t stSnrPlane0Info;

    memset(&stSnrPlane0Info, 0x0, sizeof(MI_SNR_PlaneInfo_t));
    MI_SNR_GetPlaneInfo(E_MI_SNR_PAD_ID_0, 0, &stSnrPlane0Info);
    eRawType = (MI_SYS_PixelFormat_e)RGB_BAYER_PIXEL(stSnrPlane0Info.ePixPrecision,
    stSnrPlane0Info.eBayerId);
}
```

```
return eRawType;
}
```

- 相关数据类型及接口
- N/A

3.2.3 MI_SYS_CompressMode_e

- 说明
- 定义压缩方式枚举类型。
- 定义

```
typedef enum
{
    E_MI_SYS_COMPRESS_MODE_NONE, //no compress
    E_MI_SYS_COMPRESS_MODE_SEG, //compress unit is 256 bytes as a segment
    E_MI_SYS_COMPRESS_MODE_LINE, //compress unit is the whole line
    E_MI_SYS_COMPRESS_MODE_FRAME, //compress unit is the whole frame
    E_MI_SYS_COMPRESS_MODE_TO_8BIT,
    E_MI_SYS_COMPRESS_MODE_TO_6BIT,
    E_MI_SYS_COMPRESS_MODE_AFBC,
    E_MI_SYS_COMPRESS_MODE_SFBC0,
    E_MI_SYS_COMPRESS_MODE_SFBC1,
    E_MI_SYS_COMPRESS_MODE_SFBC2,

    E_MI_SYS_COMPRESS_MODE_BUTT, //number
} MI_SYS_CompressMode_e;
```

- 成员

成员名称	描述
E_MI_SYS_COMPRESS_MODE_NONE	非压缩的视频格式

成员名称	描述
E_MI_SYS_COMPRESS_MODE_SEG	段压缩的视频格式
E_MI_SYS_COMPRESS_MODE_LINE	行压缩的视频格式，按照一行为一段进行压缩
E_MI_SYS_COMPRESS_MODE_FRAME	帧压缩的视频格式，将一帧数据进行压缩
E_MI_SYS_COMPRESS_MODE_TO_8BIT	Pixel 压缩到 8bit
E_MI_SYS_COMPRESS_MODE_TO_6BIT	Pixel 压缩到 6bit
E_MI_SYS_COMPRESS_MODE_AFBC	Arm frame buffer compression
E_MI_SYS_COMPRESS_MODE_SFBC0	Sigmastar 压缩方式 0
E_MI_SYS_COMPRESS_MODE_SFBC1	Sigmastar 压缩方式 1
E_MI_SYS_COMPRESS_MODE_SFBC2	Sigmastar 压缩方式 2
E_MI_SYS_COMPRESS_MODE_BUTT	视频压缩方式的数量

※ 注意事项

N/A

➤ 相关数据类型及接口

N/A

3.2.4 MI_SYS_FrameTileMode_e

➤ 说明

定义 Tile 格式枚举类型。

➤ 定义

```
typedef enum
{
    E_MI_SYS_FRAME_TILE_MODE_NONE = 0,
    E_MI_SYS_FRAME_TILE_MODE_16x16,    // tile mode 16x16
    E_MI_SYS_FRAME_TILE_MODE_16x32,    // tile mode 16x32
    E_MI_SYS_FRAME_TILE_MODE_32x16,    // tile mode 32x16
    E_MI_SYS_FRAME_TILE_MODE_32x32,    // tile mode 32x32
}
```

```

E_MI_SYS_FRAME_TILE_MODE_128x32, // tile mode 128x32
E_MI_SYS_FRAME_TILE_MODE_128x64, // tile mode 128x64
E_MI_SYS_FRAME_TILE_MODE_64x4,   // tile mode 64x4
E_MI_SYS_FRAME_TILE_MODE_MAX
} MI_SYS_FrameTileMode_e;

```

➤ 成员

成员名称	描述
E_MI_SYS_FRAME_TILE_MODE_NONE	None
E_MI_SYS_FRAME_TILE_MODE_16x16	16x16 mode
E_MI_SYS_FRAME_TILE_MODE_16x32	16x32 mode
E_MI_SYS_FRAME_TILE_MODE_32x16	32x16 mode
E_MI_SYS_FRAME_TILE_MODE_32x32	32x32 mode
E_MI_SYS_FRAME_TILE_MODE_128x32	128x32 mode
E_MI_SYS_FRAME_TILE_MODE_128x64	128x64 mode
E_MI_SYS_FRAME_TILE_MODE_64x4	64x4 mode

※ 注意事项

N/A

➤ 相关数据类型及接口

N/A

3.2.5 MI_SYS_FieldType_e

➤ 说明

定义枚举类型。

➤ 定义

```

typedef enum
{

```

```

E_MI_SYS_FIELDTYPE_NONE,    //< no field.
E_MI_SYS_FIELDTYPE_TOP,     //< Top field only.
E_MI_SYS_FIELDTYPE_BOTTOM,  //< Bottom field only.
E_MI_SYS_FIELDTYPE_BOTH,    //< Both fields.
E_MI_SYS_FIELDTYPE_NUM
} MI_SYS_FieldType_e;

```

➤ 成员

成员名称	描述
E_MI_SYS_FIELDTYPE_NONE	None
E_MI_SYS_FIELDTYPE_TOP	Top field only
E_MI_SYS_FIELDTYPE_BOTTOM	Bottom field only
E_MI_SYS_FIELDTYPE_BOTH	Both fields

※ 注意事项

N/A

➤ 相关数据类型及接口

N/A

3.2.6 MI_SYS_BufDataType_e

➤ 说明

定义模块 ID 枚举类型。

➤ 定义

```

typedef enum
{
    E_MI_SYS_BUFDATA_RAW = 0,
    E_MI_SYS_BUFDATA_FRAME,
    E_MI_SYS_BUFDATA_META,
    E_MI_SYS_BUFDATA_MULTIPANE,

```

```
E_MI_SYS_BUFDATA_FBC,  
E_MI_SYS_BUFDATA_MAX  
} MI_SYS_BufDataType_e;
```

➤ 成员

成员名称	描述
E_MI_SYS_BUFDATA_RAW	Raw 数据类型
E_MI_SYS_BUFDATA_FRAME	Frame 数据类型
E_MI_SYS_BUFDATA_META	Meta 数据类型
E_MI_SYS_BUFDATA_MULTIPANE	MultiPlane 数据类型
E_MI_SYS_BUFDATA_FBC	FBC 数据类型
E_MI_SYS_BUFDATA_MAX	数据类型数目

※ 注意事项

N/A

➤ 相关数据类型及接口

N/A

3.2.7 MI_SYS_FrameIspInfoType_e

➤ 说明

定义 frame data 携带的 ISP info 枚举类型。

➤ 定义

```
typedef enum  
{  
    E_MI_SYS_FRAME_ISP_INFO_TYPE_NONE,  
    E_MI_SYS_FRAME_ISP_INFO_TYPE_GLOBAL_GRADIENT  
} MI_SYS_FrameIspInfoType_e;
```

➤ 成员

成员名称	描述
E_MI_SYS_FRAME_ISP_INFO_TYPE_NONE	NONE
E_MI_SYS_FRAME_ISP_INFO_TYPE_GLOBAL_GRADIENT	ISP 的全局梯度数据类型

※ 注意事项

N/A

➤ 相关数据类型及接口

N/A

3.2.8 MI_SYS_ChnPort_t

➤ 说明

定义模块设备通道结构体。

➤ 定义

```
typedef struct MI_SYS_ChnPort_s
{
    MI_ModuleId_e eModId;
    MI_U32 u32DevId;
    MI_U32 u32ChnId;
    MI_U32 u32PortId;
} MI_SYS_ChnPort_t;
```

➤ 成员

成员名称	描述
eModId	模块号
u32DevId	设备号
u32ChnId	通道号
u32PortId	端口号

※ 注意事项

N/A

➤ 相关数据类型及接口

N/A

3.2.9 MI_SYS_MetaData_t

➤ 说明

定义码流 **MetaData** 的结构体。

➤ 定义

```
typedef struct MI_SYS_MetaData_s
{
    void* pVirAddr;
    MI_PHY phyAddr; //notice that this is miu bus addr, not cpu bus addr.

    MI_U32 u32Size;
    MI_U32 u32ExtraData; /*driver special flag*/
    MI_ModuleId_e eDataFromModule;
} MI_SYS_MetaData_t;
```

➤ 成员

成员名称	描述
pVirAddr	数据存放虚拟地址。
phyAddr	数据存放物理地址
u32Size	数据大小
u32ExtraData	driver special flag
eDataFromModule	来自哪个模块的数据

※ 注意事项

N/A

➤ 相关数据类型及接口

N/A

3.2.10 MI_SYS_RawData_t

➤ 说明

定义码流 **RawData** 的结构体。

➤ 定义

```
typedef struct MI_SYS_RawData_s
{
    void* pVirAddr;
    MI_PHY phyAddr; //notice that this is miu bus addr, not cpu bus addr.
    MI_U32 u32BufSize;

    MI_U32 u32ContentSize;
    MI_BOOL bEndOfFrame;
    MI_U64 u64SeqNum;
} MI_SYS_RawData_t;
```

➤ 成员

成员名称	描述
pVirAddr	码流包的虚拟地址。
phyAddr	码流包的物理地址
u32BufSize	Buf 大小
u32ContentSize	数据实际占用 buf 大小
bEndOfFrame	当前帧是否结束。（预留参数）当前只支持帧模式下按帧传送。
u64SeqNum	当前帧的帧序号

※ 注意事项

- 当前只支持按帧传送数据，每次需完整传送一帧数据。

- 码流帧数据附带 PTS 时，解码后输出数据输出相同 PTS。当 PTS=-1 时，不参考系统时钟输出数据帧。

➤ 相关数据类型及接口

N/A

3.2.11 MI_SYS_WindowRect_t

➤ 说明

定义 window 坐标的结构体。

➤ 定义

```
typedef struct MI_SYS_WindowRect_s
{
    MI_U16 u16X;
    MI_U16 u16Y;
    MI_U16 u16Width;
    MI_U16 u16Height;
} MI_SYS_WindowRect_t;
```

➤ 成员

成员名称	描述
u16X	window 起始位置的水平方向的值。
u16Y	window 起始位置的垂直方向的值。
u16Width	window 宽度。
u16Height	window 高度。

※ 注意事项

N/A

➤ 相关数据类型及接口

N/A

3.2.12 MI_SYS_FrameData_t

➤ 说明

定义码流 **FrameData** 的结构体。

➤ 定义

```
//N.B. in MI_SYS_FrameData_t should never support u32Size,
//for other values are enough,and not support u32Size is general standard method.
typedef struct MI_SYS_FrameData_s
{
    MI_SYS_FrameTileMode_e eTileMode;
    MI_SYS_PixelFormat_e ePixelFormat;
    MI_SYS_CompressMode_e eCompressMode;
    MI_SYS_FrameScanMode_e eFrameScanMode;
    MI_SYS_FieldType_e eFieldType;
    MI_SYS_FrameData_PhySignalType ePhylayoutType;

    MI_U16 u16Width;
    MI_U16 u16Height;
    //in case ePhylayoutType equal to REALTIME_FRAME_DATA, pVirAddr would be
    MI_SYS_REALTIME_MAGIC_PADDR and phyAddr would be MI_SYS_REALTIME_MAGIC_VADDR

    void* pVirAddr[3];
    MI_PHY phyAddr[3]; //notice that this is miu bus addr, not cpu bus addr.
    MI_U32 u32Stride[3];
    MI_U32 u32BufSize; //total size that allocated for this buffer, include consider alignment.

    MI_U16 u16RingBufStartLine; //Valid in case RINGBUF_FRAME_DATA, u16RingBufStartLine
    must be LGE than 0 and less than u16Height
    MI_U16 u16RingBufRealTotalHeight; //Valid in case RINGBUF_FRAME_DATA,
    u16RingBufStartLine must be LGE than u16Height
```

```
MI_SYS_FrameIsInfo_t stFrameIsInfo; //isp info of each frame
MI_SYS_WindowRect_t stContentCropWindow;
} MI_SYS_FrameData_t;
```

➤ 成员

成员名称	描述
eTileMode	Tile 模式
ePixelFormat	像素格式
eCompressMode	压缩格式
eFrameScanMode	Frame scan 模块
eFieldType	File 类型
ePhylayoutType	frame data 隶属的 buffer 的类型
u16Width	Frame 宽度
u16Height	Frame 高度
pVirAddr	虚拟地址
phyAddr	物理地址
u32Stride	图片每行所占字节数
u32BufSize	Sys 分配给当前 frame 的实际 buf 大小
u16RingBufStartLine	ring mode 时，frame data 开始于 ring buffer 的行数
u16RingBufRealTotalHeight	ring mode 时，frame data 的真实高度
stFrameIsInfo	ISP info struct
stContentCropWindow	Crop info struct

➤ 注意事项

N/A

➤ 相关数据类型及接口

N/A

3.2.13 MI_SYS_BufInfo_t

➤ 说明

定义码流信息的结构体。

➤ 定义

```
typedef struct MI_SYS_BufInfo_s
{
    MI_U64 u64Pts;
    MI_U64 u64SidebandMsg;
    MI_SYS_BufDataType_e eBufType;
    MI_U32 u32SequenceNumber;
    MI_U32 bEndOfStream : 1;
    MI_U32 bUsrBuf : 1;
    MI_U32 bDrop : 1;
    MI_U32 bCrcCheck : 1;
    MI_U32 u32IrFlag : 2; // For Window Hello Usage: 0x00/off, 0x01/on, 0x02/invalid
    MI_U32 u32Reserved : 26;
    union
    {
        MI_SYS_FrameData_t stFrameData;
        MI_SYS_RawData_t stRawData;
        MI_SYS_MetaData_t stMetaData;
        MI_SYS_FrameDataMultiPlane_t stFrameDataMultiPlane;
        MI_SYS_FbcData_t stFbcData;
    };
} MI_SYS_BufInfo_t;
```

➤ 成员

成员名称			描述
u64Pts			时间戳
u64SidebandMsg			带外数据
eBufType			Buf 类型
u32SequenceNumber			序列号
Bit field	Bit 0	bEndOfStream	是否已发完所有信息
	Bit 1	bUsrBuf	是否是用户使用 buf

成员名称			描述
	Bit 2	bDrop	是否需要丢弃 buf
	Bit 3	bCrcCkeck	是否开启 CRC check
	Bit 4-5	u32IrFlag	Ir 标志位
	Bit 6-31	u32Reserved	保留位
Union	stFrameData		Frame 数据格式结构体
	stRawData		Raw 数据格式结构体
	stMetaData		Meta 数据格式结构体
	stFrameDataMultiPlane		多 Frame 集合数据格式结构体
	stFbcData		Frame buffer 压缩数据结构体

※ 注意事项

N/A

➤ 相关数据类型及接口

N/A

3.2.14 MI_SYS_FrameBufExtraConfig_t

➤ 说明

定义码流 Frame buffer 额外配置的结构体。

➤ 定义

```
typedef struct MI_SYS_FrameBufExtraConfig_s
{
    //Buf alignment requirement in horizontal
    MI_U16 u16BufHAlignment;
    //Buf alignment requirement in vertical
    MI_U16 u16BufVAlignment;
    //Buf alignment requirement in chroma
    MI_U16 u16BufChromaAlignment;
```

```
// Buf alignment requirement after compress
MI_U16 u16BufCompressAlignment;
// Buf Extra add after alignment
MI_U16 u16BufExtraSize;
//Clear padding flag
MI_BOOL bClearPadding;
} MI_SYS_FrameBufExtraConfig_t;
```

➤ 成员

成员名称	描述
u16BufHAlignment	水平方向对齐值
u16BufVAlignment	垂直方向对齐值
u16BufChromaAlignment	色度 buffer size 对齐值
u16BufCompressAlignment	压缩模式 buffer size 对齐值
u16BufExtraSize	buffer size 额外分配值
bClearPadding	是否对 buffer 的边缘进行涂黑

※ 注意事项

N/A

➤ 相关数据类型及接口

N/A

3.2.15 MI_SYS_BufFrameConfig_t

➤ 说明

定义码流 Frame buffer 配置的结构体。

➤ 定义

```
typedef struct MI_SYS_BufFrameConfig_s
{
    MI_U16 u16Width;
```

```
MI_U16 u16Height;
MI_SYS_FrameScanMode_e eFrameScanMode;
MI_SYS_PixelFormat_e eFormat;
MI_SYS_CompressMode_e eCompressMode;
} MI_SYS_BufFrameConfig_t;
```

➤ 成员

成员名称	描述
u16Width	Frame 宽度
u16Height	Frame 高度
eFrameScanMode	Frame Scan Mode
eFormat	Frame 像素格式
eCompressMode	Frame 压缩模式

※ 注意事项

N/A

➤ 相关数据类型及接口

N/A

3.2.16 MI_SYS_BufRawConfig_t

➤ 说明

定义码流 raw buffer 配置的结构体。

➤ 定义

```
typedef struct MI_SYS_BufRawConfig_s
{
    MI_U32 u32Size;
} MI_SYS_BufRawConfig_t;
```

➤ 成员

成员名称	描述
u32Size	Buf 大小

※ 注意事项

N/A

➤ 相关数据类型及接口

N/A

3.2.17 MI_SYS_MetaDataConfig_t

➤ 说明

定义码流 meta buffer 配置的结构体。

➤ 定义

```
typedef struct MI_SYS_MetaDataConfig_s
{
    MI_U32 u32Size;
} MI_SYS_MetaDataConfig_t;
```

➤ 成员

成员名称	描述
u32Size	Buf 大小

※ 注意事项

N/A

➤ 相关数据类型及接口

N/A

3.2.18 MI_SYS_BufConf_t

➤ 说明

定义码流 Port Buf 配置的结构体。

➤ 定义

```
typedef struct MI_SYS_BufConf_s
{
    MI_SYS_BufDataType_e eBufType;
    MI_U32 u32Flags; //0 or MI_SYS_MAP_VA
    MI_U64 u64TargetPts;
    MI_BOOL bDirectBuf; // Direct alloc buffer by Others.
    MI_BOOL bCrcCheck;
    union
    {
        MI_SYS_BufFrameConfig_t stFrameCfg;
        MI_SYS_BufRawConfig_t stRawCfg;
        MI_SYS_MetaDataConfig_t stMetaCfg;
        MI_SYS_BufFrameMultiPlaneConfig_t stMultiPlaneCfg;
        MI_SYS_BufFrameMetaConfig_t stFrameMetaCfg; // support frame buffer has allocated by
        Others.
        MI_SYS_FbcDataConfig_t stFbcCfg;
    };
} MI_SYS_BufConf_t;
```

➤ 成员

成员名称	描述
eBufType	Buf type
u32Flags	Buf 是否 map kernel space va
u64TargetPts	设置 buf 的 Pts
bDirectBuf	Direct alloc buffer by Others
bCrcCheck	Buf 是否开启 crc 校验
Union	stFrameCfg
	设置 Frame 数据格式参数

	stRawCfg	设置 Raw 数据格式参数
	stMetaCfg	设置 Meta 数据格式参数
	stMultiPlaneCfg	设置多 Frame 数据格式参数
	stFrameMetaCfg	Frame meta data 配置
	stFbcCfg	Fbc 数据配置

※ 注意事项

N/A

➤ 相关数据类型及接口

N/A

3.2.19 MI_SYS_Version_t

➤ 说明

定义 Sys 版本信息结构体。

➤ 定义

```
typedef struct MI_SYS_Version_s
{
    MI_U8 u8Version[128];
} MI_SYS_Version_t;
```

➤ 成员

成员名称	描述
u8Version[128]	描述 sys 版本信息的字符串 buf

※ 注意事项

N/A

➤ 相关数据类型及接口

N/A

3.2.20 MI_VB_PoolListConf_t

➤ 说明

定义描述 VB Pool 链表信息的结构体。

➤ 定义

```
typedef struct MI_VB_PoolListConf_s
{
    MI_U32 u32PoolListCnt;
    MI_VB_PoolConf_t stPoolConf[MI_VB_POOL_LIST_MAX_CNT];
} MI_VB_PoolListConf_t;
```

➤ 成员

成员名称	描述
u32PoolListCnt	VB pool 链表成员数量
stPoolConf	VB pool 链表成员配置信息

※ 注意事项

N/A

➤ 相关数据类型及接口

N/A

3.2.21 MI_SYS_BindType_e

➤ 说明

定义前后级工作模式。

➤ 定义

```
typedef enum
{
```

```
E_MI_SYS_BIND_TYPE_FRAME_BASE = 0x00000001,
E_MI_SYS_BIND_TYPE_SW_LOW_LATENCY = 0x00000002,
E_MI_SYS_BIND_TYPE_REALTIME = 0x00000004,
E_MI_SYS_BIND_TYPE_HW_AUTOSYNC = 0x00000008,
E_MI_SYS_BIND_TYPE_HW_RING = 0x00000010
} MI_SYS_BindType_e;
```

➤ 成员

成员名称	描述
E_MI_SYS_BIND_TYPE_FRAME_BASE	frame mode，默认工作方式
E_MI_SYS_BIND_TYPE_SW_LOW_LATENCY	低时延工作方式
E_MI_SYS_BIND_TYPE_REALTIME	硬件直连工作方式
E_MI_SYS_BIND_TYPE_HW_AUTOSYNC	前后级 handshake，buffer size 与图像分辨率一致
E_MI_SYS_BIND_TYPE_HW_RING	前后级 handshake，ring buffer depth 可以调小于图像分辨率高

※ 注意事项

- 旧版本 MI SYS 不提供此功能，如没有找到，不用设置。
- FRAME_BASE 和 HW_RING 类型使用的内存为 DRAM，REALTIME 类型使用的内存为 SRAM。
- 由于底层限制，被 HW_RING 或 REALTIME 类型 bind 的 input port 不能使用 [MI_SYS_ChnInputPortGetBuf/ MI_SYS_ChnInputPortPutBuf](#) 接口，output port 不能使用 [MI_SYS_ChnOutputPortGetBuf/ MI_SYS_ChnOutputPortPutBuf](#) 接口。
- 不同 bindtype 使用内存大小比较：FRAME_BASE 类型 > HW_RING 类型 > REALTIME 类型。

➤ 相关数据类型及接口

N/A

3.2.22 MI_SYS_FrameData_PhySignalType

➤ 说明

描述 frame data 隶属的 buffer 类型的枚举类型。

➤ 定义

```
typedef enum
{
    REALTIME_FRAME_DATA,
    RINGBUF_FRAME_DATA,
    NORMAL_FRAME_DATA,
} MI_SYS_FrameData_PhySignalType;
```

➤ 成员

成员名称	描述
REALTIME_FRAME_DATA	以 E_MI_SYS_BIND_TYPE_REALTIME 模式产生的 frame data
RINGBUF_FRAME_DATA	以 E_MI_SYS_BIND_TYPE_HW_RING 模式产生的 frame data
NORMAL_FRAME_DATA	以 E_MI_SYS_BIND_TYPE_FRAME_BASE 模式产生的 frame data

※ 注意事项

旧版本 MI SYS 不提供此功能，如没有找到，不用设置。

➤ 相关数据类型及接口

N/A

3.2.23 MI_SYS_InsidePrivatePoolType_e

➤ 说明

描述创建的私有 MMA POOL 类型的枚举类型。

➤ 定义

```
typedef enum
{
    E_MI_SYS_VPE_TO_VENC_PRIVATE_RING_POOL = 0,
    E_MI_SYS_PER_CHN_PRIVATE_POOL=1,
    E_MI_SYS_PER_DEV_PRIVATE_POOL=2,
    E_MI_SYS_PER_CHN_PORT_OUTPUT_POOL=3,
    E_MI_SYS_PER_DEV_PRIVATE_RING_POOL=4,
} MI_SYS_InsidePrivatePoolType_e;
```

➤ 成员

成员名称	描述
E_MI_SYS_VPE_TO_VENC_PRIVATE_RING_POOL	SCL 与 VENC 的私有 Ring Pool 类型 mi_sys v2.0 表示 VPE 与 VENC
E_MI_SYS_PER_CHN_PRIVATE_POOL	通道私有 Pool 类型
E_MI_SYS_PER_DEV_PRIVATE_POOL	设备私有 Pool 类型
E_MI_SYS_PER_CHN_PORT_OUTPUT_POOL	输出端口私有 Pool 类型
E_MI_SYS_PER_DEV_PRIVATE_RING_POOL	设备私有 Ring Pool 类型 在 Maruko 及其之后的 chip，在使用 HW_Ring bind 之前必须去配置该类型的 共享私有池。

※ 注意事项

旧版本 MI SYS 不提供此功能，如没有找到，不用设置。

➤ 相关数据类型及接口

N/A

3.2.24 MI_SYS_PerChnPrivHeapConf_t

➤ 说明

定义描述通道私有 MMA Pool 的结构体。

➤ 定义

```
typedef struct MI_PerChnPrivHeapConf_s
{
    MI_ModuleId_e eModule;
    MI_U32 u32Devid;
    MI_U32 u32Channel;
    MI_U8 u8MMAHeapName[MI_MAX_MMA_HEAP_LENGTH];
    MI_U32 u32PrivateHeapSize;
} MI_SYS_PerChnPrivHeapConf_t;
```

➤ 成员

成员名称	描述
eModule	模块 ID
u32Devid	设备 ID
u32Channel	通道 ID
u8MMAHeapName	Mma heap name
u32PrivateHeapSize	私有 pool 大小

※ 注意事项

N/A

➤ 相关数据类型及接口

N/A

3.2.25 MI_SYS_PerDevPrivHeapConf_t

➤ 说明

定义描述设备私有 MMA Pool 的结构体。

➤ 定义

```
typedef struct MI_PerDevPrivHeapConf_s
```



```
{
    MI_ModuleId_e eModule;
    MI_U32 u32Devid;
    MI_U32 u32Reserve;
    MI_U8 u8MMAHeapName[MI_MAX_MMA_HEAP_LENGTH];
    MI_U32 u32PrivateHeapSize;
} MI_SYS_PerDevPrivHeapConf_t;
```

➤ 成员

成员名称	描述
eModule	模块 ID
u32Devid	设备 ID
u32Reserve	预留
u8MMAHeapName	Mma heap name
u32PrivateHeapSize	私有 pool 大小

※ 注意事项

N/A

➤ 相关数据类型及接口

N/A

3.2.26 MI_SYS_PerVpe2VencRingPoolConf_t

➤ 说明

定义描述 SCL 与 VENC 之间的私有 Ring MMA Pool 的结构体。

mi_sys v2.0 表示 VPE 与 VENC。

➤ 定义

```
typedef struct MI_SYS_PerVpe2VencRingPoolConf_s
{
    MI_U32 u32VencInputRingPoolStaticSize;
```

```
MI_U8 u8MMAHeapName[MI_MAX_MMA_HEAP_LENGTH];
} MI_SYS_PerVpe2VencRingPoolConf_t;
```

➤ 成员

成员名称	描述
u32VencInputRingPoolStaticSize	私有 pool 大小
u8MMAHeapName	Mma heap name

※ 注意事项

N/A

➤ 相关数据类型及接口

N/A

3.2.27 MI_SYS_PerChnPortOutputPool_t

➤ 说明

定义描述 output port 的私有 MMA Pool 的结构体。

➤ 定义

```
typedef struct MI_SYS_PerChnPortOutputPool_s
{
    MI_ModuleId_e eModule;
    MI_U32 u32Devid;
    MI_U32 u32Channel;
    MI_U32 u32Port;
    MI_U8 u8MMAHeapName[MI_MAX_MMA_HEAP_LENGTH];
    MI_U32 u32PrivateHeapSize;
} MI_SYS_PerChnPortOutputPool_t;
```

➤ 成员

成员名称	描述
eModule	模块 ID
u32Devid	设备 ID
u32Channel	通道 ID
u32Port	端口 ID
u8MMAHeapName	Mma heap name
u32PrivateHeapSize	私有 pool 大小

※ 注意事项

N/A

➤ 相关数据类型及接口

N/A

3.2.28 MI_SYS_PerDevPrivRingPoolConf_t

➤ 说明

定义描述设备的私有 Ring Pool 的结构体。

➤ 定义

```
typedef struct MI_SYS_PerDevPrivRingPoolConf_s
{
    MI_ModuleId_e eModule;
    MI_U32      u32Devid;
    MI_U16      u16MaxWidth;
    MI_U16      u16MaxHeight;
    MI_U16      u16RingLine;
    MI_U8       u8MMAHeapName[MI_MAX_MMA_HEAP_LENGTH];
}MI_SYS_PerDevPrivRingPoolConf_t;
```

➤ 成员

成员名称	描述
eModule	模块 ID

成员名称	描述
u32DevId	设备 ID
u16MaxWidth	Ring Pool 的宽大小
u16MaxHeight	Ring Pool 的高大小
u16RingLine	Ring Pool 总行数
u8MMAHeapName	Ring Pool 的名字

※ 注意事项

N/A

➤ 相关数据类型及接口

N/A

3.2.29 MI_SYS_GlobalPrivPoolConfig_t

➤ 说明

定义描述配置私有 MMA Pool 的结构体。

➤ 定义

```
typedef struct MI_SYS_GlobalPrivPoolConfig_s
{
    MI_SYS_InsidePrivatePoolType_e eConfigType;
    MI_BOOL bCreate;
    union
    {
        MI_SYS_PerChnPrivHeapConf_t stPreChnPrivPoolConfig;
        MI_SYS_PerDevPrivHeapConf_t stPreDevPrivPoolConfig;
        MI_SYS_PerVpe2VencRingPoolConf_t stPreVpe2VencRingPrivPoolConfig;
        MI_SYS_PerChnPortOutputPool_t stPreChnPortOutputPrivPool;
        MI_SYS_PerDevPrivRingPoolConf_t stpreDevPrivRingPoolConfig;
    }uConfig;
}
```

```
} MI_SYS_GlobalPrivPoolConfig_t;
```

➤ 成员

成员名称	描述
eConfigType	私有 pool 类型
bCreate	是否创建私有 pool。true:创建 false:销毁
uConfig	不同类型私有 pool 的结构体

※ 注意事项

N/A

➤ 相关数据类型及接口

N/A

3.2.30 MI_SYS_FrameIspInfo_t

➤ 说明

定义 frame data 中 ISP info 的结构体。

➤ 定义

```
typedef struct MI_SYS_FrameIspInfo_s
{
    MI_SYS_FrameIspInfoType_e eType;
    union
    {
        MI_U32 u32GlobalGradient;
    }uIspInfo;
} MI_SYS_FrameIspInfo_t;
```

➤ 成员

成员名称	描述
eType	ISP info type

uIspInfo	ISP info union
----------	----------------

※ 注意事项

N/A

➤ 相关数据类型及接口

N/A

3.2.31 MI_SYS_FbcData_t

➤ 说明

定义 frame buffer compress 数据结构体。

➤ 定义

typedef struct MI_SYS_FbcData_s

```
{
    MI_SYS_FrameTileMode_e eTileMode;
    MI_SYS_PixelFormat_e ePixelFormat;
    MI_SYS_CompressMode_e eCompressMode;

    MI_SYS_FrameData_PhySignalType ePhylayoutType;

    MI_U16 u16Width;
    MI_U16 u16Height;

    void* pVirAddr[2];
    MI_PHY phyAddr[2]; // notice that this is miu bus addr,not cpu bus addr.
    MI_U32 u32Stride[2];
    MI_U32 u32BufSize; // total size that allocated for this buffer,include consider alignment.

    /*
    * -----
```

```

*          |          |          ^
*          |          |          |
*  u16RingBufStartLine --> |          |----          |
*          |          | ^          |
*          |  Ring    | |          |
*          |          | u16Height  |
*          |  Heap    | |          |
*          |          | |          u16RingHeapTotalLines
*          |          | v          |
*  u16RingBufEndLine --> |          |----          |
*          |          |          |
*          |          |          v
*          -----
*/

```

```

MI_U16 u16RingBufStartLine;
MI_U16 u16RingBufEndLine;
MI_U16 u16RingHeapTotalLines;

```

```

MI_PHY phyFbcTableAddr[2];
MI_U32 u32FbcTableSize[2];
} MI_SYS_FbcData_t;

```

➤ 成员

成员名称	描述
eTileMode	Tile 格式类型
ePixelFormat	像素枚举类型
eCompressMode	压缩方式类型
ePhylayoutType	Frame data 隶属的 buffer 类型
u16Width	Frame data 的宽
u16Height	Frame data 的高
pVirAddr[2]	Frame data buffer 的虚拟地址

phyAddr[2]	Frame data buffer 的物理地址
u32Stride[2]	Frame data 每行所占字节数
u32BufSize	Frame data buffer 大小
u16RingBufStartLine	Ring buffer 起始行
u16RingBufEndLine	Ring buffer 结束行
u16RingHeapTotalLines	Ring buffer 总行数
phyFbcTableAddr[2]	把 Fbc data 转成 nv12 的 table，目前只支持转到 nv12。
u32FbcTableSize[2]	Fbc data 转成 nv12 的 table 大小。

※ 注意事项

N/A

➤ 相关数据类型及接口

N/A

3.2.32 MI_SYS_BufFrameMultiPlaneConfig_t

➤ 说明

定义码流 multiplane buffer 配置的结构体。

➤ 定义

```
typedef struct MI_SYS_BufFrameMultiPlaneConfig_s
{
    MI_U8 u8SubPlaneNum;
    MI_SYS_BufFrameConfig_t stFrameCfg;
} MI_SYS_BufFrameMultiPlaneConfig_t;
```

➤ 成员

成员名称	描述
u8SubPlaneNum	Buf 个数
stFrameCfg	定义码流 Frame buffer 配置的结构体

※ 注意事项

N/A

➤ 相关数据类型及接口

[N/A](#)

3.2.33 MI_SYS_FrameDataSubPlane_t

➤ 说明

定义码流 SubPlane FrameData 的结构体。

➤ 定义

```
typedef struct MI_SYS_FrameDataSubPlane_s
{
    MI_SYS_PixelFormat_e ePixelFormat;
    MI_SYS_CompressMode_e eCompressMode;
    MI_U64 u64FrameId;

    MI_U16 u16Width;
    MI_U16 u16Height;
    void* pVirAddr[2];
    MI_PHY phyAddr[2];
    MI_U16 u16Stride[2];
    MI_U32 u32BufSize;
} MI_SYS_FrameDataSubPlane_t;
```

➤ 成员

成员名称	描述
ePixelFormat	像素格式
eCompressMode	压缩格式
u64FrameId	Frame id
u16Width	Frame 宽度

成员名称	描述
u16Height	Frame 高度
pVirAddr	虚拟地址
phyAddr	物理地址
U16Stride	图片每行所占字节数
u32BufSize	Sys 分配给当前 frame 的实际 buf 大小

※ 注意事项

N/A

➤ 相关数据类型及接口

[N/A](#)

3.2.34 MI_SYS_BufFrameMetaConfig_t

➤ 说明

定义 frame meta data 的结构体。

➤ 定义

```
typedef struct MI_SYS_BufFrameMetaConfig_s
{
    MI_U16          u16Width;
    MI_U16          u16Height;
    MI_SYS_FrameScanMode_e eFrameScanMode; //
    MI_SYS_PixelFormat_e  eFormat;
    MI_SYS_CompressMode_e eCompressMode;

    void* pVirAddr[3];
    MI_PHY phyAddr[3];
    MI_U16 u32Stride[3];
    MI_U32 u32BufSize;
} MI_SYS_BufFrameMetaConfig_t;
```

➤ 成员

成员名称	描述
u16Width	数据的宽
u16Height	数据的高
eFrameScanMode	逐行扫描或者隔行扫描的模式
eFormat	像素格式
eCompressMode	压缩模式
pVirAddr[3]	虚拟地址
phyAddr[3]	物理地址
u32Stride[3]	图片每行所占字节数
u32BufSize	Sys 分配给当前 frame 的实际 buf 大小

※ 注意事项

N/A

➤ 相关数据类型及接口

[N/A](#)

3.2.35 MI_SYS_FrameDataMultiPlane_t

➤ 说明

定义码流 MultiPlane FrameData 的结构体。

➤ 定义

```
typedef struct MI_SYS_FrameDataMultiPlane_s
{
    MI_U8 u8SubPlaneNum;
    MI_SYS_FrameDataSubPlane_t stSubPlanes[MI_SYS_MAX_SUB_PLANE_CNT];
} MI_SYS_FrameDataMultiPlane_t;
```

➤ 成员

成员名称	描述
u8SubPlaneNum	Buf 个数
stSubPlanes	定义码流 MultiPlane FrameData 的结构体

※ 注意事项

N/A

➤ 相关数据类型及接口

[N/A](#)

3.2.36 MI_SYS_UserPictureInfo_t

➤ 说明

定义描述 user picture 信息的结构体。

➤ 定义

```
typedef struct MI_SYS_UserPictureInfo_s
{
    MI_U32 u32SrcFrc;
} MI_SYS_UserPictureInfo_t;
```

➤ 成员

成员名称	描述
u32SrcFrc	设定 Input port buf 输入的帧率

※ 注意事项

N/A

➤ 相关数据类型及接口

[N/A](#)

4. 错误码

4.1. 错误码的组成

MI 返回的错误码为 4 字节，共 32bits，由表 4-1 所示五部分组成：

表 4-6：错误码的组成

位置	所占位数	描述
BIT [0-11]	12bits	错误类型，表示该错误码的具体错误含义。
BIT [12-15]	4bits	错误等级，固定返回 2。
BIT [16-23]	8bits	模块 ID，表示该错误码是属于哪个模块。
BIT [24-31]	8bits	固定为 0xA0。

Tips:

我们可以简单的把错误码看成两个双字节（16bits）做快速解读，以错误码 0xA0092001 为例：

"A009"：表示该错误发生在模块 ID 为 9 的模块，通过查看“[MI_ModuleId_e](#)”的定义可知为 MI_SYS 模块。

"2001"：表示该错误的错误类型是 1，即“设备 ID 超出合法范围”。

4.2. MI_SYS API 错误码表

MI_SYS 模块所有 API 返回的值都为 4 字节，0 表示执行成功，其它值表示执行失败，需要注意的是，所有非 0 的返回值都应该是表 4-2 中列出的，否则为非法值。

表 4-7：MI_SYS 模块 API 错误码

错误码	宏定义	描述
0xA0092001	MI_ERR_SYS_INVALID_DEVID	设备 ID 超出合法范围
0xA0092002	MI_ERR_SYS_INVALID_CHNID	通道组号错误或无效区域句柄
0xA0092003	MI_ERR_SYS_ILLEGAL_PARAM	参数超出合法范围
0xA0092004	MI_ERR_SYS_EXIST	重复创建已存在的设备、通道或资源
0xA0092005	MI_ERR_SYS_UNEXIST	试图使用或者销毁不存在的设备、通道或者资源
0xA0092006	MI_ERR_SYS_NULL_PTR	函数参数中有空指针
0xA0092007	MI_ERR_SYS_NOT_CONFIG	模块没有配置
0xA0092008	MI_ERR_SYS_NOT_SUPPORT	不支持的参数或者功能

错误码	宏定义	描述
0xA0092009	MI_ERR_SYS_NOT_PERM	该操作不允许，如试图修改静态配置参数
0xA009200C	MI_ERR_SYS_NOMEM	分配内存失败，如系统内存不足
0xA009200D	MI_ERR_SYS_NOBUF	分配缓存失败，如申请的数据缓冲区太大
0xA009200E	MI_ERR_SYS_BUF_EMPTY	缓冲区中无数据
0xA009200F	MI_ERR_SYS_BUF_FULL	缓冲区中数据满
0xA0092010	MI_ERR_SYS_NOTREADY	系统没有初始化或没有加载相应模块
0xA0092011	MI_ERR_SYS_BADADDR	地址非法
0xA0092012	MI_ERR_SYS_BUSY	系统忙
0xA0092013	MI_ERR_SYS_CHN_NOT_STARTED	通道没有开始
0xA0092014	MI_ERR_SYS_CHN_NOT_STOPED	通道没有停止
0xA0092015	MI_ERR_SYS_NOT_INIT	模块没有初始化
0xA0092016	MI_ERR_SYS_INITED	模块已经初始化
0xA0092017	MI_ERR_SYS_NOT_ENABLE	通道或端口没有 ENABLE
0xA0092018	MI_ERR_SYS_NOT_DISABLE	通道或端口没有 DISABLE
0xA0092019	MI_ERR_SYS_TIMEOUT	超时
0xA009201A	MI_ERR_SYS_DEV_NOT_STARTED	设备没有开始
0xA009201B	MI_ERR_SYS_DEV_NOT_STOPED	设备没有停止
0xA009201C	MI_ERR_SYS_CHN_NO_CONTENT	通道没有资料
0xA009201D	MI_ERR_SYS_NOVASAPCE	映射虚拟地址失败
0xA009201E	MI_ERR_SYS_NOITEM	RingPool 中没有 record 记录
0xA009201F	MI_ERR_SYS_FAILED	未明确定义的错误

5. PROCFS 介绍

5.1. common

5.1.1 dump_mmap

➤ 调试信息

/config/dump_mmap

```
/ # /config/dump_mmap
u32TotalSize:0x40000000
u32Miu0Size:0x40000000
u32Miu1Size:0x0
u32MiuBoundary:0x80000000
u32MmapItemsNum:4
bIs4kAlign:1
bMiu1Enable:0
E_LX_MEM:GID=0,Addr=0x0,Size=0x3ffe0000,Layer=0,Align=0x1000,MemoryType=0x4,MiuNo=0,CMAID=0
E_MMAP_ID_DUMMY1:GID=1,Addr=0x0,Size=0x3f000000,Layer=1,Align=0x1000,MemoryType=0x4,MiuNo=0,CMAID=0
E_LX_LOGO_RESERVED_FB:GID=2,Addr=0x3f000000,Size=0x800000,Layer=1,Align=0x1000,MemoryType=0x4,MiuNo=0,CMAID=0
E_MMAP_ID_EMI:GID=3,Addr=0x3ffe0000,Size=0x20000,Layer=0,Align=0x1000,MemoryType=0x4,MiuNo=0,CMAID=0
```

➤ 调试信息分析

./config/dump_mmap 根据文件./config/mmap.ini 显示内存相关信息。

➤ 参数分析

参数		描述
u32TotalSize		Dram 总大小
u32Miu0Size		MIU0 的寻址大小
u32Miu1Size		MIU1 的寻址大小
u32MiuBoundary		MIU0 end address
u32MmapItemsNum		将 Dram mmap 的个数，这里是 2 个，分别是 E_LX_MEM 和 E_MMAP_ID_EMI
bIs4kAlign		读写 Dram 是否需要 4K 对齐
bMiu1Enable		MIU1 是否 enable
E_LX_MEM	GID	Mmap id
	Addr	Memory address
	Size	Memory size
	Layer	Memory layer
	Align	Memory align
	MemoryType	Memory type
e		

参数		描述
	MiuNo	MIU NO.
	CMAID	CMA id
E_MMAP_ID_DUMMY1		同上
E_LX_LOGO_RESERVED_FB		同上
E_MMAP_ID_EMI		同上

5.1.2 dump_config

➤ 调试信息

/config/dump_config

panel cnt:59

PanelName:RM68200_720x1280 Intftype:11 timing:55

PanelName:SAT070AT50H18BH_1024x600 Intftype:0 timing:55

PanelName:BT1120OUTPUT_720p_60HZ Intftype:14 timing:55

PanelName:BT656OUTPUT_576p_50HZ Intftype:12 timing:55

PanelName:ST7789V Intftype:16 timing:55

PanelName:ILI9881C_800x1280 Intftype:11 timing:55

PanelName:ILI9163C_128x128 Intftype:15 timing:55

PanelName:DACOUT_576I Intftype:7 timing:4

PanelName:DACOUT_480I Intftype:7 timing:2

PanelName:DACOUT_1080P_24 Intftype:8 timing:10

PanelName:DACOUT_1080P_25 Intftype:8 timing:11

PanelName:DACOUT_1080P_30 Intftype:8 timing:12

PanelName:DACOUT_1080P_50 Intftype:8 timing:15

PanelName:DACOUT_720P_2997 Intftype:8 timing:6

PanelName:DACOUT_720P_50 Intftype:8 timing:7

PanelName:DACOUT_720P_5994 Intftype:8 timing:8

PanelName:DACOUT_720P_60 Intftype:8 timing:9

PanelName:DACOUT_1080P_2997 Intftype:8 timing:13

PanelName:DACOUT_1080I_50 Intftype:7 timing:14

PanelName:DACOUT_1080P_5994 Intftype:8 timing:16
PanelName:DACOUT_1080I_60 Intftype:7 timing:17
PanelName:DACOUT_1080P_60 Intftype:8 timing:18
PanelName:DACOUT_576P Intftype:8 timing:5
PanelName:DACOUT_480P Intftype:8 timing:3
PanelName:DACOUT_2K2KP_30 Intftype:8 timing:27
PanelName:DACOUT_2K2KP_60 Intftype:8 timing:28
PanelName:DACOUT_4K2KP_24 Intftype:8 timing:33
PanelName:DACOUT_4K2KP_25 Intftype:8 timing:34
PanelName:DACOUT_4K2KP_30 Intftype:8 timing:35
PanelName:DACOUT_4K2KP_50 Intftype:8 timing:36
PanelName:DACOUT_4K2KP_60 Intftype:8 timing:37
PanelName:DACOUT_1440P_50 Intftype:8 timing:19
PanelName:DACOUT_1440P_60 Intftype:8 timing:20
PanelName:VGAOUT_1024x768P_60 Intftype:8 timing:45
PanelName:VGAOUT_1280x1024P_60 Intftype:8 timing:48
PanelName:VGAOUT_1366x768P_60 Intftype:8 timing:46
PanelName:VGAOUT_1440x900P_60 Intftype:8 timing:49
PanelName:VGAOUT_1280x800P_60 Intftype:8 timing:47
PanelName:VGAOUT_1680x1050P_60 Intftype:8 timing:51
PanelName:VGAOUT_1600x1200P_60 Intftype:8 timing:50
PanelName:BT1120OUT_576P_50 Intftype:14 timing:5
PanelName:BT1120OUT_480P_60 Intftype:14 timing:3
PanelName:BT1120OUT_1080P_25 Intftype:14 timing:11
PanelName:BT1120OUT_1080P_30 Intftype:14 timing:12
PanelName:BT1120OUT_1080P_50 Intftype:14 timing:15
PanelName:BT1120OUT_1080P_2997 Intftype:14 timing:13
PanelName:BT1120OUT_1080P_5994 Intftype:14 timing:16
PanelName:BT1120OUT_1080P_60 Intftype:14 timing:18
PanelName:BT1120OUT_720P_50 Intftype:14 timing:7
PanelName:BT1120OUT_720P_5994 Intftype:14 timing:8
PanelName:BT1120OUT_720P_60 Intftype:14 timing:9
PanelName:BT656OUT_576P_50 Intftype:12 timing:5

```
PanelName:BT656OUT_480P_60 Intftype:12 timing:3
PanelName:BT656OUT_1080P_25 Intftype:12 timing:11
PanelName:BT656OUT_1080P_2997 Intftype:12 timing:13
PanelName:BT656OUT_1080P_30 Intftype:12 timing:12
PanelName:BT656OUT_720P_50 Intftype:12 timing:7
PanelName:BT656OUT_720P_60 Intftype:12 timing:9
PanelName:BT656OUT_720P_5994 Intftype:12 timing:8
pq enable:0
pq table size:25892
DBC Value=
    0,0,0
    0,0,0
    0,0,0
start dump [motion_table](0, 0)
end dump
start dump [motion_hdmi_dtv_table](0, 0)
end dump
start dump [motion_comp_pc_table](0, 0)
end dump
start dump [misc_param_table](0, 0)
end dump
start dump [misc_luma_table](0, 0)
end dump
start dump [noise_table](0, 0)
end dump
start dump [misc_table](0, 0)
end dump
```

➤ 调试信息分析

./config/dump_config 得到的是/proc/mi_modules/common/里各文件的信息，含 panel size、DBC

Value、motion_table、motion_hdmi_dtv_table、motion_comp_pc_table、misc_param_table、misc_luma_table、noise_table、misc_table 等。

➤ 参数分析

参数		描述
panel	PanelName	LCD 屏名称
	Intftype	LCD 屏接口类型: E_MAPI_PNL_INTF_TTL = 0, ///< TTL type E_MAPI_PNL_INTF_LVDS, ///< LVDS type E_MAPI_PNL_INTF_RSDS, ///< RSDS type E_MAPI_PNL_INTF_MINILVDS, ///< TCON ///< Analog TCON E_MAPI_PNL_INTF_ANALOG_MINILVDS, ///< Digital TCON E_MAPI_PNL_INTF_DIGITAL_MINILVDS, E_MAPI_PNL_INTF_MFC, ///< Ursa (TTL output to Ursa) E_MAPI_PNL_INTF_DAC_I, ///< DAC output E_MAPI_PNL_INTF_DAC_P, ///< DAC output ///< For PDP(Vsync use Manually MODE) E_MAPI_PNL_INTF_PDPLVDS, E_MAPI_PNL_INTF_EXT, ///< EXT LPLL TYPE E_MAPI_PNL_INTF_MIPI_DSI, E_MAPI_PNL_INTF_BT656, E_MAPI_PNL_INTF_BT601, E_MAPI_PNL_INTF_BT1120, ///< BT1120 E_MAPI_PNL_INTF_MCU_TYPE, ///< MCU Type E_MAPI_PNL_INTF_SRGB, ///< sRGB E_MAPI_PNL_INTF_TTL_SPI_IF,
	timing	LCD 屏显示输出时序: DISPLAYTIMING_MIN = 0, DISPLAYTIMING_DACOUT_DEFAULT = 0, DISPLAYTIMING_DACOUT_FULL_HD, DISPLAYTIMING_DACOUT_480I,

参数	描述
	DISPLAYTIMING_DACOUT_480P, DISPLAYTIMING_DACOUT_576I, DISPLAYTIMING_DACOUT_576P, // 5 DISPLAYTIMING_DACOUT_720P_2997, DISPLAYTIMING_DACOUT_720P_50, DISPLAYTIMING_DACOUT_720P_5994, DISPLAYTIMING_DACOUT_720P_60, DISPLAYTIMING_DACOUT_1080P_24, DISPLAYTIMING_DACOUT_1080P_25, DISPLAYTIMING_DACOUT_1080P_30, // 12 DISPLAYTIMING_DACOUT_1080P_2997, DISPLAYTIMING_DACOUT_1080I_50, DISPLAYTIMING_DACOUT_1080P_50, DISPLAYTIMING_DACOUT_1080P_5994, DISPLAYTIMING_DACOUT_1080I_60, DISPLAYTIMING_DACOUT_1080P_60, DISPLAYTIMING_DACOUT_1440P_50, DISPLAYTIMING_DACOUT_1440P_60, DISPLAYTIMING_DACOUT_1470P_24, DISPLAYTIMING_DACOUT_1470P_30, DISPLAYTIMING_DACOUT_1470P_60, DISPLAYTIMING_DACOUT_2205P_24, // For 2k2k DISPLAYTIMING_DACOUT_2K2KP_24, // 23 DISPLAYTIMING_DACOUT_2K2KP_25, DISPLAYTIMING_DACOUT_2K2KP_30, DISPLAYTIMING_DACOUT_2K2KP_60, // For 4k0.5k DISPLAYTIMING_DACOUT_4K540P_240, // For 4k1k DISPLAYTIMING_DACOUT_4K1KP_30,

参数	描述
	DISPLAYTIMING_DACOUT_4K1KP_60, DISPLAYTIMING_DACOUT_4K1KP_120, // For 4k2k DISPLAYTIMING_DACOUT_4K2KP_24, DISPLAYTIMING_DACOUT_4K2KP_25, DISPLAYTIMING_DACOUT_4K2KP_30, // 33 DISPLAYTIMING_DACOUT_4K2KP_50, DISPLAYTIMING_DACOUT_4K2KP_60, DISPLAYTIMING_DACOUT_4096P_24, DISPLAYTIMING_DACOUT_4096P_25, DISPLAYTIMING_DACOUT_4096P_30, DISPLAYTIMING_DACOUT_4096P_50, DISPLAYTIMING_DACOUT_4096P_60, // For VGA OUTPUT DISPLAYTIMING_VGAOUT_640x480P_60, DISPLAYTIMING_VGAOUT_1280x720P_60, DISPLAYTIMING_VGAOUT_1024x768P_60, DISPLAYTIMING_VGAOUT_1366x768P_60, DISPLAYTIMING_VGAOUT_1280x800P_60, DISPLAYTIMING_VGAOUT_1280x1024P_60, DISPLAYTIMING_VGAOUT_1440x900P_60, DISPLAYTIMING_VGAOUT_1600x1200P_60, DISPLAYTIMING_VGAOUT_1680x1050P_60, DISPLAYTIMING_VGAOUT_1920x1080P_60, // 43 // For TTL output DISPLAYTIMING_TTLOUT_480X272_60, DISPLAYTIMING_DACOUT_4K2KP_120, DISPLAYTIMING_USER, DISPLAYTIMING_MAX_NUM,
pq enable	判断 pq 是否 enable
pq table size	pq table size 的值

参数		描述
DBC Value		暂时无效果
motion_table		暂时无效果
motion_hdmi_dtv_table		暂时无效果
motion_comp_pc_table		暂时无效果
misc_param_table		暂时无效果
misc_luma_table		暂时无效果
noise_table		暂时无效果
misc_table		暂时无效果

5.2. mi_log_info

5.2.1 cat

➤ 调试信息

```
# cat /proc/mi_modules/mi_log_info
```

```
----- Log Path -----
```

```
log path:
```

```
----- Store Path -----
```

```
store path: /mnt
```

```
----- Module Log Level -----
```

```
Log module      Level
```

```
-----
```

```
mi_ive          2(WRN)
```

```
mi_vdf          2(WRN)
```

```
mi_venc         2(WRN)
```

```
mi_rgn          2(WRN)
```

```
mi_ai           2(WRN)
```

```
mi_ao           2(WRN)
```

```
mi_vif          2(WRN)
```

```
mi_vpe          2(WRN)
```

```
mi_vdec         2(WRN)
```

```
mi_sys          2(WRN)
```

mi_fb	2(WRN)
mi_hdmi	2(WRN)
mi_divp	2(WRN)
mi_gfx	2(WRN)
mi_vdisp	2(WRN)
mi_disp	2(WRN)
mi_os	2(WRN)
mi_iae	2(WRN)
mi_md	2(WRN)
mi_od	2(WRN)
mi_shadow	2(WRN)
mi_warp	2(WRN)
mi_uac	2(WRN)
mi_ldc	2(WRN)
mi_sd	2(WRN)
mi_panel	2(WRN)
mi_cipher	2(WRN)
mi_sensor	2(WRN)
mi_wlan	2(WRN)
mi_ipu	2(WRN)
mi_mipitx	2(WRN)
mi_gyro	2(WRN)
mi_jpd	2(WRN)
mi_isp	2(WRN)
mi_scl	2(WRN)
mi_wbc	2(WRN)
mi_dsp	2(WRN)
mi_pcie	2(WRN)
mi_dummy	2(WRN)

help example:

```
echo mi_sys=2 > /proc/mi_modules/mi_log_info
```

```
echo mi_vdisp=1 > /proc/mi_modules/mi_log_info
```

```
echo log=/mnt > /proc/mi_modules/mi_log_info
echo storepath=/mnt > /proc/mi_modules/mi_log_info
```

➤ 调试信息分析

示意了默认的 log path 或 store path，示意了各 module 的 debug_level 的值，同时给出了 help example。

➤ 参数分析

参数		描述
mi_log_info	log path	暂时无效果
	store path	暂时无效果
	Log module	各个模块的名称
	Level	打印等级

5.2.2 echo

功能	修改模块 debug_level																							
命令	echo [ModID]=[Level] > /proc/mi_modules/mi_log_info																							
	[ModID] 模块的名字 <table><tr><td>mi_ive</td><td>mi_vdf</td><td>mi_venc</td><td>mi_rgn</td></tr><tr><td>mi_ai</td><td>mi_ao</td><td>mi_vif</td><td>mi_shadow</td></tr><tr><td>mi_vdec</td><td>mi_sys</td><td>mi_fb</td><td>mi_hdmi</td></tr><tr><td>mi_gfx</td><td>mi_vdisp</td><td>mi_disp</td><td>mi_od</td></tr><tr><td>mi_os</td><td>mi_iae</td><td>mi_md</td><td></td></tr></table>				mi_ive	mi_vdf	mi_venc	mi_rgn	mi_ai	mi_ao	mi_vif	mi_shadow	mi_vdec	mi_sys	mi_fb	mi_hdmi	mi_gfx	mi_vdisp	mi_disp	mi_od	mi_os	mi_iae	mi_md	
mi_ive	mi_vdf	mi_venc	mi_rgn																					
mi_ai	mi_ao	mi_vif	mi_shadow																					
mi_vdec	mi_sys	mi_fb	mi_hdmi																					
mi_gfx	mi_vdisp	mi_disp	mi_od																					
mi_os	mi_iae	mi_md																						
参数说明	[Level] 0 无 Debug 信息 (MI_DBG_NONE) 1 只显示 error 的信息 (MI_DBG_ERR) 2 显示 level<=2 的信息 (MI_DBG_WRN) 3 显示 level<=3 的信息 (MI_DBG_API) 4 显示 level<=4 的信息 (MI_DBG_KMSG) 5 显示 level<=5 的信息 (MI_DBG_INFO) 6 显示 level<=6 的信息 (MI_DBG_DEBUG) 7 显示 level<=7 的信息 (MI_DBG_TRACE) 8 显示所有信息 (MI_DBG_ALL)																							

举例	echo mi_sys=2 > /proc/mi_modules/mi_log_info mi_sys 的 debug_level 修改为 2
----	--

功能	修改 log 的路径
命令	echo log=[Path] > /proc/mi_modules/mi_log_info
参数说明	[Path] 路径
举例	echo log=/mnt > /proc/mi_modules/mi_log_info

功能	修改存储 log 的路径
命令	echo storepath=[Path] > /proc/mi_modules/mi_log_info
参数说明	[Path] 路径
举例	echo storepath=/mnt > /proc/mi_modules/mi_log_info

5.3. mi_global_info

5.3.1 cat

➤ 调试信息

```
# cat /proc/mi_modules/mi_global_info
```

```
miu_and_lx_info:
```

```
ARM_MIU0_BUS_BASE 0x20000000    ARM_MIU0_BASE_ADDR 0x0
```

```
ARM_MIU1_BUS_BASE 0x20000000    ARM_MIU1_BASE_ADDR 0x80000000
```

```
ARM_MIU2_BUS_BASE 0xffffffffffff ARM_MIU2_BASE_ADDR 0xffffffffffff
```

```
lx_mem_addr 0x20000000    lx_mem_size 0x3ffe0000
```

```
lx_mem2_addr 0xffffffff    lx_mem2_size 0xffffffff
```

```
lx_mem3_addr 0xffffffff    lx_mem3_size 0xffffffff
```

KernelProtect IP white list:

clientId	name
0x70	CPU_W
0x0f	MCU51_RW
0x12	USB20_H_RW
0x11	USB20_RW

0x10	USB30_RW
0x17	EMAC_RW
0x18	EMAC1_RW
0x13	SD30_RW
0x14	SDIO30_RW
0x04	AESDMA_RW
0x07	URDMA_RW
0x02	BDMA_RW
0x03	MOVDMA0_RW
0x16	SATA_RW
0x00	NONE
0x00	NONE

MmuProtect IP white list:

clientId	name
0x05	CMDQ0_R
0x06	CMDQ1_R
0x70	CPU_W
0x73	CPU_R
0x20	VCODEC_R
0x21	VCODEC_W
0x00	NONE
0x00	NONE
0x00	NONE
0x00	NONE
0x00	NONE
0x00	NONE
0x00	NONE
0x00	NONE
0x00	NONE
0x00	NONE
0x00	NONE

Sigmastar Module mi_sys version: project_commit.81f869a sdk_commit.513c3cc
build_time.20211029104555

➤ 调试信息分析

该调试信息提供了全局 **global** 的一些信息。

➤ 参数分析

参数		描述
miu_and_lx_info(以只有一个 MIU 为例)	ARM_MIU0_BUS_BASE	Miu0 bus base
	ARM_MIU0_BASE_ADDR	Miu0 base addr
	lx_mem_addr	Linux 镜像占的 memory 的起始地址 (属于 cpu bus address)
	lx_mem_size	Linux 镜像占的 memory 的 size
kernelProtect IP white list	clientId	Miu protect 的 IP 白名单里的 IP 的 id (从未分 group 的角度看的全局的 id)
	name	与 clientId 对应的该 IP 的实际的名字
	PAGE_OFFSET	the virtual address of the start of the kernel image
	TASK_SIZE	the maximum size of a user space task
	VMALLOC_START	the vmalloc memory start address
	VMALLOC_END	the vmalloc memory end address
Sigmastar Module mi_sys version	project_commit	project 对应的 commit
	sdk_commit	sdk 对应的 commit
	build_time	sdk 的 build 时间

5.4. mi_bufqueue_status

5.4.1 cat

➤ 调试信息

cat /proc/mi_modules/mi_bufqueue_status

```

/ # cat /proc/mi_modules/mi_bufqueue_status
dump Queues in input port only for enabled port:
  ModId  DevId  ChnId  PassId  InPortId  UsrInjectQ_cnt  BindInQ_cnt  InputPendingQueueSize
  33      0      0      0        0          0           2           0
  34      0      0      0        0          0           4           0
  2       0      0      0        0          0           0           0
dump Queues in output port only for enabled port:
  ModId  DevId  ChnId  PassId  OutPortId  DrvBkRefFifoQ_cnt  DrvBkRefFifoQ_size  UsrGetFifoQ_cnt  UsrGetFifoQ_size
  6       0      0      0        0          0           0           0           0
  33      0      0      0        0          0           0           0           0
  34      0      0      0        0          0           0           0           0

```

➤ 调试信息分析

Dump 当前各 modules 的 enable input/output port 相关 Queue 的信息。

参数分析

参数		描述
dump Queues in input port only for enabled port	ModId	该 input port 所在的 module id。
	DevId	该 input port 所在的 device id。
	ChnId	该 input port 所在的 channel id。
	PassId	该 input port 所在的 pass id。
	InPortId	该 input port 的 id。
	UsrInjectQ_cnt	该 InputPort 中 UsrInjectBufQueue 里的 buffer 数目。
	BindInQ_cnt	该 InputPort 中 UsrPipeInBufQueue 里的 buffer 数目。
	InputPendingQueueSize	该 InputPort 的 cur_working_input_queue 里的 buffer 总 size。

参数		描述
dump Queues in output port only for enabled port	ModId	该 output port 所在的 module id。
	DevId	该 output port 所在的 device id。
	ChnId	该 output port 所在的 channel id。
	PassId	该 output port 所在的 pass id。
	OutPortId	该 output port 的 id。
	DrvBkRefFifoQ_cnt	该 OutputPort 的 DrvBkRefFifoQueue 里的 buffer 数目。
	DrvBkRefFifoQ_size	该 OutputPort 的 DrvBkRefFifoQueue

		里的 buffer 的总 size。
	UsrGetFifoQ_cnt	该 OutputPort 的 UsrGetFifoBufQueue 里的 buffer 数目。
	UsrGetFifoQ_size	该 OutputPort 的 UsrGetFifoBufQueue 里的 buffer 的总 size。

5.5. mi_dump_buffer_delay_worker

5.5.1 cat

➤ 调试信息

```
# cat /proc/mi_modules/mi_dump_buffer_delay_worker
```

```
delay_worker_id module_name force_stop dev_id chn_id port_type port_id Queue_name
0 mi_disp 0 0 0 inport 0 BindInput
stored_dir dump_method dump_method_value
/mnt bunfnum 2
```

➤ 调试信息分析

/proc/mi_modules/mi_dump_buffer_delay_worker 文件是和 mi_modulenameid 相连的一个概念，dump device 里的 Queue 里的 buffer 采用的是 delay worker 的方式实现的，可以通过对 mi_dump_buffer_delay_worker 的 cat 操作查看还有哪些 delay worker 正在进行中，echo force_stop_dump delay_worker_id 强制结束指定的一个 delay worker 过程。

➤ 参数说明

参数	描述
delay_worker_id	该 delay worker 的 id。如果 delay worker 完成的话，该 id 会回收，给后续其他的 delay worker 作为 id 使用。
module_name	该 delay worker 所对应的 module 的 id。
force_stop	该 delay worker 当前是否处于强制结束阶段，1 表示已经被设置了强制结束，0 表示未被设置。
dev_id	该 delay worker 所对应的 device id。

参数	描述
chn_id	该 delay worker 所对应的 channel id。
port_type	该 delay worker 所对应的 port 的 type, 是 input port 或 outputport。
port_id	该 delay worker 所对应的 port 的 id。
Queue_name	该 delay worker 所对应的 Queue 的 name。
stored_dir	该 delay worker 所对应的要 dump 的 data 所要存放的绝对路径（不含最终存放的文件名）。
dump_method	该 delay worker 所对应的 dump 方法: bufnum 或 time 或 start。
dump_method_value	Dump 方法所对应的值。

5.5.2 echo

功能	强制结束 delay worker
命令	echo force_stop_dump [WorkID] >/proc/mi_modules/mi_dump_buffer_delay_worker
参数说明	[WorkID]: delay_work 的 ID
举例	echo force_stop_dump 0 > /proc/mi_modules/mi_dump_buffer_delay_worker 强制停止 delay_work 0

5.6. mi_infer_graph.py

5.6.1 cat

➤ 调试信息

```
# cat /proc/mi_modules/mi_infer_graph.py
```

```

# cat /proc/mi_modules/mi_infer_graph.py
"""
Sigmaster mi_sys infergraph debug utility.
please run it on linux with python installed.
Please run 'sudo apt-get install python-pydot' to make sure python-pydot been i
nstalled at first.
"""
try:
    # Try to load pydotplus
    import pydotplus as pydot
except ImportError:
    import pydot

def main():
    file_name = __file__
    base_name = file_name[file_name.find('.')]
    output_filename = base_name + ".mi_sys_infer_DAG.png"
    pydot_graph = pydot.Dot('infer_graph', graph_type='digraph', rankdir='LR')
    pydot_nodes = {}
    for node in pydot_nodes.values():
        pydot_graph.add_node(node)

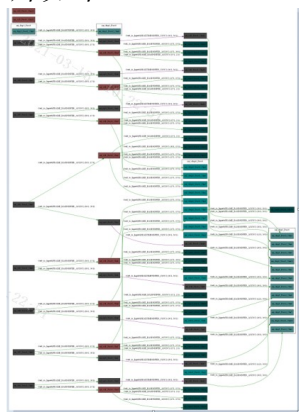
    ext = output_filename[output_filename.rfind('.')+1:]

    with open(output_filename, 'wb') as fid:
        fid.write(pydot_graph.create(format=ext))

```

➤ 调试信息分析

sys 提供 `proc fs` 生成运行时信息的 python 脚本，可以将其 `cat` 出来保存到 linux server 上，然后将其拷贝到文本里，然后在 Linux server 上使用 `pyhton2` 运行，会输出两张图片，默认的图片格式是简化版，一张是 “infer_DAG”，一张是 “perf_static”，其中一张输出的图片如下：



矩形框中表示的是 channel pass name;

线上的 label 表示的是从哪个 input port 到哪个 output port，以及 bind type，和 source frame rate、dest frame rate。

5.7. mi_graph_ctrl

5.7.1 cat

➤ 调试信息

```
# cat /proc/mi_modules/mi_graph_ctrl
```

```
/ # echo dump bind off > /proc/mi_modules/mi_graph_ctrl
/ # cat /proc/mi_modules/mi_graph_ctrl
dump bind off
```

➤ 调试信息分析

上述命令的“dump bind”字段表示 mi_infer_graph.py 脚本输出的样式，“off”表示输出简化绑定关系图，“on”表示输出详细绑定关系图。

5.7.2 echo

功能	控制 python 脚本输出的图片样式
命令	echo [command] >/proc/mi_modules/mi_graph_ctrl
参数说明	[command] 命令: dump bind on:控制输出详细绑定关系图 dump bind off:控制输出简化绑定关系图
举例	输出详细绑定关系图: # echo dump bind on > /proc/mi_modules/mi_graph_ctrl # cat /proc/mi_modules/mi_infer_graph.py

5.8. debug_level

5.8.1 cat/echo

➤ 调试信息

```
# cat /proc/mi_modules/mi_sys/debug_level
```

2

➤ 调试信息分析

每个 module（包括 sys 这个 module）都有各自的 debug level，是为了控制打印级别，各自的打印级别分别在 /proc/mi_modules/mi_modulename/debug_level 控制，其中 modulename 形如 disp ,rgn 等等。上面只是以 /proc/mi_modules/mi_sys/debug_level 为例。

功能	打印 debug 警告级别
命令	cat /proc/mi_modules/[ModuleName]/debug_level
参数说明	[ModuleName] 模块的名字

	mi_disp mi_gfx mi_rgn mi_vdec mi_vif mi_ai mi_shadow mi_vdisp mi_ao mi_hdmi mi_sys mi_venc
举例	cat /proc/mi_modules/mi_sys/debug_level 2 mi_sys 的 debug 警告级别为 2(显示 warning 和 error 的信息)

功能	修改警告级别
命令	echo [Level] > /proc/mi_modules/[ModuleName]/debug_level
参数说明	[Level] 0 无 Debug 信息 (MI_DBG_NONE) 1 只显示 error 的信息 (MI_DBG_ERR) 2 显示 level<=2 的信息 (MI_DBG_WRN) 3 显示 level<=3 的信息 (MI_DBG_API) 4 显示 level<=4 的信息 (MI_DBG_KMSG) 5 显示 level<=5 的信息 (MI_DBG_INFO) 6 显示 level<=6 的信息 (MI_DBG_DEBUG) 7 显示 level<=7 的信息 (MI_DBG_TRACE) 8 显示所有信息 (MI_DBG_ALL) [ModuleName] 模块的名字 mi_disp mi_gfx mi_rgn mi_vdec mi_vif mi_ai mi_vdisp mi_ao mi_hdmi mi_sys mi_venc
举例	echo 1 > /proc/mi_modules/mi_vdec/debug_level 将 mi_vdec 模块的警告级别修改为只显示 error 的信息

5.9. debug_file

5.9.1 echo

➤ 调试信息

```
# echo mi_sys_impl.c > /proc/mi_modules/mi_sys/debug_file
```

➤ 调试信息分析

用于 debug 时使能文件 mi_sys_impl.c 里的所有 level 等级的 DBG log 信息都可以在串口打印。

➤ 参数说明

参数	描述
命令	echo [fileName] ... > /proc/mi_modules/mi_xxx/debug_file 表示运行时使能打印函数 fileName 里的所有 level 等级的 DBG log 信息，可以同时使能多个函数，函数之间用空格隔开。 echo > /proc/mi_modules/mi_xxx/debug_file 表示关闭之前已打开的函数 fileName 的所有 level 等级的打印功能，此时函数 fileName 里的 DBG log 打印只能按 /proc/mi_modules/mi_xxx/debug_level 设置的等级打印。 注：fileName 必须属于对应的 mi_xxx 模块。
参数说明	[mi_xxx]: 表示 MI 模块名称，可取： mi_disp mi_gfx mi_rgn mi_vdec mi_vif mi_ai mi_shadow mi_vdisp mi_ao mi_hdmi mi_sys mi_venc

5.10. debug_func

5.10.1 echo

➤ 调试信息

```
# echo MI_SYS_IMPL_Init > /proc/mi_modules/mi_sys/debug_func
```

➤ 调试信息分析

用于 debug 时使能函数 MI_SYS_IMPL_Init() 里的所有 level 等级的 DBG log 信息都可以在串口打印。

➤ 参数说明

参数	描述
命令	echo [funcName] ... > /proc/mi_modules/mi_xxx/debug_func 表示运行时使能打印函数 funcName 里的所有 level 等级的 DBG log 信息，可以同时使能多个函数，函数之间用空格隔开。 echo > /proc/mi_modules/mi_xxx/debug_func 表示关闭之前已打开的函数 funcName 的所有 level 等级的打印功能，此时函数 funcName 里的 DBG log 打印只能按 /proc/mi_modules/mi_xxx/debug_level 设置的等级打印。 注：funcName 必须属于对应的 mi_xxx 模块。
参数说明	[mi_xxx]: 表示 MI 模块名称，可取： mi_disp mi_gfx mi_rgn mi_vdec mi_vif mi_ai mi_shadow mi_vdisp mi_ao mi_venc mi_hdmi mi_sys

5.11. module_version_file

5.11.1 cat

➤ 调试信息

```
# cat /proc/mi_modules/mi_ai/module_version_file
```

```
Sigmastar Module mi_ai version: project_commit.81f869a sdk_commit.513c3cc
build_time.20211029104555
```

```
# cat /proc/mi_modules/mi_global_info
```

```
.....
```

```
Sigmastar Module mi_sys version: project_commit.81f869a sdk_commit.513c3cc
build_time.20211029124046
```

```
.....
```

➤ 调试信息分析

/proc/mi_modules/mi_modulename/module_version_file 提供了份 version 信息， /proc/

mi_modules/mi_global_info 里我们也提供了份 version 信息, 不过其本质上指的是 mi_sys 模块的 version 信息, 上面只是以 mi_ai 和 mi_global_info 为例子, 每个模块都有自己对应的 version file。

➤ 参数说明

参数		描述
Version Info	project_commit	模块编译 ko 时整包的 project commit 信息, 如果单独替换 ko 时, 模块基于的 commit 有变化, 那么该 ko 的 version 里得到的 commit 也会跟着变更
	sdk_commit	模块编译 ko 时整包的 sdk commit 信息, 如果单独替换 ko 时, 模块基于的 commit 有变化, 那么该 ko 的 version 里得到的 commit 也会跟着变更
	build_time	时间指的是实际的 build 时间, 精确到秒, 即使 make clean;make image 整体来 build 的话, 各个 ko 的时间也会有差别。

5.12. show threads

5.12.1 echo

➤ 调试信息

echo show_threads > /proc/mi_modules/mi_sys/mi_sys0

```
[<c028099f>] (__schedule) from [<c0280a8f>] (schedule+0x57/0x64)
[<c0280a8f>] (schedule) from [<c001e39d>] (do_wait+0xf1/0x128)
[<c001e39d>] (do_wait) from [<c001e643>] (Sys_wait4+0x47/0x88)
[<c001e643>] (Sys_wait4) from [<c000d261>] (ret_fast_syscall+0x1/0x4c)
[<c028099f>] (__schedule) from [<c0280a8f>] (schedule+0x57/0x64)
[<c0280a8f>] (schedule) from [<c002c647>] (kthreadd+0x65/0xfa)
[<c002c647>] (kthreadd) from [<c000d331>] (ret_from_fork+0x11/0x20)
[<c028099f>] (__schedule) from [<c0280a8f>] (schedule+0x57/0x64)
[<c0280a8f>] (schedule) from [<c002e161>] (smpboot_thread_fn+0xd9/0x144)
[<c002e161>] (smpboot_thread_fn) from [<c002bf77>] (kthread+0xc3/0xd4)
[<c002bf77>] (kthread) from [<c000d331>] (ret_from_fork+0x11/0x20)
[<c028099f>] (__schedule) from [<c0280a8f>] (schedule+0x57/0x64)
[<c0280a8f>] (schedule) from [<c0029487>] (worker_thread+0x23f/0x298)
[<c0029487>] (worker_thread) from [<c002bf77>] (kthread+0xc3/0xd4)
[<c002bf77>] (kthread) from [<c000d331>] (ret_from_fork+0x11/0x20)
[<c028099f>] (__schedule) from [<c0280a8f>] (schedule+0x57/0x64)
[<c0280a8f>] (schedule) from [<c0029487>] (worker_thread+0x23f/0x298)
[<c0029487>] (worker_thread) from [<c002bf77>] (kthread+0xc3/0xd4)
[<c002bf77>] (kthread) from [<c000d331>] (ret_from_fork+0x11/0x20)
[<c028099f>] (__schedule) from [<c0280a8f>] (schedule+0x57/0x64)
[<c0280a8f>] (schedule) from [<c02830e9>] (schedule_timeout+0x137/0x16a)
[<c02830e9>] (schedule_timeout) from [<c0049ad3>] (rcu_gp_kthread+0x4af/0x548)
[<c0049ad3>] (rcu_gp_kthread) from [<c002bf77>] (kthread+0xc3/0xd4)
```

➤ 调试信息分析

查看当前所有线程的 **call stack** 状态。

— 该命令可以帮组查询&定位死锁问题。

— 该命令打印较多，最好放在 **kmsg** 中打印，以免被其他信息污染。

5.13. debug_frc

5.13.1 echo

➤ 调试信息

```
# echo debug_frc on 12 0 0 0 0 "out" > /proc/mi_modules/mi_sys/mi_sys0
```

➤ 调试信息分析

printk in _MI_SYS_IMPL_Common_WriteProc,

Switch enable debug FRC, Modid:12, Dev:0, Chn:0, Pass:0, Port:0, BufType:1, 使能帧率控制 debug 开关

➤ 参数说明

参数	描述
命令	Switch enable debug FRC: <pre>echo debug_frc on [Modid] [Devid] [Chnid] [Passid] [Portid] [in/out]</pre> <pre>> /proc/mi_modules/mi_sys/mi_sys0</pre> 打开帧率控制 debug 开关 Switch disable debug FRC: <pre>echo debug_frc off > /proc/mi_modules/mi_sys/mi_sys0</pre> 关闭帧率控制 debug 开关
参数说明	[Modid] : module id。 [Devid] : device id。 [Chnid] : channel id。 [Passid] : pass id。 [Portid] : port id。 [in/out] : input/output 方向。

5.14. set_outputport_depth

5.14.1 echo

➤ 调试信息

```
# echo set_outputport_depth 6 0 0 0 2 4 > /proc/mi_modules/mi_sys/mi_sys0
```

➤ 参数说明

参数	描述
命令	Set output port depth:echo set_outputport_depth [Modid] [Devid] [Chnid] [Passid] [Portid] [u32userFrameDepth] [u32BufQueueDepth] > /proc/mi_modules/mi_sys/mi_sys0
参数说明	[Modid] : module id。 [Devid] : device id。 [Chnid] : channel id。 [Passid] : pass id。 [Portid] : port id。 [u32userFrameDepth]: 设置该 output 用户可以拿到的 buf 最大数量 [u32BufQueueDepth]: 设置该 output 系统 buf 最大数量

5.15. set_thread_priority

5.15.1 echo

➤ 调试信息

```
#echo set_thread_priority 8 849 99 > /proc/mi_modules/mi_sys/mi_sys0
```

➤ 参数说明

参数	描述
命令	set_thread_priority by pid: echo set_thread_priority [pid] [u32Priority] > /proc/mi_modules/mi_sys/mi_sys0

参数说明	[pid] : 线程 pid [u32Priority] : 需要设定的线程等级。
------	--

5.16. mmu debug

5.16.1 echo

➤ 调试信息

```
# echo debug_mmu debug_log 1 1 1 33,34 > /proc/mi_modules/mi_sys/mi_sys0
```

➤ 参数说明

参数	描述
命令	Enable/disable alloc/free buf log: echo debug_mmu debug_log [enable/disable] [enable/disable free] [enable/disable alloc] [module id list] > /proc/mi_modules/mi_sys/mi_sys0
参数说明	[enable/disable] :enable or disable。 [module id list]: module list, Separated by commas

➤ 举例

- 1.echo debug_mmu debug_log 1 --- enable alloc/free buf log
- 2.echo debug_mmu debug_log 1 0 --- enable free buf log
- 3.echo debug_mmu debug_log 1 1 0 --- enable alloc buf log
- 4.echo debug_mmu debug_log 1 1 1 33,34 --- enable ISP & SCL alloc/free buf log
- 5.echo debug_mmu debug_log 1 0 1 33,34 --- enable ISP & SCL free buf log
- 6.echo debug_mmu debug_log 1 1 0 33,34 --- enable ISP & SCL alloc buf log

5.16.2 insmod

➤ 调试信息

```
# insmod /config/modules/4.9.227/mi_sys.ko bDebugMmu=1
```

➤ 参数说明

参数	描述
----	----

命令	For debug memory access out of bounds: insmod /config/modules/4.9.227/mi_sys.ko bDebugMmu=1/bDebugMmu=0 # Depends on enable mmu
参数说明	bDebugMmu=1 :enable bDebugMmu=0 :disable default disable

5.17. set_inputport_pattern

5.17.1 echo

➤ 调试信息

```
# echo set_inputport_pattern 33 0 0 0 0 1 > /proc/mi_modules/mi_sys/mi_sys0
```

➤ 调试信息分析

设置使能 ModId33 DevId0 ChnId0 PassId0 InputPortId0，此时 pipeline 中 ModId33 模块只会保留有一张 buf 数据向后级模块传递数据（ModId33 与后级模块的 bindtype 需要设置为 frame mode），用于 debug 定位是哪个 ModId 的 buf 数据有问题。

➤ 参数说明

参数	描述
命令	echo set_inputport_pattern [Modid] [Devid] [ChnId] [PassId] [InputPortId] [bEnable] > /proc/mi_modules/mi_sys/mi_sys0 设置 ModId 的某个 inputPort 只保留一张 buf 数据，来向后级 ModId 传递数据（该 ModId 与后级 ModId 的 bindType 需要设置为 frame mode），用于 debug 定位是哪个 ModId 的 buf 数据有问题。
参数说明	[Modid] : module id. [Devid] : device id. [ChnId] : channel id. [PassId] : pass id. [inputPortId] : inputPort Id.

[bEnable] : 是否使能, 0/1.

5.18. mi_sys0

5.18.1 cat

➤ 调试信息

cat /proc/mi_modules/mi_sys/mi_sys0

bindPeerInputPortList																
Chnid	PassId	PortId	Enable	bind_module_id	bind_module_name	bind_Devid	bind_Chnid	bind_PortId	bind_Type	bind_Param	enable					
-----module: mi_vif devid:0-----																
chn	pre	suc	busy	drop	Enq	run	done	retry	drop	bar	checkin	run	done	retry	drop	drop
0	472	8ea	0	0	46d	0	0	0	0	0	0	0	0	0	0	1
-----module: mi_isp devid:0-----																
chn	pre	suc	busy	drop	Enq	run	done	retry	drop	bar	checkin	run	done	retry	drop	drop
0	46a	0	0	0	46a	0	0	0	0	46a	0	469	0	0	0	a
-----module: mi_scl devid:0-----																
chn	pre	suc	busy	drop	Enq	run	done	retry	drop	bar	checkin	run	done	retry	drop	drop
0	46a	0	0	0	46a	0	0	0	0	46a	0	469	0	0	0	a
-----module: mi_venc devid:0-----																
chn	pre	suc	busy	drop	Enq	run	done	retry	drop	bar	checkin	run	done	retry	drop	drop
0	45f	0	0	0	45c	3	0	0	0	3	0	3	0	0	0	0
-----module: mi_venc devid:8-----																
chn	pre	suc	busy	drop	Enq	run	done	retry	drop	bar	checkin	run	done	retry	drop	drop
31	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

➤ 调试信息分析

该命令显示了当前设备 pipeline 时, 各 modId 模块数据流 buf 处理的相关信息。

➤ 参数说明

参数	描述
(mod-dev-chn info)	module devId chn
pre (咨询当前 modId 是否可以 处理 buf 的 handleinfo)	suc busy drop
Enq (将 buf 添加到当前 modId 的队列中的 handleinfo)	run done retry

参数		描述
		的次数
	drop	当前 modId 将 queue 中待处理的 buf drop 掉的次数
	bar	cmdq 相关
checkin (inputport buf handleinfo)	run	当前 modId 处理 inputport buf 操作返回处于 running 状态的次数
	done	当前 modId 已处理完成 inputport 里的 buf 个数
	retry	当前 modId 重新尝试处理 inputport buf 的系数
	drop	当前 modId 将待处理的 inputport buf drop 掉的次数
checkout (outputport buf handleinfo)	run	当前 modId 处理 outputport buf 操作返回处于 running 状态的次数
	done	当前 modId 已处理完成 outputport 里 buf 的次数
	retry	当前 modId 重新尝试处理 outputport buf 的次数
	drop	当前 modId 将待处理的 outputport buf drop 掉的次数

5.19. miu_protect

5.19.1 cat

➤ 调试信息

```
# cat /proc/mi_modules/mi_sys_mma/miu_protect
```

```
===== start miu_protect_info =====
```

```
LX :
```

```
cpu_start_addr:0x20000000 size:0x3ffe0000
```

miu_index miuBlockIndex start_cpu_bus_pa length

	miu_index	miuBlockIndex	start_cpu_bus_pa	end_cpu_bus_pa	length
Hex:	0	11	812b2000	816ac000	3fa000

MmuProtect IP white list:

clientId	name
0x70	CPU_W
0x73	CPU_R
0x20	VCODEC_R
0x21	VCODEC_W

	miu_index	miuBlockIndex	start_cpu_bus_pa	end_cpu_bus_pa	length
Hex:	0	10	180000000	180300000	300000

MmuProtect IP white list:

clientId	name
0x05	CMDQ0_R
0x06	CMDQ1_R
0x70	CPU_W
0x73	CPU_R

	miu_index	miuBlockIndex	start_cpu_bus_pa	end_cpu_bus_pa	length
Hex:	0	0	20000000	40000000	20000000

KernelProtect IP white list:

clientId	name
0x70	CPU_W
0x0f	MCU51_RW

0x12	USB20_H_RW
0x11	USB20_RW
0x10	USB30_RW
0x17	EMAC_RW
0x18	EMAC1_RW
0x13	SD30_RW
0x14	SDIO30_RW
0x04	AESDMA_RW
0x07	URDMA_RW
0x02	BDMA_RW
0x03	MOVDMA0_RW
0x16	SATA_RW

	miu_index	miuBlockIndex	start_cpu_bus_pa	end_cpu_bus_pa	length
Hex:	0	1	5f000000	5ffe0000	fe0000

KernelProtect IP white list:

clientId	name
0x70	CPU_W
0x0f	MCU51_RW
0x12	USB20_H_RW
0x11	USB20_RW
0x10	USB30_RW
0x17	EMAC_RW
0x18	EMAC1_RW
0x13	SD30_RW
0x14	SDIO30_RW
0x04	AESDMA_RW
0x07	URDMA_RW
0x02	BDMA_RW
0x03	MOVDMA0_RW

0x16

SATA_RW

➤ 调试信息分析

该命令显示了 miu protect 相关的信息。

➤ 参数分析

参数	描述
LX(以只有一个 LX 为例,LX 表示 linux 镜像对应的 memory)	cpu_start_addr 该 LX 对应的起始 CPU addr。
	size 该 LX 对应的 size。
某个 kernel protect 的 range 的相关信息	miu_index 编号。
	miuBlockIndex miu 个数范围的编号信息。
	start_cpu_bus_pa 该 range 的起始 cpu bus addr。
	length 该 range 的 length。
KernelProtect IP white list	clientId Miu protect 的 IP 白名单里的 IP 的 id (从未分 group 的角度看的全局 id)。
	name 与 clientId 对应的该 IP 的实际的名字。

5.20. mma_heap_name0

5.20.1 cat

➤ 调试信息

```
# cat /proc/mi_modules/mi_sys_mma/mma_heap_name0
```

```
heap_info:
```

```
heap_name    pa_start    length    avail
mma_heap_name0 40000000 1f000000 1b238000
```

```
heap_param:
```

```
mmuEnable    addsize_before    addsize_after    delayUnmap    freeEntryNum
1            0            0            0            361
```

```
main_chunk_mgr:
```

```
vpa_start    length    avail    used    HighPeak    LowPeak
60000000    20000000 1c238000 3dc8000 40c5000    d40000
```

offset	length	used_flag	task_name	pid	Module
0	300000	1	CMDMEM	-1	mi_sys
300000	1000	1	sys-logConfig	-1	mi_sys
301000	40000	1	sys-logBuffer	-1	mi_sys
341000	1fb000	1	vif0-out0-0	867	mi_vif
53c000	1fb000	1	vif0-out0-0	867	mi_vif
737000	1fb000	1	vif0-out0-0	867	mi_vif
932000	3fa000	1	mma_heap_codec	-1	mi_sys
d2c000	21000	1	ven_mv_0	867	mi_venc
d4d000	23000	1	ven_fcty_0	867	mi_venc
d70000	12000	1	ven_fctc_0	867	mi_venc
d82000	41000	1	ven_rdo_0	867	mi_venc
dc3000	1000	1	ven_inst_0	867	mi_venc
dc4000	1000	1	venc-algo-m	867	mi_venc
dc5000	60000	0	NA	-1	mi_sys
e25000	a3000	1	ISP_STATIS	867	mi_isp
ec8000	1000	1	ISP_WDR	867	mi_isp
ec9000	22000	1	DNR_INFO1	867	mi_isp
eeb000	32a000	1	DNR_INFO0	867	mi_isp
1215000	6000	1	ISP_IDX_CMDQ	867	mi_isp
121b000	10000	1	ISP_CMDQ	867	mi_isp
122b000	4000	1	ISP_MLOAD	867	mi_isp
122f000	152000	1	RingHeapAlloc	-1	mi_sys
1381000	1a6000	0	NA	-1	mi_sys
1527000	4000	1	ISP_MLOAD	867	mi_isp
152b000	3f5000	1	venc-ring-mem0	867	mi_venc
1920000	3b1000	1	ven_rec_0_0	867	mi_venc
1cd1000	2fd000	0	NA	-1	mi_sys
1fce000	2000000	1	venc-ring-mem1	867	mi_venc
3fce000	2fd000	1	scl0-out0-0	867	mi_scl
42cb000	1bd35000	0	NA	-1	mi_sys

(mma_heap_codec)sub_chunk_mgr:

vpa_start	length	avail	used	HighPeak	LowPeak
60932000	3fa000	2df000	11b000	11b000	0

offset	length	used_flag	task_name	pid	Module
0	ea000	1	vcodec_dev	-1	mi_venc
ea000	31000	1	ven_work_ven_ta	-1	mi_venc
11b000	2df000	0	NA	-1	mi_sys

(CMDMEM)sub_chunk_mgr:

vpa_start	length	avail
60000000	300000	195000

offset	length	used_flag	cmdq_id	pid	ModDev
0	20000	0	-1	-1	NA
20000	16b000	1	1	0	mi_isp0
20000	16b000	1	1	0	mi_scl0
20000	16b000	1	1	0	mi_venc8
18b000	175000	0	-1	-1	NA

➤ 调试信息分析

该 cat 信息对应的是该 mma heap 的基本信息和当前的一些状态。

➤ 参数分析

参数		描述
heap_info	heap_name	mma heap 的 name。
	pa_start	heap 的起始 PA。
	length	mma heap 的 length。

参数		描述
heap_param	avail	heap 里剩余未用的 memory 总量。
	mmuEnable	表示是否启用 HW_MMU，1 为启用，0 为不启用。
	addrsz_before	设置在申请的 buf 前多申请预留的 size 大小
	addrsz_after	设置在申请的 buf 后多申请预留的 size 大小
	delayUnmap	buf free 时是否开启延时 unmap 当前申请的 buf，当下次再被申请时，先 unmap 再 map 1: 开启延时 unmap 0: 关闭延时 unmap
main_chunk_mgr	freeEntryNum	空闲的 entry 的个数
	vpa_start	main chunk mgr 的起始地址，如果开启 HW_MMU，该地址为 vpa，否则为 pa（对应名字会显示 pa_start）。main chunk mgr 涵盖了整个 mma heap 的大小。
	length	chunk mgr 管理的整个 mma vpa length，由于整个 mma heap 做为一个 chunk mgr，因此当关闭 mmu 时，该值等于 mma heap length；当开启 mmu 时，该值大于 mma heap length。
	avail	chunk mgr(即 heap)里的剩余的未用的 memory 的总量。
	used	chunk mgr(即 heap)里已使用的 memory 的总量。
	HighPeak	统计的一段时间内的内存峰值
	LowPeak	统计的一段时间内的内存谷值
	offset	该 chunk 在 chunk mgr 里的 offset。
	length	该 chunk 的 length。

参数		描述
	used_flag	该 chunk 是否被 alloc 出去使用了。是的话该值为 1；否则 free 状态的话该值为 0。
	task_name	如果 used flag 为 1，则 task_name 里存的是哪个 task 使用的它；否则该值为无效值 NA。
	pid	暂时无效
	Module	该 chunk 被哪个 Module 使用了
(mma_heap_codec)sub_chunk_mgr	(mma_heap_codec)sub_chunk_mgr	属于 main chunk mgr 的子 chunk mgr，对应 task name 里的 mma_heap_codec，显示的参数表示的意思请参考 main chunk mgr。 sub chunk mgr 默认不会打印，需要下命令才会打印。下命令后，有可能没有 sub chunk mgr，或者会有不止一个 sub chunk mgr 打印出来。 需要下的命令为 echo show_privMma 1 > /proc/mi_modules/mi_sys/mi_sys0
(CMDMEM)sub_chunk_mgr	(CMDMEM)sub_chunk_mgr	属于 main chunk mgr 的子 chunk mgr，对应 task name 里的 CMDMEM。此 sub chunk mgr 会默认打印，表示 cmdq 使用 mma buf 的情况。
	vpa_start	(CMDMEM)sub chunk mgr 的起始地址，如果开启 HW_MMU，该地址为 vpa，否则为 pa（对应名字会显示 pa_start）。
	length	分配给 cmdq 的 mma buf 大小。
	avail	cmdq 剩余可使用的 mma buf 大小。
	offset	该 cmdq 申请的 buf 在名为 CMDMEM mma buf 中的起始 offset。
	length	该 cmdq 在名为 CMDMEM mma buf 中申请的 mma buf 大小。

参数		描述
	used_flag	该 buf 是否被 alloc 出去使用了。是的话该值为 1；否则 free 状态的话该值为 0。
	cmdq_id	使用 buf 对应 cmdq 的 id。
	pid	暂时无效。
	ModDev	申请 cmdq buf 的 module 以及对应的 device id。

5.20.2 echo

功能	即 dump 指定 offset 和 length 的 data 到指定的路径
命令	echo [Path] [Offset] [Length] > /proc/mi_modules/mi_sys_mma/mma_heap_name[Num]
参数说明	[Path] 文件要保存的路径,只需提供路径, 不要提供具体的文件名, 系统会根据参数自动生成对应的文件名。
	[Offset] 从该 mma heap 的哪个 offset 开始 dump data,必须 4KB 对齐
	[Length] 在该 mma heap 里 dump 的总的数据量的大小,必须 4KB 对齐
	[Num] mma head 对应的数字, 目前 mma_heap_name0 可通过 cat /proc/cmdline 查看
举例	echo /mnt/ 0 4096 > /proc/mi_modules/mi_sys_mma/mma_heap_name0 在 /mnt 下产生 mma_mma_heap_name0_0_4096.bin 文件

5.21. imi_mma_heap

5.21.1 cat

➤ 调试信息

```
# cat /proc/mi_modules/mi_sys_mma/imi_mma_heap
mma heap nameheap_base_cpu_bus_addr      length  chunk_mgr_avail
mmuEnable  addsize_before  addsize_after  bMmuDelayUnmap  free_entry_num
imi_mma_heap      20000000      43800      26800      0
0      0      0      0
chunk_mgr info:
offset  length  avail  used  HighPeak  LowPeak
```

0 43800 26800 1d000 1d000 0

each chunk info:

offset	length	used_flag	task_name	pid	Module
0	1d000	1	SRAM	-1	mi_venc
1d000	26800	0	NA	-1	mi_sys

➤ 调试信息分析

该 **cat** 信息对应的是该 **mma heap** 的基本信息和当前的一些状态。

➤ 参数分析

参数		描述
heap basic info	mma heap name	mma heap 的 name。
	heap_base_cpu_bus_addr	该 allocator 里的 allocations 中未被使用的数目。
	length	heap 的 length。
	chunk_mgr_avail	heap 里的剩余的未用的 memory 的总量
	mmuEnable	Mma heap 是否支持 HW_MMU。该值为 1 表示支持，为 0 表示不支持。
	addrsz_before	设置在申请的 buf 前多申请预留的 size 大小
	addrsz_after	设置在申请的 buf 后多申请预留的 size 大小
	bMmuDelayUnmap	buf free 时是否开启延时 unmap 当前申请的 buf，当下次再被申请时，先 unmap 再 map 1: 开启延时 unmap 0: 关闭延时 unmap
chunk_mgr info	free_entry_num	空闲的 entry 的个数
	offset	chunk mgr 的 offset,由于整个 mma heap 做为一个 chunk mgr,因此该值永

参数		描述
		远为 0。allocator 里的逻辑 offset。
	length	chunk mgr 的 length,由于整个 mma heap 做为一个 chunk mgr,因此该值永远为 mma heap length。
	avail	chunk mgr (即 heap)里空闲的 memory 的总量。
	used	chunk mgr(即 heap)里已使用的 memory 的总量。
	HighPeak	统计的一段时间内的内存峰值
	LowPeak	统计的一段时间内的内存谷值
each chunk info (chunk mgr 里各 chunk 的信息和使用情况)	offset	该 chunk 在 chunk mgr 里的 offset。
	length	该 chunk 的 length。
	used_flag	该 chunk 是否被 alloc 出去使用了。是的话该值为 1；否则 free 状态的话该值为 0。
	task_name	如果 used flag 为 1，则 task_name 里存的是哪个 task 使用的它；否则该值为无效值 NA。
	pid	该 chunk 所属的 current->tgid。
	Module	该 chunk 所属的模块

5.22. meta

5.22.1 cat

➤ 调试信息

```
# cat /proc/mi_modules/mi_sys_mma/meta
```

```
===== start meta_info =====
```

```
basic info:
```

```
total page count:16      each meta data size:256  total_count_with_metadatasize 256
```

```
total_free_count_with_metadatasize:254
```

meta info :

ref_cnt=3

each allocation info:

offset_in_pool	length	phy_addr	va_in_kern	real_used_flag
0x0	0x100	0x2336000	c1736000	0
0x100	0x100	0x2336100	c1736100	0
0x200	0x100	0x2336200	c1736200	0
0x300	0x100	0x2336300	c1736300	0
0x400	0x100	0x2336400	c1736400	0
0x500	0x100	0x2336500	c1736500	0
0x600	0x100	0x2336600	c1736600	0
0x700	0x100	0x2336700	c1736700	0
0x800	0x100	0x2336800	c1736800	0
0x900	0x100	0x2336900	c1736900	0
0xa00	0x100	0x2336a00	c1736a00	0

.....

===== end meta_info =====

➤ 调试信息分析

只要一旦有人用过了 meta allocator(有且只有一个 meta allocator),那么就会生成/proc/mi_modules/mi_sys_mma/meta 文件,且其对应的 memory 就会完全 alloc 了出来,等待被使用。

5.22.2 echo

功能	即 dump 指定 offset 和 length 的 data 到指定的路径
命令	echo [Path] [Offset] [Length] > /proc/mi_modules/mi_sys_mma/vb_pool_global
参数说明	[Path] 文件要保存的路径,只需提供路径,不要提供具体的文件名,系统会根据参数自动生成对应的文件名。
	[Offset] 从该 allocator 的哪个 offset 开始 dump data,是 allocator 的逻辑 offset。。
	在该 allocator 里 dump 的总的数据量的大小。

举例	echo /mt 0 1024 > /proc/mi_modules/mi_sys_mma/meta 在 /mnt 下产生 meta__0__1024.bin
----	--

5.23. mi_modulenameid

5.23.1 cat

➤ 调试信息

cat /proc/mi_modules/mi_scl/mi_scl0

```
/ # cat /proc/mi_modules/mi_scl/mi_scl0

-----Common info for mi_scl-----
ChnNum  EnChnNum  PassNum  InPortNum  OutPortNum  CollectSize
    34         1         1         1           6           0

-----CMDQ kickoff counter-----
PassId   CmdqId   TotalKickoff  KickoffFence  Idle
    0         1       2594         1732         1
DevId   current_buf_size  Peak_buf_size
    0           0           0

each dev buf info:
      offset          length      used_flag      task_name

-----Common info formi_scl only dump enabled chn-----
ChnId PassNum  EnInPNum  EnOutPNum  MMAHeapName
    0         0         1         1      (null)
ChnId   current_buf_size  Peak_buf_size  user_pid
    0           8f7000         bf4000         849

each chn buf info:
      offset          length      used_flag      task_name
    9c9000         2fd000         1      scl0-out0-0
    1ee9000        2fd000         1      scl0-out0-0
    21e6000        2fd000         1      scl0-out0-0

-----chn0 pass0 pipeline delay in us-----
      Flow      Min      Max      Avg      Diff
OnPreProcessInputTask      92      44167      33103      33674
  EnqueueInputTask      225      44319      33302      33833
  BarrierInputTask      261      196698      33591      34024
  CheckInputTaskStatus      31720      246445      33634      33393
  DequeueInputTask      31723      246449      33637      33396
OnPollingAsyncOutputTaskConfig      -1         0         0         0
  EnqueueAsyncOutputTask      -1         0         0         0
  BarrierAsyncOutputTask      -1         0         0         0
  CheckOutputTaskStatus      31714      246461      33624      33383
  DequeueOutputTask      31726      246473      33637      33396
  FinDMADispatch      33419      42380      33553      33540
```

➤ 调试信息分析

cat 操作的结果是分为 2 部分的，第一部分是 MI_SYS 提供的针对该 device 的通用的调试信息，从 device、channel、input port、output port 四个角度提供；第二部分是该 device 私有的信息。私有的信息各 module 的内容别处会阐述，这里 SYS 角度只阐述通用的信息。以 cat /proc/mi_modules/mi_scl/mi_scl0 为例，得到上面的结果。

➤ 参数说明

参数		描述
Common info for device	ChnNum	该 device 的总的 channel 的数目。
	EnChnNum	该 device 的 enabled 了的 channel 的数目。
	PassNum	该 device 的 pass 的数目。
	InPortNum	该 device 的 input port 的数目。
	OutPortNum	该 device 的 output port 的数目。
	CollectSize	该 dev 对应的 AllocatorCollection 含有的 allocator 的数目总和。
CMDQ kickoff counter	PassId	该 device 使用到 cmdq 的 pass id。
	CmdqId	使用的 cmdq 的 id。
	TotalKickoff	该 cmdq 当前 kickoff 的次数。
	kickoffFence	该 cmdq 当前 kickoff 时的 fence 值。
	Idle	该 cmdq 当前 kickoff 时的 Idle 次数。
	DevId	该 mi module 的 device id。
	current_buf_size	该 device 当前使用的 buf 大小。
	Peak_buf_size	该 device 使用 buf 的峰值。
each dev buf info	offset	Buf 开始的 offset。
	length	Buf 的长度。
	used_flag	记录 buf 是否使用的 flag。
	task_name	使用这块 buf 的 task name。
Common info for channel(only dump enabled channel)	ChnId	Channel 的 id 值。
	PassNum	Pass 的 id 值。
	EnInPNum	该 channel 的 enabled input port 数量。
	EnOutPNum	该 channel 的 enabled output port 数量。
	MMAHeapName	如果有 SetChnMMAConf 的话, 该值为对应的 mma heap name;如果没有设置的话, 该值为 NULL。
	current_buf_size	该 channel 当前使用的 buf size。
	Peak_buf_size	该 channel 使用 buf 的目前峰值。
	user_pid	该 channel 对应的 pid。

参数		描述
each chn buf info	offset	该 channel 使用 buf 的起始 offset。
	length	Buf 的长度。
	used_flag	记录 buf 是否使用的 flag。
	task_name	该 buf 对应的 task name。
chn pass pipeline delay in us	Flow	流程点： OnPreProcessInputTask: MI PASS 判断是否需要处理当前 task，如果可以给 output buffer 配置； EnqueueInputTask: 将 input task 交由 MI PASS 开始处理； BarrierInputTask: 插入 CMDQ 同步指令（比如 waitevent， polling engine done）； CheckInputTaskStatus: 检查 input task 是否完成，及完成状态（成功，失败，etc）； DequeueInputTask: 通知 MI PASS 释放 input task 相关联的资源； OnPollingAsyncOutputTaskConfig: 对于输入输出不同步的 MI Pass，或者只有输出没有输入的 MI Pass，MI SYS 查询是否有独立 output buf request 需求； EnqueueAsyncOutputTask: 将非同步输出的 output task 交由 MI PASS 做处理； BarrierAsyncOutputTask: 对非同步 output task 插入 cmdq 同步指令； CheckOutputTaskStatus: 检查 output task 是否完成，及完成状态（成功，失败，etc）； DequeueOutputTask: 通知 MI PASS 释放 output task 相关联的资源；

参数		描述
		FinDMADispatch: MI SYS 将 MI PASS 的 output task 中的 buffer 传递到绑定的下级 MI PASS 的 input queue。
	Min	Buffer 产生到该 MI PASS 对应 Flow 所经过的最小延时。
	Max	Buffer 产生到该 MI PASS 对应 Flow 所经过的最大延时。
	Avg	Buffer 产生到该 MI PASS 对应 Flow 所经过的平均延时。
	Diff	Buffer 产生到该 MI PASS 对应 Flow 所经过的当前延时。
Input port common info(only dump enabled Input port)	ChnId	该 input port 所在的 channel id。
	PassId	
	PortId	该 input port 的 id。
	user_buf_quota	该 InputPort 的 buff 数目的 Quota。
	UsrInjectQ_cnt	该 InputPort 里的 UsrInjectBufQueue(usr inject 进来还没处理完成的 buf 队列)里的 buff 数目。
	BindInQ_cnt	该 InputPort 里的 BindInputBufQueue(由绑定前级 enq 进来还没处理完成的 buf 队列)里的 buff 数目。
	TotalPendingBuf_size	该 inputPort 的 cur_working_input_queue(还没开始处理, 即将处理)里面各个 buf 的 size 总和。
	usrLockedInjectCnt	用户当前拿到了多少个 buf。
	newPulseQ_cnt	该 inputPort 在 new_pulse_fifo_inputqueue(input port buf 缓存队列 0)的 buf 数目。
	nextTodoPulseQ_cnt	该 inputPort 在

参数		描述
		new_todo_pulse_inputqueue(input port buf 缓存队列 1)的 buf 数目。
	curWorkingQ_cnt	该 inputPort 在 cur_working_input_queue(还没开始处理, 即将处理)的 buf 数目。
	workingTask_cnt	该 inputPort 在 input_working_tasklist(正在处理的 input task 的列表)的 task 数目。
	lazyRewindTask_cnt	该 inputPort 在 lazy_rewind_inputtask_list(即将回退到 cur_working_input_queue 队列中 task 列表)的 task 数目。
	Enable	标记该 input port 是否 enable。
	bind_module_id	与该 input port 进行了 binded 的 output port 所在的 module 的 id。
	bind_module_name	与该 input port 进行了 binded 的 output port 所在的 module 的 name。
	bind_DevId	与该 input port 进行了 binded 的 output port 所在的 device 的 id。
	bind_ChnId	与该 input port 进行了 binded 的 output port 所在的 channel 的 id。
	bind_PortId	与该 input port 进行了 binded 的 output port 的 id。
	bind_Type	绑定模式。
	bind_Param	该 input port 进行 bind 时所带的参数。
	enable	标记绑定的 output port 是否 enable。
	SrcFrmrate	Src 帧率。
	DstFrmrate	Dst 帧率。
	GetFrame/Ms	统计的帧数/统计所耗的时间。
	FPS	实际帧率。
	FinishCnt	该 Input port finish buf 的个数。

参数		描述
	RewindCnt	该 Input port rewind buf 的个数。
	DropCnt	该 Input port drop buf 的个数。
Output port common info (only dump enabled Output port)	ChnId	该 output port 所在的 channel 的 id。
	PassId	该 output port 所在的 pass 的 id。
	PortId	该 output port 的 id。
	usrDepth	User 最多可以拿到的 output buf 数量。
	BufCntQuota	该 OutputPort 的 buff 数目的 Quota。
	usrLockedCnt	从 UsrGetFifoBufQueue(用户获取 output port buf 的队列)里用户实际拿走了多少个 buffer。
	totalOutPortInUsed	totalOutputPortInUsedBuf 数目。
	DrvBkRefFifoQ_cnt	该 OutputPort 的 DrvBkRefFifoQueue(驱动预处理队列, 目前没用到)里的 buffer 数目。
	DrvBkRefFifoQ_size	该 OutputPort 的 DrvBkRefFifoQueue(驱动预处理队列, 目前没用到)里的 buffer 的总 size。
	UsrGetFifoQ_cnt	该 OutputPort 的 UsrGetFifoBufQueue(用户获取 output port buf 的队列)里的 buffer 数目。
	UsrGetFifoQ_size	该 OutputPort 的 UsrGetFifoBufQueue(用户获取 output port buf 的队列)里的 buffer 的总 size。
	UsrGetFifoQ_seqnum	被加到 UsrGetFifoBufQueue(用户获取 output port buf 的队列)的 buf 总个数。
	UsrGetFifoQ_discard num	由于 UsrGetFifoBufQueue(用户获取 output port buf 的队列)满了而被 discard 的 buf 个数。
	workingTask_cnt	当前在 output_working_tasklist(正在处理的 output task 的列表)的 task 总个数。

参数		描述
	finishedTask_cnt	当前在 output_finished_tasklist(已经处理完成还没推到下一级的列表)的 task 总个数。
	GetFrame/Ms	统计的帧数/统计所耗的时间。
	FPS	实际帧率。
	FinishCnt	该 output port finish buf 的总个数。
	RewindCnt	该 output port rewind buf 的总个数。
	GetTotalCnt	向 mi_sys 申请 output buf 的总次数。
	GetOkCnt	向 mi_sys 申请 output buf 成功的次数。
BindPeerInputPortList	ChnId	该 output port 所在的 channel 的 id。
	PassId	该 output port 所在的 pass 的 id。
	PortId	该 output port 的 id。
	Enable	标记该 output 是否 enable。
	bind_module_id	与该 output port 进行了 binded 的 input port list 中的其中一个 input port (以下简称该 binded input port) 所在的 module 的 id。
	bind_module_name	该 binded input port 所在的 module 的 name。
	bind_DevId	该 binded input port 所在的 device id。
	bind_ChnId	该 binded input port 所在的 channel id。
	bind_PortId	该 binded input port 的 id。
	bind_Type	绑定模式。
	bind_Param	绑定时所下的参数。
	enable	标记已绑定的 input port 是否 enable。

5.23.2 echo

功能	获得模块的 help 信息
命令	echo help > /proc/mi_modules/[ModName]/[ModID]

参数说明	[ModName] 模块的名字 mi_disp mi_gfx mi_rgn mi_vdec mi_scl mi_ai mi_isp mi_shadow mi_vdisp mi_ao mi_hdmi mi_sys mi_venc mi_vif
	[ModID] 模块中对应的文件一般为模块的名字+数字 如 mi_disp0 mi_gfx0 mi_rgn0
举例	echo help > /proc/mi_modules/mi_disp/mi_disp0 得到 mi_disp0 节点文件的帮助信息

功能	Dump Port 的信息
命令	echo dump_buffer [ChnID] [PortType] [PortID] [QueueName][Path] [EndMethod] > /proc/mi_modules/[ModName]/[ModID]
参数说明	[ChnID] 通道的 ID
	[PortType] iport 或 oport 分别代表 input port output port
	[PortID] 该 port 的 id
	[QueueName] 如果是 input port 的话, 值只能是"UsrInject" 或者"BindInput" 如果是 output port 的话, 值只能是"UsrGetFifo"
	[Path] 数据要保存的文件所在的路径。注意是绝对路径, 不要提供数据要保存的文件的名称, 系统会根据参数自动生成文件名, 不同的 buffer 保存在不同的文件
	[EndMethod] dump buffer 的结束方法, 目前仅支持三类: (a) "bufnum=xxx":dump 指定数目的 buffer (b) "time=xxx"(here unit is ms):dump 过程持续 time 对应的时间 (c) "start/end" pair:用 start 来开始 dump,用 end 来结束对应的 dump,要求执行这 2 个命令时, 其他的 5 个参数完全一样
	[ModName] 模块的名字 mi_disp mi_gfx mi_rgn mi_vdec mi_scl mi_ai mi_isp mi_shadow mi_vdisp mi_ao mi_hdmi mi_sys mi_venc mi_vif
	[ModID] 模块中对应的文件一般为模块的名字+数字
举例	echo dump_buffer 0 iport 0 BindInput /mnt bufnum=1 > /proc/mi_modules/

	mi_disp/mi_disp0
	dump disp 模块 inport 的数据

5.24. mem_stat

5.24.1 cat

➤ 调试信息

```
# cat /proc/mi_modules/mi_sys_mma/mem_stat
```

```
mi_sys
```

```

CMDMEM          c0000
sys-logBuffer    40000
sys-logConfig    1000
TotalSize        101000
AllTotalSize     101000
```

➤ 调试信息分析

打印出 **mi sys** 内存状态

➤ 参数说明

参数		描述
Mem_stat	mi_sys	所属 mi_sys 模块
	CMDMEM	Cmd 的内存大小
	sys-logBuffer	Sys log 最大可申请的 buf 大小
	sys-logConfig	分配（申请）给 sys log 的 buf 大小
	TotalSize	ModuleUsdTotalSize 模块使用的 size
	AllTotalSize	总的 size